

Foundations of Position-Based Rigid Body Dynamics

Tom Waterson *

September 24, 2026

Abstract

Position Based Dynamics (PBD) and its extensions are widely used in real-time physics because they are robust and efficient, but their connection to classical rigid-body mechanics is often left implicit. These notes derive position based rigid body dynamics from constrained Lagrangian mechanics, with particular attention to the treatment of angular degrees of freedom. The resulting update is then compared to sequential impulses, pseudoinverse methods and discrete variational mechanics, and the same framework is used to connect PBD with XPBD, VBD and AVBD. We also look at momentum conservation, energy behaviour, stabilization and the convergence of constraint chains. The aim is to give a clear mathematical explanation of why these commonly used simulation methods work and how they fit together.

*Guildford, UK email:tom.watson@epicgames.com

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Overview	4
	Notation	6
2	Rigid Body Configuration	8
2.1	Angular Velocity	9
2.1.1	The Path Ordered Exponential	10
2.1.2	Axis/Angle Representation and the Exponential Map	11
2.1.3	The Left Jacobian of $SO(3)$	12
2.2	Quaternion Representation*	15
2.2.1	Quaternions from the Path Ordered Exponential	16
2.2.2	Relation to Axis/Angle Representation	16
3	Equations of Motion	18
3.1	The Euler-Poincaré Method	19
3.2	Holonomic Constraints	21
3.3	Velocity-Level and Non-Holonomic Constraints	23
4	Solving the Constrained Equations of Motion	25
4.1	Solving the Unconstrained Equations of Motion	25
4.2	Finding a First Order Approximate Solution	26
4.3	An Iterative Solution Using Gauss-Seidel	28
4.4	Comparison to the Jacobi Method	30
4.5	Convergence	31
4.5.1	Rate of convergence	32
4.6	A More Accurate Update Using Non-Linear Gauss-Seidel	34
4.7	Adding Velocity-Level Constraints	35
4.8	Regularization	35
5	Interpretation and Connections to Other Known Methods	37
5.1	Moore-Penrose Inverse and Least Squares Minimization	37
5.2	Sequential Impulses	38
5.3	Position Based Dynamics	39
5.4	Discrete Variational Mechanics	39
6	Techniques to Improve Convergence and Stability	43
6.1	Relaxation	43
6.2	Warm Starting	44
6.3	Sub-stepping	45
6.4	Baumgarte Stabilization	47
6.5	Split Impulse	48

7	Stiff Constraint Examples	50
7.1	Point to Point Constraint	50
7.1.1	Comparison to an Impulse Based Approach	52
7.2	Symmetries and Conservation of Momentum	52
7.3	Energy Conservation	56
7.4	Inequality Constraints	58
7.5	Contact Constraint	58
7.6	Contact Velocity Constraint	59
7.7	Contact Friction Constraint	61
7.8	Angular Axis Constraint	63
8	Compliant Constraints	66
8.1	Generalized Implicit Non-Linear Forces	66
8.2	Connection to Vertex Block Descent	67
8.3	Connection to the Lagrange Multiplier Method and PBD	67
8.4	Compliant Constraint Forces and XPBD	68
8.4.1	XPBD as a Regularization on PBD	71
8.5	Compliant Constraint Examples	72
8.5.1	Spring Damper	72
8.5.2	Volumetric Constraint	73
9	Geometric Stiffness	75
9.1	The Second Left-Trivialized Derivative	75
9.2	Geometric Stiffness	76
9.3	Point to Point Constraint	78
10	Augmented Lagrangian Method	80
10.1	Augmented Vertex Block Descent	82
10.1.1	AVBD, PBD and Geometric Stiffness	83
10.2	Convergence Comparison vs. PBD For Large Mass Ratios	84
10.3	Point to Point Constraint	84
10.3.1	Exact Solution of the Local System	85
10.3.2	Body-Block Iterative Solution	86
11	Convergence of Constraint Chains	88
11.1	Setup	88
11.2	Effect of Iteration Count	89
11.2.1	Relaxation	90
11.2.2	Jacobi Iterations	91
11.2.3	AVBD Iterations	91
11.3	Effect of Mass Ratios	93
11.3.1	Relaxation	94
11.3.2	AVBD Iterations	95
11.4	Results	96
12	Closing Remarks and Further Reading	98

1 Introduction

1.1 Motivation

When I first started in the games industry in 2006, position-based methods for rigid-body simulation were already known and in use. At Electronic Arts, where I started out, a position-based solver was already being developed, while Müller et al. published the first paper formalising and naming Position Based Dynamics that same year [28]. But these developments themselves had roots in earlier work: EA’s physics technology through MathEngine, while Position Based Dynamics developed from a broader history of constrained dynamics and position-projection methods, including work such as [3, 32, 22] to name a few. These methods have therefore been around for some time, and have continued to evolve through later approaches such as XPBD [25] and, more recently AVBD [18], as well as many developments and improvements in between [29, 26, 24].

While the algorithms of PBD and its variants are well documented, their connection to classical mechanics is rarely presented in a single mathematical framework, particularly for the angular degrees of freedom of rigid bodies. Coming from a theoretical physics background, this was unsatisfying to me and something I felt would be interesting and valuable to go through, at least for my own understanding. To this end, I started to make my own notes on rigid body simulation and, where possible, to make the connection to classical mechanics. The motivation was to provide a solid theoretical grounding for the techniques that are commonly used in industry.

The notes here are an accumulation of that effort on and off over many years. I have used earlier versions to help onboard new team members coming into physics solver development, and I have added to the notes over the years when a new paper came out that I wanted to understand.

The purpose of these notes is not necessarily to introduce novel algorithms or methods. It is to show how these practical approaches can be derived from first principles, how they are related, and where their assumptions and ranges of applicability lie. Along the way there are also some hopefully useful observations on implementation, stability and convergence. For those after a more practical, implementation focused approach to position based dynamics I would recommend [28, 25, 7].

1.2 Overview

The main substance of these notes is the derivation of position based dynamics for rigid bodies using constrained Lagrangian mechanics. In the first section, there is some background on rigid bodies and their configuration. In particular, I try to give a careful treatment of the representation of angular degrees of freedom, and derivatives of the angular configuration variables.

The next section derives the equations of motion. Here I use some results from geometric mechanics that exploit the rotation group symmetry to simplify the calculation. The following section is where the connection to real time simulation is made, and I derive a first order approximation to the constrained equations of motion and show how this can be iteratively solved using Gauss–Seidel, then discuss convergence properties of this algorithm.

From there I show how this approach connects to the algorithms of Position Based Dynamics. I cover PBD, XPBD, Vertex Block Descent (VBD) and Augmented Vertex Block Descent (AVBD), and also show the connection to a Sequential Impulse approach, to the Moore–Penrose pseudo-inverse method, and to Tikhonov regularization. There are also examples of contact and joint constraints

to illustrate the approach, with some discussion about energy and momentum conservation. I then extend the first-order treatment to the constraint Hessian and geometric stiffness before returning to the augmented Lagrangian and AVBD formulations.

In the final section I analyze the convergence of a simple chain of connected rigid bodies under the different solver formulations, to illustrate how their convergence properties differ.

A very good recent paper by Macklin et al. [24] also tries to derive Projective Dynamics and XPBD from a common variational objective, where there they name the primal (body-space) / dual (constraint-space) distinction. Some of these notes precede that work, but I have tried to point out where the sections relate, and to adopt some of their language. In some sense, these notes can be seen as complementary to that work, going a bit deeper on the angular degrees of freedom, and extending the discussion to VBD and AVBD. There is also a more concrete analysis of convergence rates for some simple model problems.

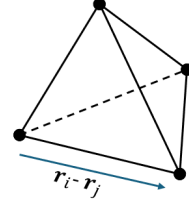
Notation

Symbol	Description
<i>Configuration and kinematics</i>	
$\mathbf{x} \in \mathbb{R}^3$	Position of the centre of mass
$\boldsymbol{\theta} \in \mathbb{R}^3$	Axis-angle rotation vector, $\boldsymbol{\theta} = \theta \hat{\boldsymbol{\theta}}$
$R(\boldsymbol{\theta}) \in SO(3)$	Rotation matrix, $R = e^{[\boldsymbol{\theta}]_{\times}}$
$\mathbf{X} = (x, \boldsymbol{\theta})$	Generalised configuration of a single body
$\mathcal{V} = (\dot{\mathbf{x}}, \boldsymbol{\omega})$	Generalised velocity of a single body
$\boldsymbol{\omega} \in \mathbb{R}^3$	World-space angular velocity
$\boldsymbol{\omega}_b \in \mathbb{R}^3$	Body-space angular velocity, $\boldsymbol{\omega}_b = R^T \boldsymbol{\omega}$
q	Unit quaternion representing orientation
$\boldsymbol{\Omega}$	Angular velocity bivector, $\boldsymbol{\Omega} = 2\dot{q}\bar{q} = -\mathcal{I}\boldsymbol{\omega}$
$\mathbf{p}(\mathbf{X})$	World-space position of a point on the body, $\mathbf{p} = \mathbf{x} + R(\boldsymbol{\theta})\mathbf{r}_0$
$\mathbf{r}_0 \in \mathbb{R}^3$	Fixed body-space offset of a point from the centre of mass
$\mathbf{r}(\theta)$	World-space offset, $\mathbf{r} = R(\boldsymbol{\theta})\mathbf{r}_0$
<i>Lie group structure</i>	
$[\cdot]_{\times}$	Map from $\mathbb{R}^3$ to 3×3 skew-symmetric matrices (hat map)
$SO(3)$	Special orthogonal group of 3×3 rotation matrices
$J_L(\boldsymbol{\theta})$	Left Jacobian of $SO(3)$, maps $\dot{\boldsymbol{\theta}}$ to $\boldsymbol{\omega}$: $\boldsymbol{\omega} = J_L \dot{\boldsymbol{\theta}}$
<i>Mass and inertia</i>	
m	Scalar mass of a body
$I(\theta)$	World-space inertia tensor, $I = R(\boldsymbol{\theta})I_0R(\boldsymbol{\theta})^T$
$I_L(\boldsymbol{\theta})$	World space inertia rotated by the left Jacobian $J_L(\boldsymbol{\theta})^T I(\boldsymbol{\theta}) J_L(\boldsymbol{\theta})$
I_b	Body-space inertia tensor
$M(\mathbf{X})$	Generalised mass matrix for a system of N bodies (block diagonal)
$M_L(\mathbf{X})$	Generalized mass matrix with left Jacobian rotated inertias I_L
μ	Scalar effective mass for a single constraint
$\mathcal{M}$	Effective mass of a constraint
<i>Constraints and forces</i>	
$\mathbf{c}(\mathbf{X})$	Vector of holonomic constraint functions
$\boldsymbol{\lambda}$	Vector of Lagrange multipliers (constraint forces)
$\mathcal{D}\mathbf{c}$	Left-trivialized constraint Jacobian, satisfies $\dot{\mathbf{c}} = \mathcal{D}\mathbf{c}\mathcal{V}$
$\mathcal{D}_{\mathcal{V}}\mathbf{c}$	Velocity-space constraint Jacobian $\partial\mathbf{c}/\partial\mathcal{V}$ for non-holonomic constraints
$\mathcal{D}^2\mathbf{c}$	Constraint Hessian using the left-trivialized derivative

Symbol	Description
$\Delta \mathbf{X}$	Incremental displacement applied to resolve constraints
$\Delta \phi$	World-space angular displacement, $\Delta \phi = J_L \Delta \theta$
$\mathbf{F} = (\mathbf{f}, \boldsymbol{\tau})$	Generalised force vector (force and torque)
$\boldsymbol{\tau}_w$	World-space torque
<i>Iterative solver</i>	
k	Iteration index
Δt	Timestep
ξ	SOR relaxation parameter ($\xi < 1$ under-relaxation, $\xi > 1$ over-relaxation)
G	Gauss–Seidel iteration matrix, $G = -(D + L)^{-1} L^T$
G_J	Jacobi iteration matrix, $G_J = -D^{-1}(L + L^T)$
G_ξ	SOR iteration matrix
$\rho(\cdot)$	Spectral radius of a matrix
D, L, L^T	Diagonal, strictly lower triangular, and strictly upper triangular parts of A
<i>Compliant constraints and regularization</i>	
A	Diagonal matrix of stiffness parameters in compliant constraints
B	Diagonal matrix of damping parameters
$\tilde{\alpha}_i$	XPBD compliance parameter, $\tilde{\alpha}_i = 1/(A_{ii}\Delta t^2)$
$\tilde{\beta}_i$	XPBD damping parameter, $\tilde{\beta}_i = B_{ii}\Delta t^2$
ρ	Diagonal matrix of penalty coefficients in the augmented Lagrangian method
ε	Tikhonov regularization parameter
<i>Convergence analysis</i>	
μ_k	Eigenvalues of the Jacobi iteration matrix, $\mu_k = \cos(k\pi/N)$
r_i	Diagonal dominance ratio for row i , $r_i = \sum_{j \neq i} A_{ij} / A_{ii} $
γ	Mass ratio m/M in the chain convergence analysis
α	Fraction of error to be removed (convergence target)

2 Rigid Body Configuration

Rigid bodies are an idealised type of object that does not deform. I.e., for any two points $\mathbf{p}_i, \mathbf{p}_j$ in the rigid body the distance $|\mathbf{p}_i - \mathbf{p}_j|$ remains constant.



They have no internal degrees of freedom, and only have six total degrees of freedom:

- Three to define the position $\mathbf{x}$
- Three to define the orientation $\boldsymbol{\theta}$

A rigid body rotates around its centre of mass and moves through space on the trajectory of its centre of mass, so these six coordinates are the coordinates of the rigid body centre of mass $\mathbf{x}$. The location of any point on the rigid body p at any time t can be deduced from the position of the centre of mass $\mathbf{x}(t)$, the orientation of the body $R(t)$ and the (fixed) offset of the point on the rigid body $\mathbf{r}_0$:

$$\mathbf{p}(t) = \mathbf{x}(t) + R(t)\mathbf{r}_0. \quad (2.1)$$

The condition that distances between points on the rigid body remain fixed places conditions on the matrix R

$$\|\mathbf{p}_i - \mathbf{p}_j\|^2 = \|R(\mathbf{r}_i - \mathbf{r}_j)\|^2 = (R(\mathbf{r}_i - \mathbf{r}_j)) \cdot (R(\mathbf{r}_i - \mathbf{r}_j)) = (\mathbf{r}_i - \mathbf{r}_j)^T R^T R (\mathbf{r}_i - \mathbf{r}_j). \quad (2.2)$$

Therefore, we require $R^T R = \mathbb{1}$ so that distances remain the same, i.e. R is an orthogonal matrix, with $R^{-1} = R^T$.

Orthogonal matrices form a group: if two matrices R_i and R_j are orthogonal then their product will also be orthogonal

$$(R_i R_j)^T (R_i R_j) = R_j^T R_i^T R_i R_j = R_j^T \mathbb{1} R_j = \mathbb{1}. \quad (2.3)$$

This group is called the *orthogonal group* in three dimensions $O(3)$.

Since $R^T R = \mathbb{1}$, the determinant

$$\det(R^T R) = \det(R)^2 = 1, \quad (2.4)$$

so the determinant is either $+1$ or -1 . Matrices with $\det(R) = +1$ form a subgroup, since the product of two of matrices with determinant $+1$ is also a matrix with determinant $+1$. This is called the *special orthogonal group* $SO(3)$ and represents rotations. Matrices with determinant -1 do not form a subgroup, as the product of two such matrices is a matrix with determinant $+1$. These represent reflections, and the product of two reflections is a rotation.

To justify that these two classes represent rotations and reflections respectively, we can look at two examples. First, a rotation about the z axis

$$R = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \det(R) = \cos^2 \theta + \sin^2 \theta = 1. \quad (2.5)$$

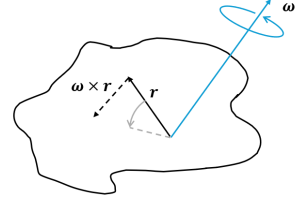
And a reflection in the xy plane

$$R = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{pmatrix}, \quad \det(R) = -1. \quad (2.6)$$

2.1 Angular Velocity

The time derivative of the offset vector $\mathbf{r}(t) = R(t)\mathbf{r}_0$ is

$$\begin{aligned} \dot{\mathbf{r}}(t) &= \dot{R}(t)\mathbf{r}_0, \\ &= \dot{R}(t)R^T(t)\mathbf{r}. \end{aligned} \quad (2.7)$$



We know that the matrix $\dot{R}R^T$ must be anti-symmetric, since $RR^T = \mathbb{1}$ so

$$\frac{d}{dt}(RR^T) = \dot{R}R^T + R\dot{R}^T = \dot{R}R^T + (\dot{R}R^T)^T = 0. \quad (2.8)$$

Since $(\dot{R}R^T)$ is anti-symmetric, we can use it to define a vector $\boldsymbol{\omega}$ using the anti-symmetric matrix $[\boldsymbol{\omega}]_{\times}$

$$\dot{R}R^T = [\boldsymbol{\omega}]_{\times} = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix}. \quad (2.9)$$

Comparing $\dot{\mathbf{r}} = \dot{R}(t)R^T(t)\mathbf{r}$ to the standard result from mechanics

$$\dot{\mathbf{r}} = \boldsymbol{\omega} \times \mathbf{r}. \quad (2.10)$$

We identify $\boldsymbol{\omega}$ as the world space angular velocity.

Body space angular velocity $\boldsymbol{\omega}_b$ can be derived from the world space angular velocity using

$$R\boldsymbol{\omega}_b = \boldsymbol{\omega}, \quad (2.11)$$

so

$$\begin{aligned} [R\boldsymbol{\omega}_b]_{\times} &= [\boldsymbol{\omega}]_{\times}, \\ R[\boldsymbol{\omega}_b]_{\times}R^T &= \dot{R}R^T, \\ [\boldsymbol{\omega}_b]_{\times} &= R^T\dot{R}, \end{aligned} \quad (2.12)$$

where we have used the fact that matrices A transform under a rotation R as $A' = RAR^T$.

2.1.1 The Path Ordered Exponential

Consider a body rotating with a constant angular velocity $\boldsymbol{\omega}(t) = \boldsymbol{\omega}_0$. For an infinitesimal time Δt a vector undergoes an infinitesimal rotation

$$\mathbf{r}' = \mathbf{r} + \boldsymbol{\omega}_0 \times \mathbf{r} \Delta t = (\mathbb{1} + [\boldsymbol{\omega}_0]_{\times} \Delta t) \mathbf{r}. \quad (2.13)$$

We can build up a finite rotation as a product of infinitesimal rotations

$$\begin{aligned} R\mathbf{r} &= (\mathbb{1} + [\boldsymbol{\omega}_0 \Delta t]_{\times}) (\mathbb{1} + [\boldsymbol{\omega}_0 \Delta t]_{\times}) \cdots (\mathbb{1} + [\boldsymbol{\omega}_0 \Delta t]_{\times}) \mathbf{r}, \\ &= \lim_{n \rightarrow \infty} \left(\mathbb{1} + \frac{1}{n} [\boldsymbol{\omega}_0 \Delta T]_{\times} \right)^n \mathbf{r}, \end{aligned} \quad (2.14)$$

where $\Delta t = \Delta T/n$. This limit gives an exponential

$$R(\Delta T) = e^{[\boldsymbol{\omega}_0 \Delta T]_{\times}}. \quad (2.15)$$

For this case of constant angular velocity we can also derive this exponential form by integrating the equation (2.9).

$$\ln(R) = \int \dot{R} R^T dt = \int [\boldsymbol{\omega}]_{\times} dt \quad (2.16)$$

It is tempting to want to generalize this for a general non-constant $\boldsymbol{\omega}(t)$ to

$$R(t) = e^{\int_0^t [\boldsymbol{\omega}(t')]_{\times} dt'} \quad (2.17)$$

but this is not exactly right, as different values of $[\boldsymbol{\omega}]_{\times}$ at different times do not commute with each other (since rotations do not commute)

$$[\boldsymbol{\omega}_1]_{\times} [\boldsymbol{\omega}_2]_{\times} \neq [\boldsymbol{\omega}_2]_{\times} [\boldsymbol{\omega}_1]_{\times}. \quad (2.18)$$

The answer is to use the *path ordered exponential*

$$\begin{aligned} R(t) &= P \exp \left(\int_0^t [\boldsymbol{\omega}(t')]_{\times} dt' \right), \\ &= 1 + \int_0^t [\boldsymbol{\omega}(t_1)]_{\times} dt_1 + \int_0^t dt_2 \int_0^{t_2} dt_1 [\boldsymbol{\omega}(t_2)]_{\times} [\boldsymbol{\omega}(t_1)]_{\times} + \cdots \end{aligned} \quad (2.19)$$

for $0 < t_1 < t_2 < t$. I.e. the angular velocities are ordered so that later times appear on the left.

This simplifies if we just want to take an incremental update going from some initial state $R(t)$ at time t to a new state at time $t + \Delta t$ for some small Δt . We can use

$$\begin{aligned} R(t + \Delta t) &= \exp \left(\int_t^{t+\Delta t} [\boldsymbol{\omega}(t')]_{\times} dt' \right) R(t), \\ &\approx \exp ([\boldsymbol{\omega}(t)]_{\times} \Delta t) R(t), \\ &= R(t) + [\boldsymbol{\omega}(t) \Delta t]_{\times} R(t) + \mathcal{O}(\Delta t^2), \end{aligned} \quad (2.20)$$

$R(t + \Delta t) \approx (\mathbb{1} + [\boldsymbol{\omega} \Delta t]_{\times}) R(t)$
(2.21)

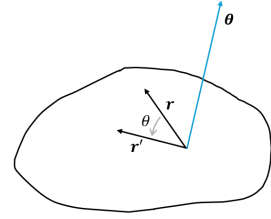
where we assume that for a small enough time Δt the angular velocity can be considered to be constant.

Note that, in practice, when updating a rotation matrix, it is important to retain its orthogonality, so in a rigid body simulation if we were updating rotation matrices in this way, we would have to orthonormalize the matrix after the update, which can be expensive, hence the general use of quaternions, which will be covered in section §2.2.

See David Tong's notes [39] for more detail on angular velocity, rotations and path ordered exponentials.

2.1.2 Axis/Angle Representation and the Exponential Map

The rotation matrix is a 3×3 matrix, so has 9 elements, but there are only three independent angular degrees of freedom. Later, when we come to look at rigid body constraints, we will need to be able to take the derivative of a function with respect to the angular degrees of freedom, so it is useful to find a representation that is a function of three independent variables. One such representation is the axis/angle representation.



Any orientation of a rigid body can be seen as a single rotation about a fixed axis $\hat{n}$ by an angle θ . This statement is known as *Euler's rotation theorem*. Therefore, we can define a vector $\boldsymbol{\theta} = \theta \hat{\boldsymbol{\theta}}$, where the magnitude θ is the angle of rotation, and the normalized direction $\hat{\boldsymbol{\theta}} = \hat{\boldsymbol{n}}$ is the axis of rotation, to encode a rotation.

This means that, using (2.15), we can write any rotation matrix as

$$R(\boldsymbol{\theta}) = e^{[\boldsymbol{\theta}]_{\times}}. \quad (2.22)$$

Technically, this exponential map is a map between elements R of the Lie group $SO(3)$ consisting of 3×3 orthogonal matrices with unit determinant and its corresponding Lie algebra elements $[\boldsymbol{\theta}]_{\times}$ in $\mathfrak{so}(3)$ consisting of 3×3 skew-symmetric matrices. See, for example [44] for more information on the theory of Lie groups.

This exponential can be expanded to give the very useful *Rodrigues Rotation Formula*

$$\begin{aligned} R(\boldsymbol{\theta}) &= e^{[\boldsymbol{\theta}]_{\times}}, \\ &= \mathbb{1} + [\boldsymbol{\theta}]_{\times} + \frac{1}{2!}[\boldsymbol{\theta}]_{\times}^2 + \frac{1}{3!}[\boldsymbol{\theta}]_{\times}^3 + \dots, \\ &= \mathbb{1} + [\boldsymbol{\theta}]_{\times} \left(1 - \frac{1}{3!}|\boldsymbol{\theta}|^2 + \dots\right) + [\boldsymbol{\theta}]_{\times}^2 \left(\frac{1}{2!} - \frac{1}{4!}|\boldsymbol{\theta}|^2 + \dots\right), \\ &= \mathbb{1} + \frac{\sin \theta}{\theta}[\boldsymbol{\theta}]_{\times} + \frac{1 - \cos \theta}{\theta^2}[\boldsymbol{\theta}]_{\times}^2 \end{aligned} \quad (2.23)$$

where we have used $[\boldsymbol{\theta}]_{\times}^3 = -|\boldsymbol{\theta}|^2[\boldsymbol{\theta}]_{\times}$.

2.1.3 The Left Jacobian of $\text{SO}(3)$

It's important to remember that $R(\boldsymbol{\theta})$ represents a single rotation, and we can't compose rotations by adding axis/angles

$$e^{[\boldsymbol{\theta}_1 + \boldsymbol{\theta}_2]_{\times}} \neq e^{[\boldsymbol{\theta}_1]_{\times}} e^{[\boldsymbol{\theta}_2]_{\times}} \quad (2.24)$$

since $[\boldsymbol{\theta}_1]_{\times}$ and $[\boldsymbol{\theta}_2]_{\times}$ do not necessarily commute. They only commute if the rotations are about the same axis. What we can do is to ask what is $R(\boldsymbol{\theta} + \Delta\boldsymbol{\theta})$ for some small incremental axis/angle $\Delta\boldsymbol{\theta}$. To explore this, we can use the *Baker-Campbell-Hausdorff Formulas*, which can be used to relate small perturbations in $[\boldsymbol{\theta}]_{\times}$ to small perturbations in $R(\boldsymbol{\theta})$.

The BCH formula finds Z for $e^Z = e^X e^Y$

$$Z = X + Y + \frac{1}{2}[X, Y] + \frac{1}{12}[X, [X, Y]] - \frac{1}{12}[Y, [X, Y]] + \dots \quad (2.25)$$

where $[A, B] = AB - BA$ is the commutator of A and B .

We want to use this to find a decomposition of the rotation for small angles, i.e. find a $J_L \Delta\boldsymbol{\theta}$ such that $e^{[\boldsymbol{\theta} + \Delta\boldsymbol{\theta}]_{\times}} = e^{[J_L \Delta\boldsymbol{\theta}]_{\times}} e^{[\boldsymbol{\theta}]_{\times}}$, or $e^{[J_L \Delta\boldsymbol{\theta}]_{\times}} = e^{[\boldsymbol{\theta} + \Delta\boldsymbol{\theta}]_{\times}} e^{-[\boldsymbol{\theta}]_{\times}}$. So, letting $Z = [J_L \Delta\boldsymbol{\theta}]_{\times}$, $X = [\boldsymbol{\theta} + \Delta\boldsymbol{\theta}]_{\times}$, and $Y = -[\boldsymbol{\theta}]_{\times}$ we have

$$\begin{aligned} [J_L \Delta\boldsymbol{\theta}]_{\times} &= [\boldsymbol{\theta} + \Delta\boldsymbol{\theta}]_{\times} - [\boldsymbol{\theta}]_{\times} - \frac{1}{2}[[\boldsymbol{\theta} + \Delta\boldsymbol{\theta}]_{\times}, [\boldsymbol{\theta}]_{\times}] - \frac{1}{12}[[\boldsymbol{\theta} + \Delta\boldsymbol{\theta}]_{\times}, [[\boldsymbol{\theta} + \Delta\boldsymbol{\theta}]_{\times}, [\boldsymbol{\theta}]_{\times}]] \\ &\quad - \frac{1}{12}[[\boldsymbol{\theta}]_{\times}, [[\boldsymbol{\theta} + \Delta\boldsymbol{\theta}]_{\times}, [\boldsymbol{\theta}]_{\times}]] + \dots, \\ &= [\Delta\boldsymbol{\theta}]_{\times} + \frac{1}{2}[[\boldsymbol{\theta}]_{\times}, [\Delta\boldsymbol{\theta}]_{\times}] + \frac{1}{6}[[\boldsymbol{\theta}]_{\times}, [[\boldsymbol{\theta}]_{\times}, [\Delta\boldsymbol{\theta}]_{\times}]] + \dots, \\ &= \left(\mathbb{1} + \frac{1}{2}\text{ad}_{[\boldsymbol{\theta}]_{\times}} + \frac{1}{6}\text{ad}_{[\boldsymbol{\theta}]_{\times}}^2 \right) [\Delta\boldsymbol{\theta}]_{\times} + \dots, \\ &= \sum_{k=0}^{\infty} \frac{1}{(k+1)!} \text{ad}_{[\boldsymbol{\theta}]_{\times}}^k [\Delta\boldsymbol{\theta}]_{\times}. \end{aligned} \quad (2.26)$$

where we have used the adjoint notation $\text{ad}_{[\boldsymbol{\theta}]_{\times}} [\Delta\boldsymbol{\theta}]_{\times} = [[\boldsymbol{\theta}]_{\times}, [\Delta\boldsymbol{\theta}]_{\times}]$. This obeys the identity

$$\begin{aligned} \text{ad}_{[\boldsymbol{\theta}]_{\times}} [\Delta\boldsymbol{\theta}]_{\times} &= [[\boldsymbol{\theta}]_{\times}, [\Delta\boldsymbol{\theta}]_{\times}], \\ &= [\boldsymbol{\theta} \times \Delta\boldsymbol{\theta}]_{\times}, \\ &= [[\boldsymbol{\theta}]_{\times} \Delta\boldsymbol{\theta}]_{\times}. \end{aligned} \quad (2.27)$$

Therefore, we have

$$\text{ad}_{[\boldsymbol{\theta}]_{\times}}^k [\Delta\boldsymbol{\theta}]_{\times} = [[\boldsymbol{\theta}]_{\times}^k \Delta\boldsymbol{\theta}]_{\times}. \quad (2.28)$$

Substituting this into the equation for $[J_L \Delta\boldsymbol{\theta}]_{\times}$ we get

$$[J_L \Delta\boldsymbol{\theta}]_{\times} = \sum_{k=0}^{\infty} \frac{1}{(k+1)!} [[\boldsymbol{\theta}]_{\times}^k \Delta\boldsymbol{\theta}]_{\times}, \quad (2.29)$$

so

$$J_L = \sum_{k=0}^{\infty} \frac{1}{(k+1)!} [\boldsymbol{\theta}]_{\times}^k \quad (2.30)$$

Now, using the fact that $[\boldsymbol{\theta}]_{\times}^3 = -\theta^2[\boldsymbol{\theta}]_{\times}$, where θ is the magnitude of $\boldsymbol{\theta}$, we see that odd powers are all functions of $[\boldsymbol{\theta}]_{\times}$ and even powers a function of $[\boldsymbol{\theta}]_{\times}^2$ and we get the closed form solution

$$J_L(\boldsymbol{\theta}) = \mathbb{1} + \frac{1 - \cos \theta}{\theta^2}[\boldsymbol{\theta}]_{\times} + \frac{\theta - \sin \theta}{\theta^3}[\boldsymbol{\theta}]_{\times}^2. \quad (2.31)$$

$J_L(\boldsymbol{\theta})$ is known as the *Left Jacobian* of $SO(3)$ [5, 36]. It allows us to approximate a new rotation given a small change in the axis/angle $\Delta\boldsymbol{\theta}$

$$\begin{aligned} R(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) &= e^{[J_L(\boldsymbol{\theta})\Delta\boldsymbol{\theta}]_{\times}} R(\boldsymbol{\theta}), \\ &\approx (\mathbb{1} + [J_L(\boldsymbol{\theta})\Delta\boldsymbol{\theta}]_{\times}) R(\boldsymbol{\theta}). \end{aligned} \quad (2.32)$$

Comparing to (2.21) we see that for small perturbations

$$\boldsymbol{\omega}(t)\Delta t \approx J_L(\boldsymbol{\theta})\Delta\boldsymbol{\theta}, \quad (2.33)$$

so J_L maps a small change in the axis-angle coordinates to the corresponding left-trivialized angular displacement. More precisely, as $\Delta t \rightarrow 0$ we see that it maps the rate of change of the axis angle $\boldsymbol{\theta}$ to the angular velocity

$$\boldsymbol{\omega} = J_L(\boldsymbol{\theta})\dot{\boldsymbol{\theta}}. \quad (2.34)$$

To see this more precisely, we can take another route. First, take the time derivative of the rotation matrix.

$$\begin{aligned} \dot{R}(t) &= \frac{d}{dt} e^{[\boldsymbol{\theta}(t)]_{\times}}, \\ &= \frac{d}{dt} \lim_{N \rightarrow \infty} \left(\mathbb{1} + \frac{1}{N}[\boldsymbol{\theta}(t)]_{\times} \right)^N, \\ &= \lim_{N \rightarrow \infty} \sum_{k=1}^N \left(\mathbb{1} + \frac{1}{N}[\boldsymbol{\theta}(t)]_{\times} \right)^{N-k} \frac{1}{N}[\dot{\boldsymbol{\theta}}(t)]_{\times} \left(\mathbb{1} + \frac{1}{N}[\boldsymbol{\theta}(t)]_{\times} \right)^{k-1}. \end{aligned} \quad (2.35)$$

Note that $[\boldsymbol{\theta}(t)]_{\times}$ and $[\dot{\boldsymbol{\theta}}(t)]_{\times}$ do not necessarily commute. Now re-scale the interval into $[0, \dots, 1]$ using $\Delta s = \Delta k/N = 1/N$, since k are integers so Δk is 1. Taking the limit as $N \rightarrow \infty$ gives

$$\dot{R}(t) = \int_0^1 e^{(1-s)[\boldsymbol{\theta}(t)]_{\times}} [\dot{\boldsymbol{\theta}}(t)]_{\times} e^{s[\boldsymbol{\theta}(t)]_{\times}} ds. \quad (2.36)$$

Now let $s' = 1 - s$

$$\begin{aligned} \dot{R}(t) &= - \int_1^0 e^{s'[\boldsymbol{\theta}(t)]_{\times}} [\dot{\boldsymbol{\theta}}(t)]_{\times} e^{(1-s')[\boldsymbol{\theta}(t)]_{\times}} ds', \\ &= \int_0^1 e^{s[\boldsymbol{\theta}(t)]_{\times}} [\dot{\boldsymbol{\theta}}(t)]_{\times} e^{(1-s)[\boldsymbol{\theta}(t)]_{\times}} ds, \\ &= \left(\int_0^1 e^{s[\boldsymbol{\theta}(t)]_{\times}} [\dot{\boldsymbol{\theta}}(t)]_{\times} e^{-s[\boldsymbol{\theta}(t)]_{\times}} ds \right) R(t). \end{aligned} \quad (2.37)$$

Looking at the expression in the integral this is $R(s\boldsymbol{\theta})[\dot{\boldsymbol{\theta}}]_{\times}R(s\boldsymbol{\theta})^T$. We know that this is how a matrix R transforms under a rotation. A vector would transform as $R(s\boldsymbol{\theta})\dot{\boldsymbol{\theta}}$, so

$$\dot{R}(t)R^T(t) = [\boldsymbol{\omega}(t)]_{\times} = \left[\left(\int_0^1 e^{s[\boldsymbol{\theta}(t)]_{\times}} ds \right) \dot{\boldsymbol{\theta}}(t) \right]_{\times}. \quad (2.38)$$

(Technically, the adjoint action of the group $SO(3)$ acting on its algebra $\mathfrak{so}(3)$ is $Ad_R[X]_{\times} = R[X]_{\times} R^T$. Elements of the Lie algebra can be mapped to vectors $\mathbf{x} \in \mathbb{R}^3$ with $Ad_R \mathbf{x} = R \mathbf{x}$).

If we define

$$J_L(\boldsymbol{\theta}) = \int_0^1 e^{s[\boldsymbol{\theta}]_{\times}} ds \quad (2.39)$$

then

$$\boldsymbol{\omega} = J_L(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}}. \quad (2.40)$$

It can be verified that this is the same J_L as we got from the BCH expansion by using the Rodrigues rotation formula and performing the integral

$$\begin{aligned} J_L(\boldsymbol{\theta}) &= \int_0^1 e^{s[\boldsymbol{\theta}]_{\times}} ds, \\ &= \int_0^1 \left(\mathbb{1} + \frac{\sin s\theta}{\theta} [\boldsymbol{\theta}]_{\times} + \frac{1 - \cos s\theta}{\theta^2} [\boldsymbol{\theta}]_{\times}^2 \right) ds, \\ &= \mathbb{1} + \frac{1 - \cos \theta}{\theta^2} [\boldsymbol{\theta}]_{\times} + \frac{\theta - \sin \theta}{\theta^3} [\boldsymbol{\theta}]_{\times}^2. \end{aligned} \quad (2.41)$$

In summary:

J_L is known as the left Jacobian of $SO(3)$ and maps the derivative of the axis angle vector to the world space angular velocity.

$$\boldsymbol{\omega} = J_L(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}}$$

It can be used to find how infinitesimal changes in the axis/angle $\boldsymbol{\theta}$ update the rotation $R = e^{[\boldsymbol{\theta}]_{\times}}$

$$e^{[\boldsymbol{\theta} + \Delta\boldsymbol{\theta}]_{\times}} \approx e^{[J_L(\boldsymbol{\theta})\Delta\boldsymbol{\theta}]_{\times}} e^{[\boldsymbol{\theta}]_{\times}} = e^{[\Delta\boldsymbol{\phi}]_{\times}} R(\boldsymbol{\theta}). \quad (2.42)$$

This is very useful for rigid body simulations where we are taking small perturbations in the configuration space, and taking derivatives of constraints with respect to the constraint rotation.

Note that the left Jacobian here is what Macklin et al [24] describe as the *kinematic map* that maps spatial velocities to system coordinate time derivatives $\mathbf{G}$. J_L is $\mathbf{G}$ for $SO(3)$.

Derivatives

It will be important to be able to take derivatives with respect to $\boldsymbol{\theta}$, e.g. in 3.2 where we want to be able to take derivatives of a constraint that is a function of $\boldsymbol{\theta}$. We can see how to do this using the left Jacobian:

Define the *left-trivialized derivative* of a function f as

$$\mathcal{D}f = \frac{\partial f}{\partial \boldsymbol{\theta}} J_L(\boldsymbol{\theta})^{-1}. \quad (2.43)$$

in this way

$$\dot{f} = \frac{\partial f}{\partial \boldsymbol{\theta}} \dot{\boldsymbol{\theta}} = \frac{\partial f}{\partial \boldsymbol{\theta}} J_L(\boldsymbol{\theta})^{-1} \boldsymbol{\omega} = (\mathcal{D}f) \boldsymbol{\omega}. \quad (2.44)$$

The left Jacobian inverse is used to convert from a rate of change with respect to the axis/angle coordinates $\boldsymbol{\theta}$ to a rate of change with respect to the angular velocity $\boldsymbol{\omega}$.

2.2 Quaternion Representation*

This section is mostly tangential to the rest of the notes which, on the whole, use the axis/angle and exponential map representation of rotations. The exception is in integrating rigid body rotations, where maintaining orthogonality is important, so we use the quaternion representation as it is more efficient to normalize.

In this section, we will be using and assuming some familiarity with geometric algebra, the natural and most intuitive setting for quaternions. Good references for this are [20, 16]. For readers familiar with Hamilton's formulation of quaternions, the end result will be familiar and you can safely replace $\mathcal{I}$ with the unit complex number i .

When using quaternions, we define a point on a rigid body by its centre of mass $\mathbf{x}(t)$ and a quaternion $q(t)$ representing the orientation, so that a point on the body is

$$\mathbf{p}(t) = \mathbf{x}(t) + q(t) \mathbf{r}_0 \bar{q}(t), \quad (2.45)$$

where $\bar{q}$ is the quaternion conjugate. The condition that the distance between two points on the body remains fixed means that, using the geometric product

$$\|\mathbf{p}_i - \mathbf{p}_j\|^2 = q(\mathbf{p}_i - \mathbf{p}_j) \bar{q} q(\mathbf{p}_i - \mathbf{p}_j) \bar{q} \quad (2.46)$$

So we require that $q\bar{q} = \bar{q}q = 1$ for this to be invariant.

The time derivative of a vector $\mathbf{r}(t) = q(t) \mathbf{r}_0 \bar{q}(t)$ is

$$\begin{aligned} \dot{\mathbf{r}}(t) &= \dot{q}(t) \mathbf{r}_0 \bar{q}(t) + q(t) \mathbf{r}_0 \dot{\bar{q}}(t), \\ &= \dot{q}(t) \bar{q}(t) \mathbf{r}(t) + \mathbf{r}(t) q(t) \dot{\bar{q}}(t), \\ &= (\dot{q}(t) \bar{q}(t)) \mathbf{r}(t) - \mathbf{r}(t) (\dot{q}(t) \bar{q}(t)). \end{aligned} \quad (2.47)$$

Now define the bivector $\boldsymbol{\Omega} = 2\dot{q}(t) \bar{q}(t)$. This is then

$$\begin{aligned} \dot{\mathbf{r}}(t) &= \frac{1}{2} (\boldsymbol{\Omega} \mathbf{r} - \mathbf{r} \boldsymbol{\Omega}), \\ &= \boldsymbol{\Omega} \cdot \mathbf{r}, \end{aligned} \quad (2.48)$$

where $\boldsymbol{\Omega} \cdot \mathbf{r} = \frac{1}{2} (\boldsymbol{\Omega} \mathbf{r} - \mathbf{r} \boldsymbol{\Omega})$ is the inner product of a bivector with a vector. $\boldsymbol{\Omega}$ is the dual of the angular velocity vector $\boldsymbol{\omega}$

$$\boldsymbol{\omega} = \mathcal{I} \boldsymbol{\Omega}, \quad \boldsymbol{\Omega} = -\mathcal{I} \boldsymbol{\omega}, \quad (2.49)$$

where $\mathcal{I} = \mathbf{e}_1 \mathbf{e}_2 \mathbf{e}_3$ is the unit pseudoscalar, with the property $\mathcal{I}^2 = -1$.

2.2.1 Quaternions from the Path Ordered Exponential

Just as we found a differential equation for the rotation matrix $\dot{R} = [\boldsymbol{\omega}]_{\times} R$, we can find the equivalent differential equation for the quaternion q . Starting from our definition of the angular velocity bivector $\boldsymbol{\Omega} = 2\dot{q}\bar{q}$ we have

$$\dot{q}(t) = \frac{1}{2}\boldsymbol{\Omega}(t)q(t). \quad (2.50)$$

This equation is identical in form to the one we solved for rotation matrices. If the body is rotating with a constant angular velocity bivector $\boldsymbol{\Omega}_0$, we can integrate this directly to find the quaternion at time t :

$$q(t) = \exp\left(\frac{1}{2}\boldsymbol{\Omega}_0 t\right). \quad (2.51)$$

However, for a general non-constant angular velocity $\boldsymbol{\Omega}(t)$, the bivectors at different times do not commute. Just as in the rotation matrix case, we must use the *path ordered exponential* to build up the finite rotation from infinitesimal steps:

$$q(t) = P \exp\left(\frac{1}{2} \int_0^t \boldsymbol{\Omega}(t') dt'\right) \quad (2.52)$$

In practice, for physics simulations and games, we usually want to perform an incremental update over a small timestep Δt . We can assume the angular velocity is roughly constant over this small window, giving the discrete update step:

$$\begin{aligned} q(t + \Delta t) &= \exp\left(\frac{1}{2} \int_t^{t+\Delta t} \boldsymbol{\Omega}(t') dt'\right) q(t), \\ &\approx \exp\left(\frac{1}{2}\boldsymbol{\Omega}(t)\Delta t\right) q(t), \\ &\approx \exp\left(-\frac{1}{2}\mathcal{I}\boldsymbol{\omega}(t)\Delta t\right) q(t). \end{aligned} \quad (2.53)$$

This quaternion update is generally favoured in numerical integration as normalizing the quaternion to avoid numerical drift is computationally easier than orthonormalizing a 3×3 matrix.

2.2.2 Relation to Axis/Angle Representation

To connect this back to our axis/angle representation $\boldsymbol{\theta} = \theta\hat{\mathbf{n}}$, we need to evaluate the exponential map of the bivector. Let $\hat{B}$ be the unit bivector representing the plane of rotation, which is (minus) the dual of the rotation axis $\hat{\mathbf{n}}$, $\hat{B} = -I\hat{\mathbf{n}}$, consistent with $\boldsymbol{\Omega} = -\mathcal{I}\boldsymbol{\omega}$ above. If a body rotates by a total angle θ in this plane, the integral of the angular velocity over that time is $\int \boldsymbol{\Omega} dt = \theta\hat{B}$. Substituting this into our exponential solution, the quaternion representing a rotation by $\boldsymbol{\theta}$ is:

$$q(\boldsymbol{\theta}) = \exp\left(\frac{1}{2}\theta\hat{B}\right). \quad (2.54)$$

Because the square of a unit bivector in 3D Euclidean space is negative ($\hat{B}^2 = -1$), the Taylor series expansion of this exponential gives Euler's formula for complex numbers:

$$\begin{aligned}
\exp\left(\frac{1}{2}\theta\hat{B}\right) &= 1 + \frac{1}{2}\theta\hat{B} + \frac{1}{2!}\left(\frac{1}{2}\theta\hat{B}\right)^2 + \frac{1}{3!}\left(\frac{1}{2}\theta\hat{B}\right)^3 + \dots \\
&= \left(1 - \frac{1}{2!}\left(\frac{\theta}{2}\right)^2 + \dots\right) + \hat{B}\left(\frac{\theta}{2} - \frac{1}{3!}\left(\frac{\theta}{2}\right)^3 + \dots\right) \\
&= \cos\left(\frac{\theta}{2}\right) + \hat{B}\sin\left(\frac{\theta}{2}\right).
\end{aligned} \tag{2.55}$$

This is the link between the axis/angle representation and the quaternion. A unit quaternion q is an even-grade multivector composed of a scalar part and a bivector part:

$$q = \cos\left(\frac{\theta}{2}\right) - \mathcal{I}\hat{n}\sin\left(\frac{\theta}{2}\right). \tag{2.56}$$

In standard (Hamilton) quaternion notation the unit bivectors $-\mathcal{I}\mathbf{e}_1, -\mathcal{I}\mathbf{e}_2, -\mathcal{I}\mathbf{e}_3$ are the imaginary units i, j, k (this is the identification for which $ij = k$), so the pure quaternion $(0, \boldsymbol{\omega})$ is exactly $\boldsymbol{\Omega}$ and $\dot{q} = \frac{1}{2}\boldsymbol{\Omega}q$ is the familiar quaternion kinematic equation $\dot{q} = \frac{1}{2}(0, \boldsymbol{\omega})q$. Notice the appearance of the half-angle $\theta/2$. This occurs because the vector transformation law $\mathbf{r} = q\mathbf{r}_0\bar{q}$ applies the quaternion twice (once on the left, once on the right). To rotate a vector by a full angle θ , each side of the sandwich product must contribute half of the rotation.

3 Equations of Motion

The goal is to find the equations of motion for a rigid body. To do this we follow a Lagrangian derivation. The Lagrangian for a system is the difference between the kinetic and potential energy. For a single rigid body this is

$$L = \frac{1}{2}m\|\dot{\mathbf{x}}\|^2 + \frac{1}{2}\boldsymbol{\omega} \cdot I(\boldsymbol{\theta})\boldsymbol{\omega} - V(\mathbf{x}, \boldsymbol{\theta}), \quad (3.1)$$

for mass m , inertia $I(\boldsymbol{\theta})$ and potential energy $V(\mathbf{x}, \boldsymbol{\theta})$. For standard variational mechanics, we want to make this a function of some configuration variable $\mathbf{X}$ and its time derivative $\dot{\mathbf{X}}$. In this notation

The Lagrangian for a single rigid body is

$$L(\mathbf{X}, \dot{\mathbf{X}}) = \frac{1}{2}\dot{\mathbf{X}} \cdot M_L(\mathbf{X})\dot{\mathbf{X}} - V(\mathbf{X}) \quad (3.2)$$

where

$$M_L(\mathbf{X}) = \begin{pmatrix} m\mathbb{1} & 0 \\ 0 & J_L(\boldsymbol{\theta})^T I(\boldsymbol{\theta}) J_L(\boldsymbol{\theta}) \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} \mathbf{x} \\ \boldsymbol{\theta} \end{pmatrix}, \quad \dot{\mathbf{X}} = \begin{pmatrix} \mathbf{v} \\ J_L(\boldsymbol{\theta})^{-1}\boldsymbol{\omega} \end{pmatrix}. \quad (3.3)$$

Notice that we used the left Jacobian $J_L(\boldsymbol{\theta})$ to map the angular velocity $\boldsymbol{\omega}$ to the axis angle derivative $\dot{\boldsymbol{\theta}}$, and factored it into the mass matrix $M_L(\mathbf{X})$, so that we get a kinetic energy in terms of $\dot{\mathbf{X}}$.

In standard Lagrangian mechanics we vary the action according to

$$\begin{aligned} \delta S &= \int dt \delta L(\dot{\mathbf{X}}, \mathbf{X}), \\ &= \int dt \left(\frac{\partial L}{\partial \dot{\mathbf{X}}} \delta \dot{\mathbf{X}} + \frac{\partial L}{\partial \mathbf{X}} \delta \mathbf{X} \right), \\ &= \int dt \left(-\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\mathbf{X}}} \right) + \frac{\partial L}{\partial \mathbf{X}} \right) \delta \mathbf{X} + \left[\frac{\partial L}{\partial \dot{\mathbf{X}}} \delta \mathbf{X} \right]_{\text{boundary}}. \end{aligned} \quad (3.4)$$

Where the last step uses integration by parts. The term $\delta \mathbf{X}$ is taken to be zero on the boundary, so we are left with the Euler-Lagrange equations of motion

$$-\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\mathbf{X}}} \right) + \frac{\partial L}{\partial \mathbf{X}} = 0, \quad (3.5)$$

which, for our rigid body Lagrangian gives

$$\frac{d}{dt} \left(M_L(\mathbf{X}) \dot{\mathbf{X}} \right) = -\frac{\partial V}{\partial \mathbf{X}} + \frac{\partial M_L}{\partial \mathbf{X}} \dot{\mathbf{X}}. \quad (3.6)$$

For the linear term this gives Newton's second law for the linear motion of the centre of mass

$$m\ddot{\mathbf{x}} = -\frac{\partial V}{\partial \mathbf{x}} = \mathbf{f}. \quad (3.7)$$

for force $\mathbf{f}$.

The angular degrees of freedom give

$$\frac{d}{dt} \left(I_L(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}} \right) = -\frac{\partial V}{\partial \boldsymbol{\theta}} + \frac{1}{2} \dot{\boldsymbol{\theta}} \cdot \frac{\partial I_L}{\partial \boldsymbol{\theta}} \dot{\boldsymbol{\theta}} \quad (3.8)$$

with $I_L(\boldsymbol{\theta}) = J_L(\boldsymbol{\theta})^T I(\boldsymbol{\theta}) J_L(\boldsymbol{\theta})$. This approach can be used (see, for example [8]) to recover Euler's equations

$$I(\boldsymbol{\theta}) \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (I(\boldsymbol{\theta}) \boldsymbol{\omega}) = \boldsymbol{\tau} \quad (3.9)$$

for a torque $\boldsymbol{\tau}$, but the algebra is complicated due to the third order tensor term $\frac{\partial I}{\partial \boldsymbol{\theta}}$. Instead, we can use the symmetry of the Lagrangian to find a simpler derivation.

3.1 The Euler-Poincaré Method

The Euler Poincaré method can improve on the standard Lagrangian approach when dealing with actions that are symmetric under some group action. The angular configuration of a rigid body lies in $SO(3)$, rather than a linear vector space, so it makes sense that when computing variations to look for ones that preserve that group symmetry. In general, this approach leads to the *Euler-Poincaré equations of motion*.

Start by considering the variation of a rotation matrix R . Since $RR^T = 1$ we have

$$\delta RR^T + R \delta R^T = 0, \quad (3.10)$$

so

$$(\delta RR^T) + (\delta RR^T)^T = 0 \quad (3.11)$$

meaning that δRR^T must be antisymmetric. Say

$$\delta RR^T = [\eta]_{\times}. \quad (3.12)$$

Now, from (2.9) we have $[\boldsymbol{\omega}]_{\times} = \dot{R}R^T$. From this we can compute the variation of $\boldsymbol{\omega}$.

$$\begin{aligned} [\delta \boldsymbol{\omega}]_{\times} &= \delta \dot{R}R^T + \dot{R} \delta R^T, \\ &= [\dot{\eta}]_{\times} + [\eta]_{\times} (\dot{R}R^T) - (\dot{R}R^T) [\eta]_{\times}, \\ &= [\dot{\eta}]_{\times} + [\eta]_{\times} [\boldsymbol{\omega}]_{\times} - [\boldsymbol{\omega}]_{\times} [\eta]_{\times}, \\ &= [\dot{\eta}]_{\times} - [\boldsymbol{\omega} \times \eta]_{\times}. \end{aligned} \quad (3.13)$$

Therefore,

$$\delta \boldsymbol{\omega} = \dot{\eta} - \boldsymbol{\omega} \times \eta. \quad (3.14)$$

The kinetic energy is $T = \frac{1}{2} \boldsymbol{\omega} \cdot I \boldsymbol{\omega}$, so taking the variation gives

$$\delta T = (I \boldsymbol{\omega}) \cdot \delta \boldsymbol{\omega} + \frac{1}{2} \boldsymbol{\omega} \cdot (\delta I \boldsymbol{\omega}). \quad (3.15)$$

Using $I = R I_b R^T$, where I_b is the fixed body space inertia, we have

$$\begin{aligned} \delta I &= \delta R R^T I + I R \delta R^T, \\ &= [\eta]_{\times} I - I [\eta]_{\times}. \end{aligned} \quad (3.16)$$

So

$$\begin{aligned}
\delta T &= (I\boldsymbol{\omega}) \cdot (\dot{\eta} - \boldsymbol{\omega} \times \eta) + \frac{1}{2} \boldsymbol{\omega} \cdot ([\eta]_{\times} I - I[\eta]_{\times}) \boldsymbol{\omega}, \\
&= (I\boldsymbol{\omega}) \cdot (\dot{\eta} - \boldsymbol{\omega} \times \eta) + \boldsymbol{\omega} \cdot (\eta \times (I\boldsymbol{\omega})), \\
&= (I\boldsymbol{\omega}) \cdot \dot{\eta}.
\end{aligned} \tag{3.17}$$

The terms constant in η cancel, as they must. If η is constant then the variation $\delta R = [\eta]_{\times} R$ is a rigid rotation of the whole configuration $R \rightarrow e^{[\eta]_{\times}} R$. The kinetic energy is unchanged under a fixed rotation of the system, so terms with constant η must cancel.

We can now compute the variation of the action

$$\begin{aligned}
\delta S &= \delta \int dt \left(\frac{1}{2} \boldsymbol{\omega} \cdot I\boldsymbol{\omega} - V(\boldsymbol{\theta}) \right), \\
&= \int dt \left((I\boldsymbol{\omega}) \cdot \dot{\eta} - \frac{\partial V}{\partial \boldsymbol{\theta}} \cdot \delta \boldsymbol{\theta} \right), \\
&= [(I\boldsymbol{\omega}) \cdot \eta]_{\text{boundary}} + \int dt \left(-\frac{d}{dt} (I\boldsymbol{\omega}) \cdot \eta - \frac{\partial V}{\partial \boldsymbol{\theta}} \cdot \delta \boldsymbol{\theta} \right),
\end{aligned} \tag{3.18}$$

where in the third step we used integration by parts and, by construction, variation η is zero on the boundary.

For the variation $\delta \boldsymbol{\theta}$ consider the variation δR this time using (2.42).

$$\begin{aligned}
\delta R &= \frac{d}{d\epsilon} e^{[\boldsymbol{\theta} + \epsilon \delta \boldsymbol{\theta}]_{\times}} \Big|_{\epsilon=0}, \\
&= \frac{d}{d\epsilon} e^{[\epsilon J_L \delta \boldsymbol{\theta}]_{\times}} R \Big|_{\epsilon=0}, \\
&= \frac{d}{d\epsilon} (\mathbb{1} + \epsilon [J_L \delta \boldsymbol{\theta}]_{\times} + \dots) R \Big|_{\epsilon=0}, \\
&= [J_L \delta \boldsymbol{\theta}]_{\times} R.
\end{aligned} \tag{3.19}$$

So

$$\delta R R^T = [J_L \delta \boldsymbol{\theta}]_{\times} = [\eta]_{\times} \tag{3.20}$$

and

$$\delta \boldsymbol{\theta} = J_L^{-1} \eta. \tag{3.21}$$

Therefore, the variation in the action is

$$\delta S = \int dt \left(-\frac{d}{dt} (I\boldsymbol{\omega}) - \frac{\partial V}{\partial \boldsymbol{\theta}} J_L^{-1} \right) \cdot \eta \tag{3.22}$$

Giving the equations of motion

$$\frac{d}{dt} (I\boldsymbol{\omega}) = -(\mathcal{D}V)^T = \boldsymbol{\tau}_w, \tag{3.23}$$

where we have used the left Jacobian derivative $\mathcal{D}$ notation from (2.43), and identified $-(\mathcal{D}V)^T$ as the world space torque $\boldsymbol{\tau}_w$.

Note that, using the time derivative of the inertia

$$\begin{aligned}
\dot{I}(\boldsymbol{\theta}) &= \frac{d}{dt} (R(\boldsymbol{\theta}) I_b R(\boldsymbol{\theta})^T), \\
&= \dot{R}(\boldsymbol{\theta}) I_b R(\boldsymbol{\theta})^T + R(\boldsymbol{\theta}) I_b \dot{R}(\boldsymbol{\theta})^T, \\
&= \dot{R}(\boldsymbol{\theta}) R(\boldsymbol{\theta})^T I(\boldsymbol{\theta}) + I(\boldsymbol{\theta}) R(\boldsymbol{\theta}) \dot{R}(\boldsymbol{\theta})^T. \\
&= \dot{R}(\boldsymbol{\theta}) R(\boldsymbol{\theta})^T I(\boldsymbol{\theta}) - I(\boldsymbol{\theta}) \left(\dot{R}(\boldsymbol{\theta}) R(\boldsymbol{\theta})^T \right), \\
&= [\boldsymbol{\omega}]_{\times} I(\boldsymbol{\theta}) - I(\boldsymbol{\theta}) [\boldsymbol{\omega}]_{\times},
\end{aligned} \tag{3.24}$$

we recover Euler's equations in world space

$$I(\boldsymbol{\theta}) \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (I(\boldsymbol{\theta}) \boldsymbol{\omega}) = \boldsymbol{\tau}_w. \tag{3.25}$$

Using this result combined with the linear equations of motion, we can write down a compact form for the equations of motion of N rigid bodies:

The equations of motion for a system of N rigid bodies can be written as

$$\frac{d}{dt} (M(\mathbf{X}) \mathcal{V}) = \mathbf{F} \tag{3.26}$$

where

$$M(\mathbf{X}) = \begin{pmatrix} m_1 \mathbb{1} & & & \\ & I_1(\boldsymbol{\theta}_1) & & \\ & & \ddots & \\ & & & m_N \mathbb{1} \\ & & & & I_N(\boldsymbol{\theta}_N) \end{pmatrix} \tag{3.27}$$

and

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 \\ \boldsymbol{\theta}_1 \\ \vdots \\ \mathbf{x}_N \\ \boldsymbol{\theta}_N \end{pmatrix}, \quad \mathcal{V} = \begin{pmatrix} \dot{\mathbf{x}}_1 \\ \boldsymbol{\omega}_1 \\ \vdots \\ \dot{\mathbf{x}}_N \\ \boldsymbol{\omega}_N \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} \mathbf{f}_1 \\ \boldsymbol{\tau}_1 \\ \vdots \\ \mathbf{f}_N \\ \boldsymbol{\tau}_N \end{pmatrix} \tag{3.28}$$

and the world space force $\mathbf{f}_i$ and torque $\boldsymbol{\tau}_i$ body i can be derived from a potential V

$$\mathbf{f}_i = -\frac{\partial V(\mathbf{X})}{\partial \mathbf{x}_i}, \quad \boldsymbol{\tau}_i = -\frac{\partial V(\mathbf{X})}{\partial \boldsymbol{\theta}_i} J_L^{-1}(\boldsymbol{\theta}_i) = -(\mathcal{D}_i V)^T. \tag{3.29}$$

3.2 Holonomic Constraints

In rigid body simulation, we want to add constraints to the system to model things like joints and contacts between bodies. We first consider holonomic constraints. These are constraints that are a function of the configuration $\mathbf{X}$ and not the velocity $\dot{\mathbf{X}}$. These constraints can be added to the system using the method of Lagrange multipliers. For each (scalar) constraint function $c_i(\mathbf{X})$ we add a term $\lambda_i c_i(\mathbf{X})$, where λ_i is a Lagrange multiplier. The Lagrange multiplier is an additional

free parameter of the system to be solved, i.e., $L(\mathbf{X}, \dot{\mathbf{X}}) \rightarrow L(\mathbf{X}, \dot{\mathbf{X}}, \lambda)$ and we also vary the action with respect to λ . For a vector of constraint functions $\mathbf{c}$ and Lagrange multipliers $\boldsymbol{\lambda}$ we have

$$L(\mathbf{X}, \dot{\mathbf{X}}, \boldsymbol{\lambda}) = \frac{1}{2} \dot{\mathbf{X}} \cdot M_L(\mathbf{X}) \dot{\mathbf{X}} - V(\mathbf{X}) + \boldsymbol{\lambda} \cdot \mathbf{c}(\mathbf{X}). \quad (3.30)$$

Varying with respect to the Lagrange multipliers will just give back the constraint equations

$$\mathbf{c}(\mathbf{X}) = 0, \quad (3.31)$$

and the Euler-Lagrange equations pick up an additional term $\left(\frac{\partial \mathbf{c}}{\partial \mathbf{X}}\right)^T \boldsymbol{\lambda}$

$$\frac{d}{dt} \left(M_L(\mathbf{X}) \dot{\mathbf{X}} \right) = -\frac{\partial V}{\partial \mathbf{X}} + \frac{1}{2} \dot{\mathbf{X}} \cdot \frac{\partial M_L}{\partial \mathbf{X}} \dot{\mathbf{X}} + \left(\frac{\partial \mathbf{c}}{\partial \mathbf{X}} \right)^T \boldsymbol{\lambda}. \quad (3.32)$$

This additional term is identified as the generalized constraint force

$$\mathbf{F}_c = \left(\frac{\partial \mathbf{c}}{\partial \mathbf{X}} \right)^T \boldsymbol{\lambda}. \quad (3.33)$$

For linear degrees of freedom we have for each constraint i a set of forces per body j of

$$\mathbf{f}_{ij} = \left(\frac{\partial c_i}{\partial \mathbf{x}_j} \right)^T \lambda_i. \quad (3.34)$$

For angular degrees of freedom we go back into velocity space (rather than $\dot{\mathbf{X}}$ space) using the Euler-Poincaré formulation.

Now, with the addition of $\boldsymbol{\lambda} \cdot \mathbf{c}(\mathbf{X})$ to the Lagrangian, the variation of the action (3.22) becomes

$$\delta S = \int dt \left(-\frac{d}{dt} (I\boldsymbol{\omega}) - \frac{\partial V}{\partial \boldsymbol{\theta}} J_L^{-1} + \left(\frac{\partial \mathbf{c}}{\partial \boldsymbol{\theta}} J_L^{-1} \right)^T \boldsymbol{\lambda} \right) \cdot \boldsymbol{\eta}, \quad (3.35)$$

and the equation of motion becomes

$$\frac{d}{dt} (I\boldsymbol{\omega}) = \boldsymbol{\tau}_w + (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}. \quad (3.36)$$

The derivative $\mathcal{D}\mathbf{c}$ of a vector of constraint functions is

$$\mathcal{D}\mathbf{c} = \begin{pmatrix} \frac{\partial c_1}{\partial \mathbf{x}_1} & \frac{\partial c_1}{\partial \boldsymbol{\theta}_1} J_L(\boldsymbol{\theta}_1)^{-1} & \dots & \frac{\partial c_1}{\partial \mathbf{x}_N} & \frac{\partial c_1}{\partial \boldsymbol{\theta}_N} J_L(\boldsymbol{\theta}_N)^{-1} \\ \vdots & & \ddots & & \\ \frac{\partial c_M}{\partial \mathbf{x}_1} & \frac{\partial c_M}{\partial \boldsymbol{\theta}_1} J_L(\boldsymbol{\theta}_1)^{-1} & \dots & \frac{\partial c_M}{\partial \mathbf{x}_N} & \frac{\partial c_M}{\partial \boldsymbol{\theta}_N} J_L(\boldsymbol{\theta}_N)^{-1} \end{pmatrix}. \quad (3.37)$$

Using this notation, we can summarize the equations of motion with constraints as

$$\frac{d}{dt} (M(\mathbf{X})\mathcal{V}) = \mathbf{F} + (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}. \quad (3.38)$$

The result of correctly using the left-trivialized derivative $\mathcal{D}\mathbf{c}$ is that the angular equations are in the space of angular velocity, rather than the axis/angle space.

3.3 Velocity-Level and Non-Holonomic Constraints

Velocity-level constraints appear in several forms in rigid body simulation. The classical non-holonomic case is a kinematic constraint on the allowed velocities, typically of the form $\mathbf{a}(\mathbf{X}) \cdot \dot{\mathbf{X}} = 0$. These are handled differently than holonomic constraints. As described in [9] and [19], if you try to add an ideal non-holonomic constraint directly to the Lagrangian and take the variation, as we did with holonomic constraints, you get the incorrect answer. The reason is that, as the Lagrange multipliers represent physical constraint forces, substituting the constraints into the Lagrangian before differentiation effectively assumes that the constraint forces can be derived from a potential, which is not necessarily true for non-holonomic systems.

Instead, we can use the Lagrange-d'Alembert Principle. This states that the non-conservative constraint forces do no virtual work. This modifies our Lagrangian variation by adding an extra term to enforce this condition

$$\delta \int L(\dot{\mathbf{X}}, \mathbf{X}) dt + \int \mathbf{Q} \cdot \delta \mathbf{X} = 0 \quad (3.39)$$

where $\mathbf{Q}$ are the non-conservative constraint forces.

If we have a non-holonomic constraint of the form $c(\dot{\mathbf{X}}, \mathbf{X}) = \mathbf{a}(\mathbf{X}) \cdot \dot{\mathbf{X}} = 0$ this means that we are preventing displacement in the direction $\mathbf{a}(\mathbf{X})$, so the constraint force $\mathbf{Q}$ must be proportional to that direction $Q_i = \lambda_i \mathbf{a}_i(\mathbf{X})$.

Therefore, these non-holonomic constraints can be added to the system as

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\mathbf{X}}} \right) - \frac{\partial L}{\partial \mathbf{X}} = \mathbf{a} \cdot \boldsymbol{\lambda} = \left(\frac{\partial \mathbf{c}}{\partial \dot{\mathbf{X}}} \right)^T \boldsymbol{\lambda}. \quad (3.40)$$

I.e. exactly the same form as the holonomic constraints, but with the constraint function differentiated with respect to the velocity $\dot{\mathbf{X}}$.

Note that, as with the previous section, we have to be careful about distinguishing between differentiation with respect to the axis/angle variable $\dot{\boldsymbol{\theta}}$ and the angular velocity $\boldsymbol{\omega}$. The constraint torque needs to be multiplied by J_L^{-T} in order to bring it to world space

$$\boldsymbol{\tau}_w = J_L^{-T} \left(\frac{\partial \mathbf{c}}{\partial \dot{\boldsymbol{\theta}}} \right)^T \boldsymbol{\lambda} \quad (3.41)$$

But also

$$\frac{\partial \mathbf{c}}{\partial \dot{\boldsymbol{\theta}}} = \frac{\partial \mathbf{c}}{\partial \boldsymbol{\omega}} \frac{\partial \boldsymbol{\omega}}{\partial \dot{\boldsymbol{\theta}}} = \frac{\partial \mathbf{c}}{\partial \boldsymbol{\omega}} J_L \quad (3.42)$$

since $\boldsymbol{\omega} = J_L \dot{\boldsymbol{\theta}}$. So the constraint torque in world space is

$$\left(\frac{\partial \mathbf{c}}{\partial \boldsymbol{\omega}} \right)^T \boldsymbol{\lambda}. \quad (3.43)$$

As before with the holonomic constraints, we extend the definition of $\mathcal{D}_V$ to include linear derivatives.

The left-trivialized velocity derivative is defined as

$$\mathcal{D}_V \mathbf{c} = \begin{pmatrix} \frac{\partial c_1}{\partial \dot{\mathbf{x}}_1} & \frac{\partial c_1}{\partial \dot{\boldsymbol{\omega}}_1} & \cdots & \frac{\partial c_1}{\partial \dot{\mathbf{x}}_N} & \frac{\partial c_1}{\partial \dot{\boldsymbol{\omega}}_N} \\ \vdots & & & & \\ \frac{\partial c_M}{\partial \dot{\mathbf{x}}_1} & \frac{\partial c_M}{\partial \dot{\boldsymbol{\omega}}_1} & \cdots & \frac{\partial c_M}{\partial \dot{\mathbf{x}}_N} & \frac{\partial c_M}{\partial \dot{\boldsymbol{\omega}}_N} \end{pmatrix}. \quad (3.44)$$

With this, the constrained equations of motion for both holonomic and ideal non-holonomic constraints are

$$\frac{d}{dt} (M(\mathbf{X})\mathcal{V}) = \mathbf{F} + (\mathcal{D}\mathbf{c}_h)^T \boldsymbol{\lambda}_h + (\mathcal{D}_V \mathbf{c}_{nh})^T \boldsymbol{\lambda}_{nh}. \quad (3.45)$$

The same velocity-space Jacobian and effective-mass algebra will also be useful later for restitution and friction. Those are impact and dissipation laws rather than ideal non-holonomic constraints in the geometric-mechanics sense, but they fit naturally into the same velocity-level solver machinery.

4 Solving the Constrained Equations of Motion

In this section we show how to solve the system of equations of motion as a first order approximation and show the relationship between this solution and other approaches, including PBD, sequential impulses and discrete variational mechanics.

4.1 Solving the Unconstrained Equations of Motion

We solve the constrained equations of motion incrementally as a first order approximation to the solution to the unconstrained (zero constraint forces) equations, so first we will briefly cover integrating the unconstrained system.

Finding the incremental time advancement of the unconstrained equations of motion to find the provisional state $\mathbf{X}^{t+1}$, $\dot{\mathbf{X}}^{t+1}$ can be done with a standard numerical integration scheme. A common choice in real-time rigid body simulation is a semi-implicit (symplectic) Euler integration scheme. This is a simple first order scheme with low cost, high stability, and stable long-term energy behaviour compared with explicit Euler. In this scheme, first the velocity is updated, then the new velocity is used to update the position.

For linear degrees of freedom the equation of motion is

$$m\ddot{\mathbf{x}} = \mathbf{f} \quad (4.1)$$

and the discrete integration step for symplectic Euler is

$$\begin{aligned} \dot{\mathbf{x}}^{t+1} &= \dot{\mathbf{x}}^t + \frac{1}{m} \mathbf{f} \Delta t, \\ \mathbf{x}^{t+1} &= \mathbf{x}^t + \dot{\mathbf{x}}^{t+1} \Delta t. \end{aligned} \quad (4.2)$$

For angular degrees of freedom the equation of motion is

$$\frac{d}{dt}(I\boldsymbol{\omega}) = \boldsymbol{\tau} \quad (4.3)$$

and the discrete integration step for symplectic Euler is

$$\begin{aligned} \boldsymbol{\omega}^{t+1} &= \boldsymbol{\omega}^t + I^{-1} (\boldsymbol{\tau} - \boldsymbol{\omega}^t \times (I\boldsymbol{\omega}^t)) \Delta t, \\ \mathbf{q}^{t+1} &= e^{\frac{1}{2}\boldsymbol{\Omega}^{t+1}\Delta t} \mathbf{q}^t = e^{-\frac{1}{2}\mathcal{I}\boldsymbol{\omega}^{t+1}\Delta t} \mathbf{q}^t \end{aligned} \quad (4.4)$$

where the inertia $I(t)$ is evaluated at the current configuration at t and we are using the quaternion representation for orientation. We could equally choose to use the rotation matrix representation where

$$\mathbf{R}^{t+1} = e^{[\boldsymbol{\omega}^{t+1}\Delta t] \times} \mathbf{R}^t \approx (\mathbb{1} + [\boldsymbol{\omega}^{t+1}\Delta t] \times) \mathbf{R}^t \quad (4.5)$$

but prefer the quaternion approach as it is important to maintain orthogonality / normalization of the rotations, and that is more efficient with quaternions, as discussed in section §2.2.

Note that, for simplicity, we have shown the gyroscopic term $\boldsymbol{\omega} \times I\boldsymbol{\omega}$ evaluated at time t , i.e. it is treated explicitly. Unlike the linear equation of motion, this term is nonlinear in the angular velocity and couples the rotational degrees of freedom when the inertia is non-spherical. If the angular velocity is high relative to the timestep, this explicit treatment can therefore cause instability. In

practical rigid body simulation it is therefore common either to integrate angular momentum or to treat the gyroscopic term implicitly [23, 12]. Catto, for example, describes a practical implicit Euler treatment in the body frame solved with a small number of Newton iterations. The particular choice of rotational integration scheme is largely orthogonal to the constraint formulation developed here. In what follows we only require that a solution at $t+1$ can be found that is first order accurate in Δt . The particular integration scheme used to obtain this provisional unconstrained state does not affect the subsequent constraint derivation.

4.2 Finding a First Order Approximate Solution

Now, given the solution to the unconstrained equations of motion, we assume that the solution to the constrained equations at $t+1$ is given by a small perturbation $\Delta \mathbf{X}$ away from the solution to the unconstrained equations at $t+1$. I.e. for $\boldsymbol{\lambda} = 0$ we solve the unconstrained equations to find $\mathbf{X}^{t+1}, \dot{\mathbf{X}}^{t+1}$ using a standard rigid body integration scheme, then we apply a perturbation $\Delta \mathbf{X}$ where

$$\Delta \mathbf{X} = \begin{pmatrix} \Delta \mathbf{x}_1 \\ \Delta \phi_1 \\ \vdots \\ \Delta \mathbf{x}_N \\ \Delta \phi_N \end{pmatrix} \quad (4.6)$$

and we define

$$\Delta \phi_i = J_L(\boldsymbol{\theta}_i) \Delta \boldsymbol{\theta}_i. \quad (4.7)$$

This is interpreted as an angular displacement: the world-space rotational displacement associated with $\Delta \boldsymbol{\theta}$. The corresponding angular velocity correction is $\Delta \phi / \Delta t$.

With this definition, the updated configuration is

$$\begin{aligned} \mathbf{x}_i^{t+1} &\rightarrow \mathbf{x}_i^{t+1} + \Delta \mathbf{x}_i, & R_i^{t+1} &\rightarrow e^{[\Delta \phi_i]_{\times}} R_i^{t+1}, \\ \mathcal{V}_i^{t+1} &\rightarrow \mathcal{V}_i^{t+1} + \Delta \mathbf{X}_i / \Delta t. \end{aligned} \quad (4.8)$$

We solve the constrained equations to first order in $\Delta \mathbf{X}$ and $\boldsymbol{\lambda}$ to find an approximate solution to the constrained equations at $t+1$. As long as the time step is small (making the displacement due to applying the constraint forces over Δt small) then this remains a valid approximation.

Start with the linear degrees of freedom. We have

$$\begin{aligned} \frac{1}{\Delta t} (m(\dot{\mathbf{x}}^{t+1} \Delta t + \Delta \mathbf{x}) - m\dot{\mathbf{x}}^{t+1} \Delta t) &= \mathbf{f}(\mathbf{x}^{t+1} + \Delta \mathbf{x}) \Delta t + (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t \\ \Rightarrow \tilde{M} \Delta \mathbf{x} &= (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2, \end{aligned} \quad (4.9)$$

where

$$\tilde{M} = m\mathbb{1} - \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \Delta t^2 \quad (4.10)$$

as we have assumed that the unconstrained equation is satisfied at $(\mathbf{x}^{t+1}, \dot{\mathbf{x}}^{t+1})$. Here $\tilde{M}$ is a mass term including the derivative of any force that we choose to linearize together with the constraint solve. For the PBD derivation below we treat the non-constraint forces as part of the provisional unconstrained step and use $\tilde{M} = m\mathbb{1}$; the general implicit-force case is developed separately in section §8.1.

Now expand the constraint equation, as this must be satisfied at $\mathbf{x}^{t+1} + \Delta\mathbf{x}$

$$\mathbf{c}(\mathbf{x}^{t+1} + \Delta\mathbf{x}) = \mathbf{c}(\mathbf{x}^{t+1}) + (\mathcal{D}\mathbf{c})\Delta\mathbf{x} + \mathcal{O}(\Delta\mathbf{x})^2 = 0. \quad (4.11)$$

Taking these together we find an equation for $\boldsymbol{\lambda}$

$$\left(\mathcal{D}\mathbf{c} \tilde{M}^{-1}(\mathcal{D}\mathbf{c})^T\right) \boldsymbol{\lambda} \Delta t^2 = -\mathbf{c}(\mathbf{x}^{t+1}). \quad (4.12)$$

We solve this for $\boldsymbol{\lambda}$ and then obtain $\Delta\mathbf{x}$ from

$$\Delta\mathbf{x} = \tilde{M}^{-1}(\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2. \quad (4.13)$$

Now we do the same for the angular equations of motion. Start with

$$\frac{d}{dt} (I(\boldsymbol{\theta})\boldsymbol{\omega}) = \boldsymbol{\tau}(\boldsymbol{\theta}) + (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}. \quad (4.14)$$

Expanding, using the assumption that $\boldsymbol{\theta}^{t+1}, \dot{\boldsymbol{\theta}}^{t+1}$ solve the unconstrained equation, we get

$$\frac{1}{\Delta t} (I(\boldsymbol{\theta}^{t+1} + \Delta\boldsymbol{\theta})(\boldsymbol{\omega}^{t+1}\Delta t + \Delta\boldsymbol{\phi}) - I(\boldsymbol{\theta}^{t+1})\boldsymbol{\omega}^{t+1}\Delta t) = \boldsymbol{\tau}_w(\boldsymbol{\theta}^{t+1} + \Delta\boldsymbol{\theta})\Delta t + (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t$$

$\Rightarrow$

$$\begin{aligned} & (\mathbb{1} + [\Delta\boldsymbol{\phi}]_{\times})I(\boldsymbol{\theta}^{t+1})(\mathbb{1} - [\Delta\boldsymbol{\phi}]_{\times})(\boldsymbol{\omega}^{t+1}\Delta t + \Delta\boldsymbol{\phi}) - I(\boldsymbol{\theta}^{t+1})\boldsymbol{\omega}^{t+1}\Delta t \\ & = \boldsymbol{\tau}_w(\boldsymbol{\theta}^{t+1})\Delta t^2 + (\mathcal{D}\boldsymbol{\tau})^T \Delta\boldsymbol{\phi}\Delta t^2 + (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2. \end{aligned} \quad (4.15)$$

which simplifies to

$$\tilde{I}\Delta\boldsymbol{\phi} = (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2, \quad (4.16)$$

Where

$$\tilde{I} = I + I[\boldsymbol{\omega}^{t+1}\Delta t]_{\times} - [I\boldsymbol{\omega}^{t+1}\Delta t]_{\times} - (\mathcal{D}\boldsymbol{\tau})^T \Delta t^2 \quad (4.17)$$

is the modified inertia matrix. Note that for spherical inertia or for small $\boldsymbol{\omega}^{t+1}\Delta t$, and if the torque $\boldsymbol{\tau}$ is constant, this reduces to the standard world space inertia. The angular velocity modification is a result of the precession term in the equations of motion and means, for instance, a spinning body configured as a gyroscope will be more difficult to rotate when applying the $\Delta\boldsymbol{\theta}$ due to the constraint correction.

In practice, this term can usually be ignored, as the time step Δt is taken to be small and so $|\boldsymbol{\omega}\Delta t| \ll 1$ and usually inertias in rigid body simulation are reasonably uniform $I_{ii}/I_{jj} \sim 1$. These assumptions break down for fast spinning objects with highly non-spherical inertia, where gyroscopic effects are important. In this case, although the motion can be correctly modelled if we include the $\boldsymbol{\omega} \times I\boldsymbol{\omega}$ term in the integration, constraint interactions will be incorrect. Dropping the extra terms means, for example, that a constraint impulse applied to a fast spinning body produces a simple tilt rather than the precession that the gyroscopic coupling should give, with an error of order $\|\boldsymbol{\omega}\Delta t\|$ per step.

A perhaps more important argument for ignoring this term is that its addition makes the mass matrix non-symmetric, which means that the matrix $\mathcal{D}\mathbf{c} I^{-1}(\mathcal{D}\mathbf{c})^T$ is not necessarily symmetric. This means that the system is no longer guaranteed to converge, as we explore in the next section on convergence §4.5.

For these reasons, we will omit this correction term in our analysis from now on, and assume $\tilde{I} = I$. It is easy to add it back though by just replacing I with $\tilde{I}$, but the convergence analysis will not be valid in this case.

Back to the constraint function. We again expand to first order

$$\mathbf{c}(\boldsymbol{\theta}^{t+1} + \Delta\boldsymbol{\theta}) = \mathbf{c}(\boldsymbol{\theta}^{t+1}) + \mathcal{D}\mathbf{c} \Delta\boldsymbol{\phi} + \mathcal{O}(\Delta t^2) = 0, \quad (4.18)$$

and solving gives

$$\begin{aligned} \left(\mathcal{D}\mathbf{c} I^{-1} (\mathcal{D}\mathbf{c})^T \right) \boldsymbol{\lambda} \Delta t^2 &= -\mathbf{c}(\boldsymbol{\theta}^{t+1}), \\ \Delta\boldsymbol{\phi} &= I^{-1} (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2. \end{aligned} \quad (4.19)$$

This brings the angular and linear equations into the same form. We can then combine all degrees of freedom together into a single linear system.

First solve the following linear system to find $\boldsymbol{\lambda}$

$$\left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right) \boldsymbol{\lambda} \Delta t^2 = -\mathbf{c}(\mathbf{X}^{t+1}) \quad (4.20)$$

and then use this to solve for $\Delta\mathbf{X}$

$$\Delta\mathbf{X} = M^{-1} (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2, \quad (4.21)$$

This equation for $\boldsymbol{\lambda}$ which is then used to solve for $\Delta\mathbf{X}$ is a linear system that can be solved using a number of numerical methods.

One approximation in this result is worth making explicit. We have linearized the constraint function, but when relating the multiplier to the displacement we have treated the constraint force term $(\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}$ as fixed. A full Newton linearization would also differentiate the Jacobian, producing the tangent term

$$(\boldsymbol{\lambda} \cdot \mathcal{D}^2 \mathbf{c}) \Delta\mathbf{X}. \quad (4.22)$$

This is the *geometric stiffness* term. Omitting it gives the Gauss–Newton form used by the standard PBD effective-mass update. We derive this term and the corresponding constraint Hessian in section §9, before returning to its role in the augmented Lagrangian and AVBD formulation in section §10.1.

4.3 An Iterative Solution Using Gauss–Seidel

We now turn to the method for solving the system of equations iteratively. We need to solve $\boldsymbol{\lambda}$ from

$$\left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right) \boldsymbol{\lambda} \Delta t^2 = -\mathbf{c}(\mathbf{X}^{t+1}), \quad (4.23)$$

which can then be used to compute $\Delta\mathbf{X}$ using

$$\Delta\mathbf{X} = M^{-1} (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2. \quad (4.24)$$

The equation is a linear system $A\boldsymbol{\lambda} = -\mathbf{c}$, where $A = \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T\right) \Delta t^2$ is a square matrix. Writing out the rows explicitly, this is

$$\begin{aligned} A_{11}\lambda_1 + A_{12}\lambda_2 + \cdots + A_{1m}\lambda_m &= -c_1, \\ A_{21}\lambda_1 + A_{22}\lambda_2 + \cdots + A_{2m}\lambda_m &= -c_2, \\ &\vdots \\ A_{m1}\lambda_1 + A_{m2}\lambda_2 + \cdots + A_{mm}\lambda_m &= -c_m \end{aligned}$$

which can be solved using the Gauss–Seidel method. See, for example, [42].

We start with an initial guess for $\boldsymbol{\lambda}^0$, say $\boldsymbol{\lambda}^{(0)} = 0$, then solve each equation i in turn for λ_i using the latest value from previous solutions, and then iterate to a solution:

$$\begin{aligned} \lambda_1^{(1)} &= \frac{1}{A_{11}} \left(-c_1 - A_{12}\lambda_2^{(0)} - \cdots - A_{1m}\lambda_m^{(0)} \right), \\ \lambda_2^{(1)} &= \frac{1}{A_{22}} \left(-c_2 - A_{21}\lambda_1^{(1)} - A_{23}\lambda_3^{(0)} \cdots - A_{2m}\lambda_m^{(0)} \right), \\ &\vdots \\ \lambda_i^{k+1} &= \frac{1}{A_{ii}} \left(-c_i - \sum_{j<i} A_{ij}\lambda_j^{k+1} - \sum_{j>i} A_{ij}\lambda_j^k \right), \\ &= \lambda_i^k - \frac{1}{A_{ii}} (\mathbf{c} + A\boldsymbol{\lambda}^*)_i, \end{aligned} \tag{4.25}$$

where $\boldsymbol{\lambda}^*$ is the latest value of all of the λ s. For our system this is

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} \left(\mathbf{c}(\mathbf{X}^{t+1}) + \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right) \boldsymbol{\lambda}^* \Delta t^2 \right)_i. \tag{4.26}$$

Using (4.24) we can update the displacements incrementally per iteration

$$\Delta \mathbf{X}'_j = \Delta \mathbf{X}_j + M_j^{-1} (\mathcal{D}\mathbf{c})_{ij}^T (\boldsymbol{\lambda}^{k+1} - \boldsymbol{\lambda}^k)_i \Delta t^2 \tag{4.27}$$

and use this to rewrite the update for λ_i as

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} \left(\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta \mathbf{X} \right)_i. \tag{4.28}$$

Summary:

The equations of motion can be solved using a Gauss–Seidel update. We start with an initial guess for a solution for λ , typically $\lambda = 0$, and set all of the displacements $\Delta \mathbf{X} = 0$. Then for $k = 0$ to $N - 1$ iterations we solve one row at a time for each constraint’s Lagrange multiplier using the current value of the displacement

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \left(\mathcal{D} \mathbf{c} \ M^{-1} (\mathcal{D} \mathbf{c})^T \right)_{ii}^{-1} (\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D} \mathbf{c} \ \Delta \mathbf{X})_i. \quad (4.29)$$

and then update the current displacement

$$\Delta \mathbf{X}'_j = \Delta \mathbf{X}_j + M_j^{-1} (\mathcal{D} \mathbf{c})_{ij}^T (\lambda^{k+1} - \lambda^k)_i \Delta t^2. \quad (4.30)$$

In the end we are left with a set of displacements that resolve the constraints and can be used to update the configuration of the system to give a first order solution to the constrained equations of motion at time $t + 1$

$$\begin{aligned} \dot{\mathbf{x}}_i^{t+1} &\rightarrow \dot{\mathbf{x}}_i^{t+1} + \Delta \mathbf{x}_i / \Delta t, \\ \omega_i^{t+1} &\rightarrow \omega_i^{t+1} + \Delta \phi_i / \Delta t, \\ \mathbf{x}^{t+1} &\rightarrow \mathbf{x}_i^{t+1} + \Delta \mathbf{x}_i, \\ q(\boldsymbol{\theta}^{t+1}) &\rightarrow e^{-\frac{1}{2} \mathcal{I} \Delta \phi_i} q(\boldsymbol{\theta}^{t+1}) \end{aligned} \quad (4.31)$$

4.4 Comparison to the Jacobi Method

The Jacobi method is similar to the Gauss–Seidel method except that instead of feeding the result from the previous row into the solution for the next row, each row is solved independently using the solution from the previous iteration. In our system this means instead of the Gauss–Seidel update

$$\lambda_i^{k+1} = \frac{1}{A_{ii}} \left(-c_i - \sum_{j < i} A_{ij} \lambda_j^{k+1} - \sum_{j > i} A_{ij} \lambda_j^k \right), \quad (4.32)$$

we would have a Jacobi update of

$$\lambda_i^{k+1} = \frac{1}{A_{ii}} \left(-c_i - \sum_{j \neq i} A_{ij} \lambda_j^k \right). \quad (4.33)$$

The Jacobi update is known to converge slower than Gauss–Seidel, which we will demonstrate with a simple example in section §11, but as solving each λ is independent, the update can be parallelized much more easily than Gauss–Seidel, making it popular for GPU based simulations.

One known convergence problem for Jacobi is that, for instance, when solving a system with multiple contact constraints per body pair, as each constraint is solved independently, the system tends to over correct the position each iteration. A useful trick to alleviate this is *mass splitting* where the effective mass is divided by the number of constraint points [40] making the total per iteration correction correct.

4.5 Convergence

It's useful to look at the convergence criteria for the Gauss–Seidel update of the linear system (4.20). If we take the unmodified inertia, then the matrix $A = \mathcal{D}c M^{-1}(\mathcal{D}c)^T$ is symmetric positive semidefinite, since M is SPD. I.e. for any $\mathbf{x}$, $\mathbf{x} \cdot A\mathbf{x} \geq 0$. It is strictly positive definite (SPD) if $(\mathcal{D}c)^T \mathbf{x} = 0$ implies that $\mathbf{x} = 0$. I.e. $\mathcal{D}c$ has full row rank, meaning that all of the constraints are linearly independent. The Gauss–Seidel update will converge for any SPD matrix A , so our system is guaranteed to converge under this independence assumption. Redundant or degenerate constraints make A only semidefinite and require either regularization, removal of the redundant rows, or a solver that handles the null space explicitly.

To show this, we construct an energy function

$$E(\boldsymbol{\lambda}) = \frac{1}{2} \boldsymbol{\lambda}^T A \boldsymbol{\lambda} + \boldsymbol{\lambda}^T \mathbf{c}. \quad (4.34)$$

This is a convex function for SPD matrix A with a unique minimum at $A\boldsymbol{\lambda}^* = -\mathbf{c}$. Taking a single λ_i and fixing the energy for all of the other λ_j s the energy as a function of λ_i is

$$E(\lambda_i) = \frac{1}{2} A_{ii} |\lambda_i|^2 + \lambda_i \left(\sum_{j \neq i} A_{ij} \lambda_j + c_i \right), \quad (4.35)$$

and minimizing with respect to λ_i gives

$$\frac{\partial E(\lambda_i)}{\partial \lambda_i} = A_{ii} \lambda_i + \sum_{j \neq i} A_{ij} \lambda_j + c_i = \mathbf{r}_i = 0 \quad (4.36)$$

for a residual $\mathbf{r}_i$. The value of λ_i^{k+1} that sets this residual to zero is

$$\begin{aligned} \lambda_i^{k+1} &= -A_{ii}^{-1} \left(\sum_{j \neq i} A_{ij} \lambda_j^k + c_i \right), \\ &= \lambda_i^k - A_{ii}^{-1} (A\boldsymbol{\lambda} + \mathbf{c})_i, \end{aligned} \quad (4.37)$$

i.e. exactly the iterative update we defined in (4.29).

Using $e_i = (A\boldsymbol{\lambda} + \mathbf{c})_i$ we compute the change in energy

$$\begin{aligned} E(\lambda_i^{k+1}) - E(\lambda_i^k) &= \frac{1}{2} A_{ii} |\lambda_i^k - A_{ii}^{-1} e_i|^2 + (\lambda_i^k - A_{ii}^{-1} e_i) \left(\sum_{j \neq i} A_{ij} \lambda_j^k + c_i \right) \\ &\quad - \frac{1}{2} A_{ii} |\lambda_i^k|^2 + \lambda_i^k \left(\sum_{j \neq i} A_{ij} \lambda_j^k + c_i \right), \\ &= -\lambda_i^k e_i + \frac{1}{2} A_{ii}^{-1} |e_i|^2 - A_{ii}^{-1} e_i \left(\sum_{j \neq i} A_{ij} \lambda_j^k + c_i \right), \\ &= -\frac{1}{2} A_{ii}^{-1} |e_i|^2. \end{aligned} \quad (4.38)$$

Since A is SPD then $A_{ii} > 0$ and so the energy decreases each iteration. Combining this with the fact that the energy is bounded below by the minimum $A\boldsymbol{\lambda}^* = -\mathbf{c}$ means that the solution converges to $\boldsymbol{\lambda}^*$.

4.5.1 Rate of convergence

To look at the rate of convergence we write the update as

$$\lambda_i^{k+1} = A_{ii}^{-1} \left(-c_i - \sum_{j<i} A_{ij} \lambda_j^{k+1} - \sum_{j>i} A_{ij} \lambda_j^k \right) \quad (4.39)$$

as, with Gauss–Seidel, the result from solving a previous row of λ_j^{k+1} with $j < i$ is fed into the update for λ_i^{k+1} . So this is

$$A_{ii} \lambda_i^{k+1} + \sum_{j<i} A_{ij} \lambda_j^{k+1} = - \sum_{j>i} A_{ij} \lambda_j^k - c_i. \quad (4.40)$$

Writing the matrix A in terms of its diagonal D , its lower triangular matrix L and upper U , where $U = L^T$ due to A being symmetric, we have

$$(L + D) \boldsymbol{\lambda}^{k+1} = -L^T \boldsymbol{\lambda}^k - \mathbf{c} \quad (4.41)$$

so

$$\boldsymbol{\lambda}^{k+1} = -(L + D)^{-1} L^T \boldsymbol{\lambda}^k - (L + D)^{-1} \mathbf{c}. \quad (4.42)$$

Letting

$$\boldsymbol{\epsilon}^k = \boldsymbol{\lambda}^k - \boldsymbol{\lambda}^* \quad (4.43)$$

be the error at iteration k from the solution $\boldsymbol{\lambda}^*$ defined by $A \boldsymbol{\lambda}^* = -\mathbf{c}$, then

$$\boldsymbol{\epsilon}^{k+1} = -(D + L)^{-1} L^T \boldsymbol{\epsilon}^k. \quad (4.44)$$

So, for an initial error $\boldsymbol{\epsilon}_0$ the error goes like

$$\boldsymbol{\epsilon}^k = G^k \boldsymbol{\epsilon}_0 \quad (4.45)$$

for

$$G = -(D + L)^{-1} L^T. \quad (4.46)$$

This means that the convergence of the system depends on the spectral radius $\rho(G)$. Express $\boldsymbol{\epsilon}$ in terms of the basis of eigenvalues of G

$$\boldsymbol{\epsilon} = \sum_i c_i \mathbf{v}_i \quad (4.47)$$

for constants c_i and eigenvectors $\mathbf{v}_i$. From this, if α_i are the eigenvalues of G then

$$\boldsymbol{\epsilon}^k = \sum_i \alpha_i^k c_i \mathbf{v}_i. \quad (4.48)$$

So the error becomes dominated by the largest eigenvalue α_i . This needs to be less than one for the error to shrink, hence the convergence criterion is

$$\rho(G) < 1 \quad (4.49)$$

and how fast the system will converge depends on how small the spectral radius of G is.

In practice, it is often more practical to use the diagonal dominance of the matrix A in order to give an intuition on the rate of convergence of the system. Strict diagonal dominance is defined as

$$|A_{ii}| > \sum_{j \neq i} |A_{ij}|. \quad (4.50)$$

The claim is that for a strictly diagonally dominant matrix A the Gauss–Seidel update will converge, and the more diagonally dominant, the faster the rate of convergence. To see this consider the update for the first row

$$\epsilon_1^{k+1} = -A_{11}^{-1} \sum_{j>1} A_{1j} \epsilon_j^k. \quad (4.51)$$

Taking absolute values

$$\begin{aligned} |\epsilon_1^{k+1}| &\leq |A_{11}|^{-1} \sum_{j>1} |A_{1j}| |\epsilon_j^k|, \\ &\leq r_1 \|\epsilon^k\|_{\max}, \end{aligned} \quad (4.52)$$

where

$$r_1 = \frac{\sum_{j>1} |A_{1j}|}{|A_{11}|}, \quad (4.53)$$

which for a diagonally dominant matrix is less than 1, and $\|\epsilon^k\|_{\max}$ is the maximum error from the previous iteration. So

$$|\epsilon_1^{k+1}| < \|\epsilon^k\|_{\max}. \quad (4.54)$$

The error for the second row is

$$\epsilon_2^{k+1} = -A_{22}^{-1} \left(A_{21} \epsilon_1^{k+1} + \sum_{j>2} A_{2j} \epsilon_j^k \right). \quad (4.55)$$

Taking absolutes

$$\begin{aligned} |\epsilon_2^{k+1}| &\leq |A_{22}|^{-1} \left(|A_{21}| |\epsilon_1^{k+1}| + \sum_{j>2} |A_{2j}| |\epsilon_j^k| \right), \\ &\leq \left(\frac{|A_{21}|}{|A_{22}|} + \frac{\sum_{j>2} |A_{2j}|}{|A_{22}|} \right) \|\epsilon^k\|_{\max}, \end{aligned} \quad (4.56)$$

where we have used the previous result for ϵ_1^{k+1} .

Continuing, row by row we get

$$|\epsilon_i^{k+1}| \leq \frac{\sum_{j \neq i} |A_{ij}|}{|A_{ii}|} \|\epsilon^k\|_{\max} = r_i \|\epsilon^k\|_{\max} \quad (4.57)$$

and overall

$$\|\epsilon^{k+1}\|_{\max} \leq r \|\epsilon^k\|_{\max} \quad (4.58)$$

where

$$r = \max_i \frac{\sum_{j \neq i} |A_{ij}|}{|A_{ii}|}. \quad (4.59)$$

So, if A is strictly diagonally dominant then $r < 1$ and each iteration the error is reduced, hence the system converges.

Summary:

The Gauss–Seidel algorithm for finding the solution to $A\mathbf{x} = \mathbf{b}$ is guaranteed to converge for any symmetric positive definite matrix A . For the constrained equations of motion $A = \mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T$ is symmetric positive semidefinite in general, and positive definite when the constraints are linearly independent, so that $\mathcal{D}\mathbf{c}$ has full row rank. Redundant constraints make A only positive semidefinite. If they are consistent, as with several contact points on a resting face, Gauss–Seidel still converges: $\Delta\mathbf{X}$ is unique, while $\boldsymbol{\lambda}$ converges to one of many equivalent solutions depending on the ordering and initial guess. If they conflict, so that no $\Delta\mathbf{X}$ satisfies them all, the multipliers grow without bound and the displacements do not settle. This is the case that needs regularization (§4.8) or removal of the conflicting rows.

The rate of convergence depends on the spectral radius $\rho(G)$ of the Gauss–Seidel iteration matrix $G = -(D+L)^{-1}L^T$, where we split $A = (D+L+L^T)$ into its diagonal, lower triangular and upper triangular parts. The per iteration error goes like

$$\|\epsilon^k\| \lesssim \rho(G)^k \|\epsilon_0\|. \quad (4.60)$$

Diagonal dominance is determined by the ratio

$$r = \max_i \frac{\sum_{j \neq i} |A_{ij}|}{|A_{ii}|}. \quad (4.61)$$

The more diagonally dominant the matrix is (the smaller the value of r) the faster the rate of convergence.

4.6 A More Accurate Update Using Non-Linear Gauss–Seidel

We can improve on the result (4.29) by noticing that the term $\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta\mathbf{X}$ is the first order expansion of $\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X})$, i.e. the constraint function evaluated at the most recent value of the displacement $\Delta\mathbf{X}$.

In order to obtain a more accurate update, we can replace the linearized term $\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta\mathbf{X}$ with the full constraint evaluation at the latest version of $\Delta\mathbf{X}$.

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} \mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X})_i, \quad (4.62)$$

This turns the update from a linear solution to (4.23) into a non-linear iterative algorithm.

This is now straying from the standard Gauss–Seidel method, but it is demonstrated to converge better when applied in [29]. This non-linear update is particularly important when solving rigid body systems with rotation when the time step is large, as errors due to using a linear approximation to rotations can be large, leading to inaccuracies in the solution and instability.

Another thing that is often done to improve the convergence of the result is to recompute the constraint derivatives in the “resistance” term $\left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1}$ for each iteration. However,

this is often expensive, and having an accurate measure of the error is the more important factor in improving stability and the robustness of the result in the case of large angular velocities and/or time steps.

Given that this update strays from the linearized Gauss–Seidel update, we can’t use the same convergence arguments that we did in the previous section. However, if the constraints are smooth then the constraint function is well approximated by its first order approximation near the solution and the nonlinear Gauss–Seidel update converges asymptotically to the linear Gauss–Seidel update and the local convergence analysis holds. Near the solution, provided the linearisation remains accurate and the effective system remains sufficiently diagonally dominant, the same argument suggests that the system will remain convergent.

What we gain by this method though is that we are no longer accumulating linearization error, and so the frame to frame stability in cases of large angular velocity is greatly improved. This is the practical reason for the better convergence observed in [29].

4.7 Adding Velocity-Level Constraints

We saw in 3.3 that ideal non-holonomic constraints follow the holonomic constraint formulation but now with the velocity-space constraint force

$$(\mathcal{D}_{\mathcal{V}}c)^T \boldsymbol{\lambda}. \quad (4.63)$$

The derivation of the linearized update directly follows the holonomic constraint derivation, leading to a per iteration update of

$$\begin{aligned} \lambda_i^{k+1} \Delta t &= \lambda_i^k \Delta t - \left(\mathcal{D}_{\mathcal{V}}c \, M^{-1} (\mathcal{D}_{\mathcal{V}}c)^T \right)_{ii}^{-1} \mathbf{c}(\dot{\mathbf{X}} + \Delta \mathbf{X}^k / \Delta t)_i, \\ \Delta \mathbf{X}^{k+1} &= \Delta \mathbf{X}^k + M^{-1} (\mathcal{D}_{\mathcal{V}}c)^T (\lambda^{k+1} - \lambda^k) \Delta t^2. \end{aligned} \quad (4.64)$$

So with this update we can see that velocity constraints and position constraints can be part of the same solver loop, since they are both incrementally updating the displacement $\Delta \mathbf{X}$. Alternatively, we can treat the velocity solve as being separate from the position solve and solve the position constraints first, then solve the velocity constraints, with the $\Delta \mathbf{X}$ ’s from the velocity solve only updating the velocities and not the positions.

4.8 Regularization

A useful extension to the method of Lagrange multipliers is the method of regularization, achieved by applying a quadratic term in the Lagrange multiplier to the Lagrangian. The idea is to add a term $\frac{1}{2} \boldsymbol{\lambda} \cdot \mathbf{E} \boldsymbol{\lambda}$ to the Lagrangian with $\mathbf{E} = \text{diag}\{\epsilon_1, \dots, \epsilon_M\}$ being a diagonal matrix of small constant values that adds a penalty term to the Lagrange multipliers ensuring that they, and hence the constraint forces, remain small.

$$L(\mathbf{X}, \dot{\mathbf{X}}, \boldsymbol{\lambda}) \rightarrow \frac{1}{2} \dot{\mathbf{X}} \cdot M(\mathbf{X}) \dot{\mathbf{X}} - V(\mathbf{X}) + \boldsymbol{\lambda} \cdot \mathbf{c}(\mathbf{X}) + \frac{1}{2} \boldsymbol{\lambda} \cdot \mathbf{E} \boldsymbol{\lambda}. \quad (4.65)$$

This new term alters the constraint equation

$$\mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) = \mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \, \Delta \mathbf{X} + \mathbf{E} \boldsymbol{\lambda} = 0. \quad (4.66)$$

The result of regularization is that equation to solve for λ now becomes

$$\left(\epsilon_i \mathbf{1} + \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii} \lambda_i \Delta t^2 = -c_i(\mathbf{X}^{t+1}). \quad (4.67)$$

for some small ϵ_i

This additional term ensures that if $\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T$ becomes singular, for instance if there are degenerate constraints, then the inversion of the matrix for λ does not also become singular. This is particularly important if the matrix solver uses a direct solve for all or part of the matrix.

This type of regularization is generally called *Tikhonov Regularization* [38], and is equivalent to compliance in XPBD, as we will see later in §8.4.1.

Algorithm 1 Basic Solver Update

- 1: **Input:** Predicted state $\mathbf{X}^{t+1}$, mass matrix M , timestep Δt
- 2: **(Optional) Warm Starting:** $\lambda^0 = \eta \lambda^{\text{prev}}$, for $0 \leq \eta \leq 1$, $\Delta \mathbf{X}^0 = M^{-1} (\mathcal{D}\mathbf{c})^T \lambda^0 \Delta t^2$
- 3: **for** each constraint i **do**
- 4: Compute the constraint derivative $\mathcal{D}\mathbf{c}$ and effective mass

$$\mu_i = \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1}$$

- 5: **end for**
- 6: **for** $k = 0$ to $N - 1$ **do**
- 7: **for** each constraint i **do**
- 8: Compute the Lagrange multiplier update using the latest value of the constraint error

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \xi \mu_i c(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k)$$

- 9: Update the incremental displacement

$$\Delta \mathbf{X}_j^{k+1} = \Delta \mathbf{X}_j^k + M_j^{-1} (\mathcal{D}\mathbf{c})_{ij} (\lambda_i^{k+1} - \lambda_i^k) \Delta t^2$$

- 10: **end for**
- 11: **end for**
- 12: **State Update:**

$$\begin{aligned} \mathbf{x}^{t+1} &\rightarrow \mathbf{x}^{t+1} + \Delta \mathbf{x} & \dot{\mathbf{x}}^{t+1} &\rightarrow \dot{\mathbf{x}}^{t+1} + \Delta \mathbf{x} / \Delta t \\ q^{t+1} &\rightarrow e^{-\frac{1}{2} \mathcal{I} \Delta \phi} q^{t+1} & \omega^{t+1} &\rightarrow \omega^{t+1} + \Delta \phi / \Delta t \end{aligned}$$

5 Interpretation and Connections to Other Known Methods

5.1 Moore-Penrose Inverse and Least Squares Minimization

The Moore-Penrose inverse [30] (also called the pseudoinverse) of a matrix A^+ is a generalization of the matrix inverse for non-square matrices. For a full row rank matrix A (so that (AA^T) is invertible) this is defined as

$$A^+ = A^T(AA^T)^{-1}. \quad (5.1)$$

The Lagrange multiplier method gave us a solution for $\Delta \mathbf{X}$ of

$$\Delta \mathbf{X} = -M^{-1}(\mathcal{D}\mathbf{c})^T \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)^{-1} \mathbf{c}(\mathbf{X}^{t+1}). \quad (5.2)$$

If M were equal to the identity then this would reduce to

$$\Delta \mathbf{X} = -(\mathcal{D}\mathbf{c})^T \left(\mathcal{D}\mathbf{c} (\mathcal{D}\mathbf{c})^T \right)^{-1} \mathbf{c}(\mathbf{X}^{t+1}). \quad (5.3)$$

This is exactly the Moore-Penrose inverse solution to the constraint equation at first order in $\Delta \mathbf{X}$

$$\mathcal{D}\mathbf{c} \Delta \mathbf{X} = -\mathbf{c}(\mathbf{X}^{t+1}). \quad (5.4)$$

So the method of Lagrange multipliers applied to the first order approximation of the constrained equations of motion reduces to a mass weighted version of the pseudoinverse of the constraint Jacobian.

The linearization of the constrained equations of motion is equivalent to solving the constraint equations subject to a least squares minimization of $\Delta \mathbf{X}$ with a distance measure using “metric” M .

$$\min_{\Delta \mathbf{X}} \frac{1}{2} \Delta \mathbf{X}^T \cdot M \Delta \mathbf{X} \quad (5.5)$$

subject to the constraint

$$\mathcal{D}\mathbf{c} \Delta \mathbf{X} = -\mathbf{c}(\mathbf{X}^{t+1}). \quad (5.6)$$

We are finding a solution that satisfies the constraint equations while minimizing the energy term $\frac{1}{2} \Delta \mathbf{X} \cdot M \Delta \mathbf{X}$.

This is an extension of the Moore-Penrose Pseudoinverse method $A^+ = A^T(AA^T)^{-1}$ to a space with an inner product $\langle, \rangle_M$.

Tikhonov regularization can be used together with the pseudoinverse [4] for ill posed problems where $(AA^T)^{-1}$ does not exist. In this case for small ϵ the inverse is corrected by

$$A^+ = A^T(\epsilon \mathbb{1} + AA^T)^{-1} \quad (5.7)$$

and, applying this to the solution of the constraint equations with a metric M we recover our previous result

$$\Delta \mathbf{X} = -M^{-1}(\mathcal{D}\mathbf{c})^T \left(\epsilon \mathbb{1} + \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)^{-1} \mathbf{c}(\mathbf{X}^{t+1}) \quad (5.8)$$

5.2 Sequential Impulses

It is useful here to distinguish the Lagrange multiplier, which has the units of a force, from the impulse accumulated by a single constraint. Define the scalar constraint impulse

$$p_i = \lambda_i \Delta t. \quad (5.9)$$

For constraint row i , the corresponding generalized impulse applied to the bodies is

$$\mathbf{j}_i = (\mathcal{D}\mathbf{c}_i)^T p_i, \quad (5.10)$$

which has dimensions $[MLT^{-1}]$. The change in generalized velocity is therefore

$$\Delta \mathbf{v}_i = M^{-1} \mathbf{j}_i, \quad (5.11)$$

and over the timestep this produces the displacement

$$\Delta \mathbf{X}_i = M^{-1} \mathbf{j}_i \Delta t. \quad (5.12)$$

Define the scalar effective mass

$$\mu_i = \left(\mathcal{D}\mathbf{c}_i M^{-1} (\mathcal{D}\mathbf{c}_i)^T \right)^{-1}. \quad (5.13)$$

The scalar μ_i measures the resistance of the system to motion in the direction that resolves the i -th constraint.

Using $p_i = \lambda_i \Delta t$, the non-linear Gauss–Seidel update (4.62) becomes

$$p_i^{k+1} = p_i^k - \frac{\mu_i}{\Delta t} c_i(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k). \quad (5.14)$$

The incremental generalized impulse is

$$\Delta \mathbf{j}_i = (\mathcal{D}\mathbf{c}_i)^T (p_i^{k+1} - p_i^k), \quad (5.15)$$

so the solver can equivalently update

$$\Delta \mathbf{X}^{k+1} = \Delta \mathbf{X}^k + M^{-1} \Delta \mathbf{j}_i \Delta t. \quad (5.16)$$

This is the usual sequential impulse structure: solve one scalar constraint impulse at a time, immediately apply its change to the bodies, and let the next constraint see the updated state. The difference is in the residual being driven to zero. A conventional velocity-level sequential impulse solve uses a relative velocity (plus any bias) as the residual, whereas here the residual is the current position error divided by Δt . Re-evaluating the non-linear position constraint after every impulse is the position-based version of the same Gauss–Seidel mechanism.

If $p_i = \lambda_i \Delta t$ is the scalar impulse accumulated by constraint i , then $(\mathcal{D}\mathbf{c}_i)^T p_i$ is the generalized impulse applied to the bodies. The PBD Gauss–Seidel update can therefore be interpreted as a sequential impulse method whose target is formed from the current configuration error rather than only from a velocity error [10].

5.3 Position Based Dynamics

The equation (4.62) for the Lagrange multipliers λ_i together with the subsequent $\Delta \mathbf{X}$ update (4.24) can be directly related to the original position based dynamics update [28]. In this paper, they dealt with only point particles, so no angular degrees of freedom, and they defined an iterative update of the positions as

$$\delta \mathbf{p}_i = -s w_i \nabla_{\mathbf{p}_i} C(\mathbf{p}_1, \dots, \mathbf{p}_n), \quad (5.17)$$

with

$$s = \frac{C(\mathbf{p}_1, \dots, \mathbf{p}_n)}{\sum_i w_i |\nabla_{\mathbf{p}_i} C(\mathbf{p}_1, \dots, \mathbf{p}_n)|^2} \quad (5.18)$$

and $w_i = 1/m_i$.

In our notation, this is just

$$s = \left(\mathcal{D} \mathbf{c} M^{-1} (\mathcal{D} \mathbf{c})^T \right)^{-1} \mathbf{c}(\mathbf{x}^{t+1} + \Delta \mathbf{x}^k) \quad (5.19)$$

and

$$\begin{aligned} \lambda_i^{k+1} \Delta t^2 &= \lambda_i^k \Delta t^2 - s, \\ \Delta \mathbf{x}^{k+1} &= \Delta \mathbf{x}^k + M^{-1} (\mathcal{D} \mathbf{c})^T (\lambda_i^{k+1} - \lambda_i^k) \Delta t^2. \end{aligned} \quad (5.20)$$

In the PBD paper, they introduce a stiffness factor by multiplying the corrections at each iteration by a constant k . In our formulation, we can see that this is equivalent to adding a relaxation $k = \xi$ to the update. So, this stiffness factor is just slowing the convergence of the hard constraint, making it effectively a soft constraint, but with the softness dependent on the number of iterations. The authors address this in their paper using a modified stiffness $k' = 1 - (1 - k)^{1/n}$, where n is the number of iterations.

The Gauss–Seidel method applied to solve the linearized constrained equations of motion reproduces the PBD algorithm, except in PBD the constraint derivative $\mathcal{D} \mathbf{c}$ and effective mass are re-evaluated at each iteration.

Re-evaluating $\mathcal{D} \mathbf{c}$ and the effective mass each iteration can be expensive. Note also that evaluating $\mathcal{D} \mathbf{c}$ at the predicted state, whether once or every iteration, conserves linear momentum but not angular momentum exactly, as we will see in §7.2.

PBD for rigid bodies is explored in Deul et al. [15], which develops practical positional contact, stacking and joint constraints. The emphasis here is different though. Rather than taking the positional update as the starting point, we are deriving why the rigid-body update has this form from the constrained equations of motion.

5.4 Discrete Variational Mechanics

The previous section derived PBD by starting from the continuous Euler–Lagrange equations of motion then perturbing the solution of the unconstrained equations to find a first order correction satisfying the constrained equations of motion. This provides the most direct route to the algorithm, but does not show how the resulting discrete correction relates to the underlying variational formulation.

In a discrete variational formulation, rather than first deriving and then discretising the equations of motion, the action itself is discretised. The resulting discrete Euler–Lagrange equations define a discrete dynamical system. This approach has several benefits, including preserving symplecticity, which can lead to better long-term behaviour, e.g. for energy conservation, than discretising the continuous equations of motion directly.

Consider a mechanical system with configuration $\mathbf{x}$ and Lagrangian

$$L(\mathbf{x}, \dot{\mathbf{x}}) = \frac{1}{2} \dot{\mathbf{x}}^T M \dot{\mathbf{x}} - V(\mathbf{x}), \quad (5.21)$$

where M is the mass matrix. The continuous action is

$$S = \int_{t_0}^{t_1} L(\mathbf{x}, \dot{\mathbf{x}}) dt. \quad (5.22)$$

Discrete variational mechanics replaces the continuous path by a sequence of configurations

$$\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_N \quad (5.23)$$

at times $t_k = k\Delta t$, and introduces a *discrete Lagrangian* L_d satisfying

$$L_d(\mathbf{x}_k, \mathbf{x}_{k+1}) \approx \int_{t_k}^{t_{k+1}} L(\mathbf{x}, \dot{\mathbf{x}}) dt. \quad (5.24)$$

The discrete action is then

$$S_d = \sum_{k=0}^{N-1} L_d(\mathbf{x}_k, \mathbf{x}_{k+1}). \quad (5.25)$$

Requiring the discrete action to be stationary with respect to variations of the configuration gives the discrete Euler–Lagrange equations,

$$D_2 L_d(\mathbf{x}_{k-1}, \mathbf{x}_k) + D_1 L_d(\mathbf{x}_k, \mathbf{x}_{k+1}) = 0, \quad (5.26)$$

where D_i stands for differentiation with respect to the i -th argument.

Consider the discrete Lagrangian

$$L_d(\mathbf{x}_k, \mathbf{x}_{k+1}) = \Delta t \left(\frac{1}{2} \left(\frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{\Delta t} \right)^T M \left(\frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{\Delta t} \right) - V(\mathbf{x}_k) \right). \quad (5.27)$$

The derivatives of this discrete Lagrangian are

$$D_1 L_d = -M \frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{\Delta t} - \Delta t \nabla V(\mathbf{x}_k), \quad (5.28)$$

and

$$D_2 L_d = M \frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{\Delta t}. \quad (5.29)$$

Substitution into (5.26) gives

$$M \frac{\mathbf{x}_{k+1} - 2\mathbf{x}_k + \mathbf{x}_{k-1}}{\Delta t^2} = -\nabla V(\mathbf{x}_k). \quad (5.30)$$

So the unconstrained prediction we used in the previous sections can be seen as being the discrete Euler–Lagrange equation of a discrete action.

Constraints can be incorporated into this scheme as before by using Lagrange multipliers.

$$L_d(\mathbf{x}_k, \mathbf{x}_{k+1}, \boldsymbol{\lambda}_k) = L_d(\mathbf{x}_k, \mathbf{x}_{k+1}) + \Delta t \boldsymbol{\lambda}_k \cdot \mathbf{c}(\mathbf{x}_k). \quad (5.31)$$

Variation with respect to $\boldsymbol{\lambda}_k$ gives the constraint equations

$$\mathbf{c}(\mathbf{x}_k) = 0, \quad (5.32)$$

and variation with respect to $\mathbf{x}_k$ gives

$$M \frac{\mathbf{x}_{k+1} - 2\mathbf{x}_k + \mathbf{x}_{k-1}}{\Delta t^2} = -\nabla V(\mathbf{x}_k) + J_k^T \boldsymbol{\lambda}_k, \quad (5.33)$$

where

$$J_k = \left. \frac{\partial \mathbf{c}}{\partial \mathbf{x}} \right|_{\mathbf{x}_k} \quad (5.34)$$

Equation (5.33) is a discrete variational constrained dynamics problem and is the basis of constraint methods such as SHAKE [33]. The closely related RATTLE method also enforces the corresponding velocity constraints [1], making it a useful comparison to the velocity constraints in §4.7 with split velocity and position solve that we explore later in §6.5. PBD can be understood as a first-order approximation to this discrete constrained dynamics problem.

Let $\tilde{\mathbf{x}}_{k+1}$ denote the unconstrained prediction obtained from the discrete Euler–Lagrange equations. We want to find a new configuration

$$\mathbf{x}_{k+1} = \tilde{\mathbf{x}}_{k+1} + \Delta \mathbf{x}. \quad (5.35)$$

That solves the constraint equations

$$\mathbf{c}(\mathbf{x}_{k+1}) = 0. \quad (5.36)$$

Rather than solving the full nonlinear constrained discrete equations, the constraint function is approximated to first-order about the configuration $\mathbf{x}_k$,

$$\mathbf{c}(\mathbf{x}_{k+1}) = \mathbf{c}(\mathbf{x}_k) + J_k(\mathbf{x}_{k+1} - \mathbf{x}_k) + \cdots. \quad (5.37)$$

So

$$0 = \mathbf{c}(\mathbf{x}_k) + J_k(\tilde{\mathbf{x}}_{k+1} - \mathbf{x}_k) + J_k \Delta \mathbf{x} + \cdots. \quad (5.38)$$

From (5.33) we have

$$\Delta \mathbf{x} = \Delta t^2 M^{-1} J_k^T \boldsymbol{\lambda}_k. \quad (5.39)$$

Substituting into the linearized constraint gives

$$J_k M^{-1} J_k^T \boldsymbol{\lambda}_k = -\frac{1}{\Delta t^2} (\mathbf{c}(\mathbf{x}_k) + J_k(\tilde{\mathbf{x}}_{k+1} - \mathbf{x}_k)). \quad (5.40)$$

Hence

$$\Delta \mathbf{x} = -M^{-1} J_k^T (J_k M^{-1} J_k^T)^{-1} [\mathbf{c}(\mathbf{x}_k) + J_k(\tilde{\mathbf{x}}_{k+1} - \mathbf{x}_k)]. \quad (5.41)$$

The discrete variational formulation clarifies a point that we brushed over in the PBD formulation. Since the constrained discrete Lagrangian (5.31) evaluates the constraint at $\mathbf{x}_k$, its variation produces the Jacobian

$$J_k = \left. \frac{\partial \mathbf{c}}{\partial \mathbf{x}} \right|_{\mathbf{x}_k}. \quad (5.42)$$

The first-order approximation uses the constraint gradient at the beginning of the time step. In an iterative solve the constraint value can be re-evaluated at the latest iteration state, as in §4.6, but the Jacobian stays J_k , as in SHAKE.

The same argument applies to the angular degrees of freedom, to leading order in $\|\boldsymbol{\omega} \Delta t\|$. As derived previously, an incremental change in the rotation coordinates is related to the corresponding spatial angular perturbation by

$$\Delta \phi = J_L(\boldsymbol{\theta}_k) \Delta \boldsymbol{\theta}, \quad (5.43)$$

so we need to use

$$J_k \rightarrow \mathcal{D}\mathbf{c}|_{\mathbf{x}_k}, \quad (5.44)$$

and the angular correction follows the same form as the linear equation

$$\Delta \phi = \Delta t^2 I_k^{-1} (\mathcal{D}\mathbf{c})_k^T \boldsymbol{\lambda}_k, \quad (5.45)$$

where the constraint derivative and world-space inertia are evaluated at the beginning of the step. As with the modified inertia (4.17), this drops the gyroscopic coupling of order $\|\boldsymbol{\omega} \Delta t\|$ - an exact variational treatment of the rotation needs a Lie group integrator [27].

Rigid body constraints couple the linear and angular degrees of freedom, so the two are solved together. With M_k the mass matrix at the start of the step, the resulting constraint solve has the same form as the linear result derived above,

$$\Delta \mathbf{X} = -M_k^{-1} (\mathcal{D}\mathbf{c})_k^T \left[\mathcal{D}\mathbf{c}_k M_k^{-1} (\mathcal{D}\mathbf{c})_k^T \right]^{-1} \left[\mathbf{c}(\mathbf{X}_k) + \mathcal{D}\mathbf{c}_k \begin{pmatrix} \tilde{\mathbf{x}}_{k+1} - \mathbf{x}_k \\ \boldsymbol{\omega}_k^{t+1} \Delta t \end{pmatrix} \right], \quad (5.46)$$

with one such pair of entries per body, where $\boldsymbol{\omega}_k^{t+1} \Delta t$ is the predicted rotation over the step, $\tilde{R}_{k+1} = e^{[\boldsymbol{\omega}_k^{t+1} \Delta t] \times} R_k$, and to first order it equals $J_L(\boldsymbol{\theta}_k)$ times the change in axis-angle coordinates, $\tilde{\boldsymbol{\theta}}_{k+1} - \boldsymbol{\theta}_k$.

6 Techniques to Improve Convergence and Stability

6.1 Relaxation

A well known modification to the Gauss–Seidel algorithm is to introduce a relaxation parameter ξ . This can be used to control how much of the error per constraint is fixed up per iteration, and thus can be used to control the convergence of the system.

To implement relaxation we modify the λ update (4.62) with the addition of a relaxation factor ξ

$$\lambda_i^{k+1} \Delta t^2 = \lambda_i^k \Delta t^2 - \xi \left(\mathcal{D} \mathbf{c} M^{-1} (\mathcal{D} \mathbf{c})^T \right)_{ii}^{-1} \mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k)_i, \quad (6.1)$$

The relaxation factor scales the per frame correction to λ .

This relaxation factor can help with the convergence of the system. Under-relaxation with $\xi < 1$ can allow for systems that are otherwise divergent to converge, and over-relaxation $\xi > 1$ can help speed up the convergence of an already convergent system.

Previously, in (4.38) we saw the change in energy per iteration $\Delta E = -\frac{1}{2} A_{ii}^{-1} |e_i|^2$. With the addition of the relaxation parameter, this becomes

$$\Delta E = -\xi \left(1 - \frac{\xi}{2} \right) A_{ii}^{-1} |e_i|^2. \quad (6.2)$$

So, for SPD matrix A , the energy decreases, and the system converges if

$$0 < \xi < 2 \quad (6.3)$$

With the additional relaxation factor, the iteration matrix goes to

$$G_\xi = (D + \xi L)^{-1} ((1 - \xi)D - \xi L^T), \quad (6.4)$$

which reduces to the regular Gauss–Seidel iteration matrix for $\xi = 1$.

In the original PBD formulation, relaxation was also used as a way of making constraints appear soft, by applying only a fraction of the constraint correction at each iteration. This makes the resulting stiffness dependent on the number of solver iterations, an issue addressed by XPBD by introducing compliance directly into the constraint formulation. Adding compliance through relaxation was a useful, cheap numerical tool for controlling, but not the correct use of relaxation, which is a way to control the rate of convergence. PBD for stiff constraints is not inherently more iteration dependent than XPBD, both methods require iterative convergence, but XPBD correctly moves the constraint stiffness formulation from the relaxation factor to the constraint formulation.

Under-Relaxation

If $\xi < 1$ this is called under-relaxation. With relaxation the update of the per iteration error (4.58) goes to

$$|\epsilon^{k+1}| \leq (1 - \xi(1 - r_i)) \|\epsilon^k\|_{\max} \quad (6.5)$$

so converges for

$$|1 - \xi(1 - r_i)| < 1. \quad (6.6)$$

This means that having $\xi < 1$ worsens the convergence rate but at the trade off of greater stability. We are reducing the per constraint per iteration step size, fixing less of the error each frame. This slows convergence but can also reduce potential oscillation between competing constraint targets or errors due to highly non-linear constraints.

Over-Relaxation

Setting $\xi > 1$ is called successive over-relaxation, or SOR. We are effectively extrapolating the error, which can speed up convergence, but may cause divergence depending on how large the spectral radius of the iteration matrix is.

The optimal relaxation factor ξ^* for a given system is related to the spectral radius of the Jacobi iteration matrix $\rho(G_J)$ by [43].

$$\xi^* \sim \frac{2}{1 + \sqrt{1 - \rho(G_J)^2}}. \quad (6.7)$$

where, from (4.33) it can be seen that the Jacobi iteration matrix is

$$G_J = -D^{-1}(L + L^T). \quad (6.8)$$

Under special conditions where the matrix $A = \mathcal{D}\mathbf{c} M^{-1}(\mathcal{D}\mathbf{c})^T$ is consistently ordered, e.g. has a block tri-diagonal structure (as it would have for a simple chain of constrained bodies) the spectral radius of the Jacobi matrix is related to the spectral radius of the Gauss–Seidel matrix by $\rho(G_{GS}) = \rho(G_J)^2$.

If the spectral radius of the iteration matrix is high (which will be true if A is not diagonally dominant) then SOR can be used to speed up convergence, and an optimal value of ξ can be found in the range $1 < \xi < 2$.

Adaptive Relaxation

In practice, for rigid body simulations where constraints are added and removed dynamically and the optimal relaxation factor can't be computed beforehand, a reasonable scheme can be to modify the relaxation per iteration based on the measured convergence of the system. If $|\epsilon^{k+1}| < |\epsilon^k|$ then the system is converging and we can increase ξ . If $|\epsilon^{k+1}| \geq |\epsilon^k|$ the system is diverging and we can try to recover by decreasing ξ .

Note that this adaptive relaxation is a practical heuristic, not a consequence of the convergence analysis above.

6.2 Warm Starting

The iteration loop presented in §4.3 sets the Lagrange multipliers to zero at the start of each update. This is a safe choice, as the Lagrange multipliers correspond to the constraint forces, and the goal is to find a minimum set of constraint forces to simultaneously solve all of the constraints. However, for stable systems where the constraints are not changing from frame to frame, such as solving a chain of bodies connected by joint constraints or a stable stack of rigid bodies, the result from the previous frame is often a good guess for what the result will be for the subsequent frame, so we can speed up convergence by starting the next frame's solve using the resulting Lagrange multipliers from the previous frame. For stable systems we are effectively allowing for convergence of the system over multiple frames, which can mean greater stability and reduce the number of

iterations required per frame. However, for highly dynamic systems where constraints are changing significantly between frames, this can produce instability as we can end up injecting forces into the system that do not solve the new set of constraints well.

Concretely, if the set of Lagrange multipliers from the previous frame was $\boldsymbol{\lambda}^{prev}$, then set the initial Lagrange multipliers for the next frame (the initial guess for the solution to the system of equations) as

$$\boldsymbol{\lambda}^0 = \eta \boldsymbol{\lambda}^{prev}, \quad 0 \leq \eta \leq 1. \quad (6.9)$$

For systems where the set of constraints is not changing much from frame to frame - e.g. a stack of bodies, or a chain of constraints - setting η to be large, say 0.8 – 1.0 is reasonable.

After initializing $\boldsymbol{\lambda}^0$ it is also important to initialize $\Delta \mathbf{X}$ with the initial guess before the iteration loop begins:

$$\Delta \mathbf{X}^0 = M^{-1}(\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}^0 \Delta t^2. \quad (6.10)$$

Warm starting does not improve the rate of convergence or change the convergence analysis of section §4.5 - the Gauss–Seidel update converges for any SPD matrix - what it does is to potentially reduce the initial error. For temporally coherent systems the previous result $\boldsymbol{\lambda}^{prev}$ is often a much better guess to the next frame’s solution than starting from zero. Warm starting effectively allows the system to converge over multiple frames. Over N frames with k iterations the original error decays like $\rho(G)^{Nk}$ so, it is possible to solve at a lower number of iterations and amortize the cost of converging the system over multiple frames.

Warm starting assumes that we have a consistent set of constraints and that the solution is only slowly changing from frame to frame. Contact constraints are generally regenerated each frame, so warm starting requires identifying contacts from one frame to the next. This is typically done via proximity to a previous frame contact, or, more robustly, via some form of feature id matching. See, for example [11] for more information on contact caching.

Warm starting sets

$$\begin{aligned} \boldsymbol{\lambda}^0 &= \eta \boldsymbol{\lambda}^{prev}, \quad 0 \leq \eta \leq 1, \\ \Delta \mathbf{X}^0 &= M^{-1}(\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}^0 \Delta t^2 \end{aligned} \quad (6.11)$$

before starting the iteration loop.

It does not affect the rate of convergence but does affect the initial error, and can amortize the cost of convergence over multiple frames, allowing each frame to run at a lower number of iterations.

It requires identifying constraints between frames and can inject energy when the constraints are changing rapidly.

6.3 Sub-stepping

So far we have considered the solution of a system going from a state at t to a state at $t + \Delta t$ in a single step. A common method of increasing accuracy and stability of the solution, particularly for

large steps is to use sub-stepping, where a step of Δt is divided up into n smaller steps of $\Delta t/n$. Typically, collision detection is performed once per frame and then the system is solved for the same set of constraints over n steps of length $\Delta t/n$:

1. Perform collision detection and generate contact constraints.
2. Set the substep size $\Delta t = \Delta T/n$.
3. For each substep $s = 1, \dots, n$:
 - (a) Integrate the unconstrained equations of motion over Δt to obtain the predicted state $\mathbf{X}^{t+1}, \dot{\mathbf{X}}^{t+1}$.
 - (b) Solve the constraint system at the predicted state and apply the state update.

See, for example, [26]. In this paper, they explore reducing the number of iterations and increasing the number of sub-steps.

The benefits of sub-stepping are:

- **Smaller initial error:** The constraint violation produced by one unconstrained integration scales with Δt . Error $\sim v_{\text{rel}}\Delta t$ for some relative velocity. Halving the time step halves the initial error. Where the relative velocity is zero, e.g. for stacking, then the error scales with the acceleration - Error $\sim g\Delta t^2$, where g is the gravity acceleration. In these cases the error shrinks quadratically, while the number of steps increases linearly.
- **Smaller linearization error:** For the linear version of the solver the rotational error is of order $\mathcal{O}(\omega\Delta t)$. Sub-stepping reduces this error. For the non-linear solver the error is computed exactly, so this benefit does not apply, although sub-stepping does mean that other solver terms, like the constraint derivative $\mathcal{D}c$ are computed each sub-step making them more accurate.
- **Weaker gyroscopic coupling:** The correction $\tilde{I}$ we applied to the inertia in (4.17) differs from I by factors of order $\mathcal{O}(\omega\Delta t)$. The additional terms came from the gyroscopic terms in the Euler equation $\boldsymbol{\omega} \times I\boldsymbol{\omega}$. We dropped the modified inertia due to it making the matrix to solve non-SPD. Sub-stepping makes this approximation more valid.

Per frame there are two kinds of error. The first is solver error, the constraint violation left after k iterations, which decays like $\rho(G)^k\epsilon_0$. The second is discretization error - that of the first-order time step itself, the $\mathcal{O}(\omega\Delta t)$ relative error from dropping the gyroscopic terms and from evaluating $\mathcal{D}c$ once per step, and, for the linear solver, the error from linearizing the constraint c about $\mathbf{X}^{t+1}$. These are at most $\mathcal{O}(\Delta t^2)$ per step, which accumulates to $\mathcal{O}(\Delta t)$ over a fixed simulated time. Iterations only reduce the first kind of error. As k increases the solver error vanishes, but the trajectory is still only accurate to $\mathcal{O}(\Delta t)$, a floor that no iteration count can remove. Sub-stepping lowers it.

For resting or near-equilibrium constraints the initial error is $\epsilon_0 \sim g\Delta t^2$. Consider a fixed budget of n Gauss-Seidel iterations per frame of length ΔT , and suppose each iteration removes a fraction $1 - \rho$ of the accumulated error while the unconstrained integration adds $\sim g\Delta t^2$ per step. The residual then obeys $e_{s+1} = \rho(e_s + g\Delta t^2)$, with steady state $e_\infty = \rho g\Delta t^2/(1 - \rho)$; with k iterations per step the same holds with $\rho \rightarrow \rho^k$. Hence

- n substeps $\times$ 1 iteration: $e_\infty = \frac{\rho}{1-\rho} g \left(\frac{\Delta T}{n} \right)^2$
- 1 step $\times$ n iterations ($\Delta t = \Delta T$): $e_\infty = \frac{\rho^n}{1-\rho^n} g \Delta T^2$

The ratio of the iterated to the sub-stepped residual is

$$\frac{n^2(1-\rho)\rho^{n-1}}{1-\rho^n} \rightarrow n \quad \text{as } \rho \rightarrow 1, \quad (6.12)$$

since $1-\rho^n \approx n(1-\rho)$. Sub-stepping is therefore roughly n times better when the system is poorly conditioned. E.g for $\rho = 0.9, n = 5$ the ratio is ~ 4.0 . For $\rho = 0.5, n = 5$ the ratio is ~ 0.8 . So sub-stepping wins in cases where there is work to do to converge the system. When the system is close to convergence iterating wins, but in that case we are already close to the solution so it doesn't matter. We cover convergence in more detail in §11.

6.4 Baumgarte Stabilization

The position based methods in these notes correct constraint error at the position level: the displacement $\Delta \mathbf{X}$ modifies the configuration directly. Historically, however, most rigid body engines solved constraints purely at the velocity level, i.e. using only the non-holonomic constraints of section §3.3. At the velocity level, position error is invisible: a velocity solve maintains $\dot{\mathbf{c}} = 0$ but does nothing about $\mathbf{c} \neq 0$, so drift accumulated from integration and linearization error grows without bound. Joints slowly pull apart and contacts sink.

The classical fix is Baumgarte stabilization [6], which replaces the velocity level constraint $\dot{\mathbf{c}} = 0$ with

$$\dot{\mathbf{c}}(\mathbf{X}, \dot{\mathbf{X}}) + \frac{\beta}{\Delta t} \mathbf{c}(\mathbf{X}) = 0, \quad 0 < \beta \leq 1, \quad (6.13)$$

so that the constraint velocity is driven towards a value that removes a fraction β of the position error over one timestep. (In the continuous setting this is $\dot{\mathbf{c}} + \gamma \mathbf{c} = 0$, giving exponential decay of the error $\mathbf{c}(t) \sim e^{-\gamma t}$; the discrete form identifies $\gamma = \beta/\Delta t$.)

In the PBD framework presented here this is a non-holonomic constraint with a configuration dependent bias. The velocity derivative is unchanged, so the effective mass is unchanged, and the per iteration update becomes

$$\lambda_i^{k+1} \Delta t = \lambda_i^k \Delta t - \xi \left(\mathcal{D}_{\mathcal{V}c} M^{-1} (\mathcal{D}_{\mathcal{V}c})^T \right)_{ii}^{-1} \left[\dot{\mathbf{c}}(\dot{\mathbf{X}}^{t+1} + \Delta \mathbf{X}^k / \Delta t) + \frac{\beta}{\Delta t} \mathbf{c}(\mathbf{X}^{t+1}) \right]_i. \quad (6.14)$$

Comparing this to the position level update (6.1), the position error term is the same. The crucial difference is where the correction is applied. In the Baumgarte scheme the entire correction, including the part sourced by the position error $\beta \mathbf{c}$, is applied to the velocity, with the position only recovering in subsequent frames as the biased velocity is integrated. The correcting velocity is *real*: it persists after the solve.

This has two well known consequences. The first is energy injection - a body penetrating a surface receives an outward velocity proportional to the penetration depth, and if the penetration was created in a single frame (a fast impact with a large Δt) the body can be launched out of the surface. The second is interference with restitution - the bias velocity adds to the restitution target

of the contact velocity constraint, making the effective coefficient of restitution depth dependent. Tuning β is then a compromise between drift (small β) and popping (large β).

PBD solves the position directly, essentially the $\beta = 1$ limit, and applies the correction to the position directly as well as the velocity. This avoids the one frame delay in correcting the error, but still retains the problem of velocity injection, which we will cover in the next section on split impulse.

Baumgarte stabilization feeds a fraction β of the position error into the velocity solve as a bias

$$\dot{\mathbf{c}} + \frac{\beta}{\Delta t} \mathbf{c} = 0.$$

The correcting velocity is real and persists after the solve, injecting energy and perturbing restitution. Position projection corrects the same error directly at the position level.

6.5 Split Impulse

Even within a position based scheme, the state update (4.31) couples position and velocity: the velocity is incremented by $\Delta \mathbf{X} / \Delta t$, so any displacement used to resolve position error also changes the momentum of the system. For constraint error that reflects a genuine dynamic interaction this is correct, and gives position based methods their correct physical behaviour. However, some position error is an artifact of an imperfect method solving the system: penetration created by collision detection running at discrete times over a finite number of iterations, or joint drift accumulated from linearization error. Converting the correction of this error into velocity injects momentum that was never physically applied, producing the same popping artifacts as Baumgarte stabilization.

The split impulse method, see e.g. [10], decomposes the solve into an “impulse” that corrects both position and velocity, and one that only affects position.

$$\boldsymbol{\lambda} = \boldsymbol{\lambda}_{\text{vel}} + \boldsymbol{\lambda}_{\text{pos}}, \quad (6.15)$$

where the constraint error at the end of the step is split into the part generated during this step and the part that was already present at its start,

$$\mathbf{c}(\mathbf{X}^{t+1}) = [\mathbf{c}(\mathbf{X}^{t+1}) - \mathbf{c}(\mathbf{X}^t)] + \mathbf{c}(\mathbf{X}^t). \quad (6.16)$$

$\boldsymbol{\lambda}_{\text{vel}}$ removes the first bracket, the error created by the unconstrained motion over the step (to first order $\dot{\mathbf{c}} \Delta t$), which is a genuine dynamic interaction; $\boldsymbol{\lambda}_{\text{pos}}$ removes the second, the initial configuration error $\mathbf{c}(\mathbf{X}^t)$. Together they give $\mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) = 0$ at convergence without correcting the same error twice. In a perfect system $\mathbf{c}(\mathbf{X}^t)$ would be zero as a result of the solve from the previous frame, but in reality the solve may not converge fully or the system could be modified between solves to introduce some initial configuration error.

Both are iterated with the same Gauss–Seidel machinery and the same effective masses. The difference is confined to the state update: $\boldsymbol{\lambda}_{\text{vel}}$ updates positions and velocities as in (4.31), while $\boldsymbol{\lambda}_{\text{pos}}$ produces a *pseudo-displacement* that updates positions only,

$$\begin{aligned} \mathbf{x}^{t+1} &\rightarrow \mathbf{x}^{t+1} + \Delta \mathbf{x}_{\text{vel}} + \Delta \mathbf{x}_{\text{pos}}, \\ \dot{\mathbf{x}}^{t+1} &\rightarrow \dot{\mathbf{x}}^{t+1} + \Delta \mathbf{x}_{\text{vel}} / \Delta t, \end{aligned} \quad (6.17)$$

and likewise for the angular terms. In the language of the sequential impulse interpretation, $\mathbf{j}_{\text{pos}} = M\Delta\mathbf{X}_{\text{pos}}/\Delta t$ is a *pseudo-impulse*: it moves the configuration but carries no momentum into the next frame. This is the scheme anticipated at the end of the non-holonomic constraint discussion, where the position solve and velocity solve are separated, made precise: the position solve's $\Delta\mathbf{X}$ is withheld from the velocity update.

Note the contrast with pure position based dynamics, where the velocity is *defined* by the position change, $\dot{\mathbf{x}} = \Delta\mathbf{x}/\Delta t$, so that position correction always changes velocity. Split impulse is the explicit statement that these two roles of $\Delta\mathbf{X}$ can, and for artifact error should, be decoupled. The cost is that the pseudo-displacement does not participate in restitution or friction (usually desirable), and that the momentum conservation arguments of section §7.1 apply only to the λ_{vel} part: the pseudo-impulses are still equal and opposite per constraint, but they deliberately do not represent physical forces.

Interaction With Warm Starting

The split also makes it clear which part of the solution should be warm started, since the two components have different temporal characters. λ_{vel} represents sustained constraint forces – the weight of a stack transmitted through its contacts, the tension in a chain of joints etc. It is this quantity for which λ^{prev} is a good predictor, and it should be cached. λ_{pos} , however, represents the correction of *this frame's* initial error - if the previous frame's solve converged then the previous frame's position error should have already been removed so the position impulse would be zero. In practice however, the solver may not converge completely to solve the error exactly, and new initial error may be introduced (e.g. by artificially reconfiguring the system). Warm starting this initial error would double count the correction and manifest as jitter.

In an engine that does not split the impulse, the cached λ conflates the two components, and warm starting silently re-applies position corrections. This is one of the practical reasons that engines that warm start aggressively also tend to adopt a split impulse or a separate position correction pass: the split is what makes the cached quantity a pure, persistent force.

Split impulse decomposes the solution into a velocity part and a position part

$$\lambda = \lambda_{\text{vel}} + \lambda_{\text{pos}},$$

where the position part is applied as a pseudo-displacement that updates the configuration but not the velocity. This removes the energy injection from correcting artifact position error such as penetration, and identifies the warm-startable component of the solution: cache λ_{vel} , recompute λ_{pos} each frame.

7 Stiff Constraint Examples

7.1 Point to Point Constraint

Define a constraint between two bodies i and j such that the distance between a point $\mathbf{p}_i$ on body i and a point $\mathbf{p}_j$ on body j is zero.

Let $\mathbf{r}_0$ be the offset of the point $\mathbf{p}$ from the centre of mass $\mathbf{x}$ of the body so that

$$\mathbf{p}(\mathbf{X}) = \mathbf{x} + R(\boldsymbol{\theta})\mathbf{r}_0 = \mathbf{x} + \mathbf{r}(\boldsymbol{\theta}), \quad (7.1)$$

The regular holonomic point-to-point constraint is the vector constraint

$$\mathbf{c}(\mathbf{X}) = \mathbf{p}_i - \mathbf{p}_j = \delta\mathbf{p} = 0, \quad (7.2)$$

which supplies three independent scalar constraint equations. For the positional projection below, however, it is useful to solve the radial scalar error

$$c(\mathbf{X}) = \|\mathbf{p}_i - \mathbf{p}_j\| = \|\delta\mathbf{p}\|. \quad (7.3)$$

This scalar form is an algorithmic non-linear projector rather than a regular coordinate description of the continuous ball-joint constraint: its gradient is undefined at $\delta\mathbf{p} = 0$. Its useful property is that every finite correction is along $\delta\mathbf{p}$, which, as we will see in section §7.2, gives exact angular-momentum conservation per iteration. We can find the constraint derivative of the scalar radial error by using $\dot{c} = \mathcal{D}c \mathcal{V}$ with

$$\mathcal{V} = \begin{pmatrix} \dot{\mathbf{x}}_i \\ \boldsymbol{\omega}_i \\ \dot{\mathbf{x}}_j \\ \boldsymbol{\omega}_j \end{pmatrix} \quad (7.4)$$

So computing

$$\begin{aligned} \dot{c}(\mathbf{X}) &= (\dot{\mathbf{p}}_i - \dot{\mathbf{p}}_j) \cdot \hat{\delta\mathbf{p}}, \\ &= (\dot{\mathbf{x}}_i + \boldsymbol{\omega}_i \times \mathbf{r}_i - \dot{\mathbf{x}}_j - \boldsymbol{\omega}_j \times \mathbf{r}_j) \cdot \hat{\delta\mathbf{p}} \end{aligned} \quad (7.5)$$

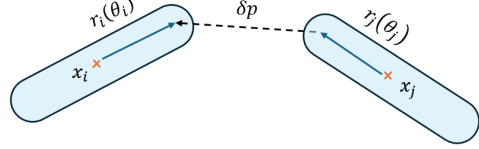
where $\hat{\delta\mathbf{p}}$ is the unit vector in the direction $\delta\mathbf{p}$

$$\hat{\delta\mathbf{p}} = \frac{\delta\mathbf{p}}{\|\delta\mathbf{p}\|} \quad (7.6)$$

gives

$$\mathcal{D}c = \begin{pmatrix} \hat{\delta\mathbf{p}}^T & (\mathbf{r}_i \times \hat{\delta\mathbf{p}})^T & -\hat{\delta\mathbf{p}}^T & -(\mathbf{r}_j \times \hat{\delta\mathbf{p}})^T \end{pmatrix}. \quad (7.7)$$

Note that, when the scalar radial constraint is exactly satisfied, $\|\delta\mathbf{p}\| = 0$ and its unit direction $\hat{\delta\mathbf{p}}$ is undefined. This is the singularity mentioned above, not a singularity of the underlying vector point-to-point constraint. In a solver using the radial projection there is no correction to apply while $\delta\mathbf{p} = 0$; if another constraint subsequently produces a non-zero separation, the direction can be reconstructed from that new separation. A solver that needs a well-defined reaction space at the exactly satisfied configuration should use the vector constraint $\mathbf{c} = \delta\mathbf{p}$ instead.



Define the effective mass μ for this constraint by

$$\begin{aligned}
\mu^{-1} &= \mathcal{D}c M^{-1} (\mathcal{D}c)^T, \\
&= \begin{pmatrix} \hat{\delta \mathbf{p}}^T & (\mathbf{r}_i \times \hat{\delta \mathbf{p}})^T & -\hat{\delta \mathbf{p}}^T & -(\mathbf{r}_j \times \hat{\delta \mathbf{p}})^T \end{pmatrix} \\
&\quad \begin{pmatrix} m_i \mathbb{1} & & & \\ & I_i & & \\ & & m_j \mathbb{1} & \\ & & & I_j \end{pmatrix}^{-1} \begin{pmatrix} \hat{\delta \mathbf{p}} \\ \mathbf{r}_i \times \hat{\delta \mathbf{p}} \\ -\hat{\delta \mathbf{p}} \\ -\mathbf{r}_j \times \hat{\delta \mathbf{p}} \end{pmatrix}, \\
&= \left(\frac{1}{m_i} + \frac{1}{m_j} \right) \mathbb{1} + (\mathbf{r}_i \times \hat{\delta \mathbf{p}}) \cdot I_i^{-1} (\mathbf{r}_i \times \hat{\delta \mathbf{p}}) + (\mathbf{r}_j \times \hat{\delta \mathbf{p}}) \cdot I_j^{-1} (\mathbf{r}_j \times \hat{\delta \mathbf{p}}), \\
&= \hat{\delta \mathbf{p}} \cdot \left(\left(\frac{1}{m_i} + \frac{1}{m_j} \right) \mathbb{1} - [\mathbf{r}_i]_{\times} I_i^{-1} [\mathbf{r}_i]_{\times} - [\mathbf{r}_j]_{\times} I_j^{-1} [\mathbf{r}_j]_{\times} \right) \hat{\delta \mathbf{p}}.
\end{aligned} \tag{7.8}$$

This is the projection of the inverse of the effective mass matrix $\mathcal{M}^{-1}$ for the two body system along the direction $\hat{\delta \mathbf{p}}$.

Define the inverse of the effective mass as $\mathcal{M}^{-1}$ and its projection in the constraint direction as μ^{-1}

$$\mathcal{M}^{-1} = \left(\frac{1}{m_i} + \frac{1}{m_j} \right) \mathbb{1} - [\mathbf{r}_i]_{\times} I_i^{-1} [\mathbf{r}_i]_{\times} - [\mathbf{r}_j]_{\times} I_j^{-1} [\mathbf{r}_j]_{\times}, \tag{7.10}$$

$$\mu^{-1} = \hat{\delta \mathbf{p}} \cdot \mathcal{M}^{-1} \hat{\delta \mathbf{p}}. \tag{7.11}$$

The effective mass measures the resistance to movement of the two body system

Using this, the update is

$$\lambda^{k+1} \Delta t^2 = \lambda^k \Delta t^2 - \xi \mu \|\delta \mathbf{p}(\mathbf{X}^{t+1} + \Delta \mathbf{X})\|, \tag{7.12}$$

with

$$\begin{aligned}
\Delta \mathbf{X}' &= \Delta \mathbf{X} + M^{-1} (\mathcal{D}c)^T (\lambda^{k+1} - \lambda^k) \Delta t^2, \\
&= \Delta \mathbf{X} + \begin{pmatrix} \frac{1}{m_i} \mathbb{1} & & & \\ & I_i^{-1} & & \\ & & \frac{1}{m_j} \mathbb{1} & \\ & & & I_j^{-1} \end{pmatrix} \begin{pmatrix} \hat{\delta \mathbf{p}} \\ \mathbf{r}_i \times \hat{\delta \mathbf{p}} \\ -\hat{\delta \mathbf{p}} \\ -\mathbf{r}_j \times \hat{\delta \mathbf{p}} \end{pmatrix} (\lambda^{k+1} - \lambda^k) \Delta t^2,
\end{aligned} \tag{7.13}$$

Overall:

Constraint update for a point to point constraint:

Lagrange multiplier update:

$$\lambda^{k+1} \Delta t^2 = \lambda^k \Delta t^2 - \xi \mu \|\delta \mathbf{p}(\mathbf{X}^{t+1} + \Delta \mathbf{X})\|$$

Delta:

$$\Delta = -\xi\mu\delta\mathbf{p}(\mathbf{X}^{t+1} + \Delta\mathbf{X})$$

Configuration update:

$$\begin{aligned}\Delta\mathbf{x}'_i &= \Delta\mathbf{x}_i + \frac{1}{m_i}\Delta, \\ \Delta\boldsymbol{\phi}'_i &= \Delta\boldsymbol{\phi}_i + I_i^{-1}\mathbf{r}_i \times \Delta, \\ \Delta\mathbf{x}'_j &= \Delta\mathbf{x}_j - \frac{1}{m_j}\Delta, \\ \Delta\boldsymbol{\phi}'_j &= \Delta\boldsymbol{\phi}_j - I_j^{-1}\mathbf{r}_j \times \Delta.\end{aligned}\tag{7.14}$$

7.1.1 Comparison to an Impulse Based Approach

We can see that the constraint update is essentially an incremental impulse update

$$\begin{aligned}(m_i\Delta\mathbf{x}_i/\Delta t)' &= (m_i\Delta\mathbf{x}_i/\Delta t) + (\Delta/\Delta t), \\ (I_i\Delta\boldsymbol{\phi}_i/\Delta t)' &= (I_i\Delta\boldsymbol{\phi}_i/\Delta t) + \mathbf{r}_i \times (\Delta/\Delta t), \\ (m_j\Delta\mathbf{x}_j/\Delta t)' &= (m_j\Delta\mathbf{x}_j/\Delta t) - (\Delta/\Delta t), \\ (I_j\Delta\boldsymbol{\phi}_j/\Delta t)' &= (I_j\Delta\boldsymbol{\phi}_j/\Delta t) - \mathbf{r}_j \times (\Delta/\Delta t).\end{aligned}\tag{7.15}$$

The only difference being that in an incremental impulse update you would typically be computing the relative velocity between the two points on i and j , $\delta\dot{\mathbf{p}}$, rather than computing the change in position coordinates $\mathbf{x}$ and $\boldsymbol{\theta}$ then using that to find the exact error $\delta\mathbf{p}(\mathbf{X}^{t+1} + \Delta\mathbf{X})$ and dividing by the time step. A velocity/impulse based approach results in a linearization of this error, which can cause problems when the angular velocities are high.

7.2 Symmetries and Conservation of Momentum

In continuous mechanics conservation laws follow from symmetries of the action via Noether's theorem [19]. For example, consider shifting all position coordinates by a small constant vector $\boldsymbol{\epsilon}$: $\mathbf{x}_i \rightarrow \mathbf{x}_i + \boldsymbol{\epsilon}$. Then

$$\begin{aligned}S &\rightarrow \int dt \left[L(\dot{\mathbf{x}}, \mathbf{x}) + \sum_i \frac{\partial L}{\partial \mathbf{x}_i} \cdot \boldsymbol{\epsilon} + \boldsymbol{\lambda} \cdot \left(c(\mathbf{x}) + \sum_i \frac{\partial c}{\partial \mathbf{x}_i} \boldsymbol{\epsilon} \right) \right], \\ &= S + \int dt \sum_i \left[-\frac{\partial V}{\partial \mathbf{x}_i} + \left(\frac{\partial c}{\partial \mathbf{x}_i} \right)^T \boldsymbol{\lambda} \right] \cdot \boldsymbol{\epsilon}, \\ &= S + \int dt \sum_i \left[\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\mathbf{x}}_i} \right) \right] \cdot \boldsymbol{\epsilon}, \\ &= S + \int dt \frac{d}{dt} \left[\sum_i (m_i \dot{\mathbf{x}}_i) \right] \cdot \boldsymbol{\epsilon}.\end{aligned}\tag{7.16}$$

So, if the action is symmetric under $\mathbf{x} \rightarrow \mathbf{x} + \boldsymbol{\epsilon}$, then that implies that $\sum_i (m_i \dot{\mathbf{x}}_i)$ is a constant, i.e. the total linear momentum of the system is conserved. This means we need constraints to be

symmetric under translations in order to preserve linear momentum. E.g. a two body constraint should be a function of $(\mathbf{x}_i - \mathbf{x}_j)$.

The conservation of angular momentum follows in the same way from rotational symmetry of the action. Consider an infinitesimal rotation of the whole system by $\boldsymbol{\epsilon}$, $R(\boldsymbol{\epsilon}) = e^{[\boldsymbol{\epsilon}]_\times} \approx \mathbb{1} + [\boldsymbol{\epsilon}]_\times$. Each body's orientation is left-multiplied, $R_i \rightarrow R(\boldsymbol{\epsilon})R_i$, so $\delta R_i R_i^T = [\boldsymbol{\epsilon}]_\times$ and, by (2.42), the change in the axis-angle vector is $\delta \boldsymbol{\theta}_i = J_L(\boldsymbol{\theta}_i)^{-1} \boldsymbol{\epsilon}$. The centres of mass rotate about the origin, so altogether,

$$\begin{aligned} \delta \mathbf{x}_i &= \boldsymbol{\epsilon} \times \mathbf{x}_i, \\ \delta \boldsymbol{\theta}_i &= J_L(\boldsymbol{\theta}_i)^{-1} \boldsymbol{\epsilon}. \end{aligned} \quad (7.17)$$

To find the quantity conserved by this symmetry, suppose $L + \boldsymbol{\lambda} \cdot \mathbf{c}$ is invariant under it, so that

$$\begin{aligned} 0 &= \delta \left(L(\dot{\mathbf{X}}, \mathbf{X}) + \boldsymbol{\lambda} \cdot \mathbf{c}(\mathbf{X}) \right) = \sum_i \left[\frac{\partial L}{\partial \dot{\mathbf{X}}_i} \cdot \delta \dot{\mathbf{X}}_i + \left(\frac{\partial L}{\partial \mathbf{X}_i} + \left(\frac{\partial \mathbf{c}}{\partial \mathbf{X}_i} \right)^T \boldsymbol{\lambda} \right) \cdot \delta \mathbf{X}_i \right], \\ &= \sum_i \left[\mathbf{P}_i \cdot \delta \dot{\mathbf{X}}_i + \dot{\mathbf{P}}_i \cdot \delta \mathbf{X}_i \right], \\ &= \frac{d}{dt} \sum_i \mathbf{P}_i \cdot \delta \mathbf{X}_i, \end{aligned} \quad (7.18)$$

where $\mathbf{P}_i = \partial L / \partial \dot{\mathbf{X}}_i = M_{L,i} \dot{\mathbf{X}}_i$ is the momentum conjugate to $\mathbf{X}_i$, and in the second line we used the Euler–Lagrange equations, $\dot{\mathbf{P}}_i = \partial L / \partial \mathbf{X}_i + (\partial \mathbf{c} / \partial \mathbf{X}_i)^T \boldsymbol{\lambda}$. Hence

$$Q = \sum_i \mathbf{P}_i \cdot \delta \mathbf{X}_i \quad (7.19)$$

is conserved.

The conjugate momenta are

$$\mathbf{P}_i = \begin{pmatrix} m_i \dot{\mathbf{x}}_i \\ J_L(\boldsymbol{\theta}_i)^T I_i \boldsymbol{\omega}_i \end{pmatrix}, \quad (7.20)$$

so the momentum conjugate to $\boldsymbol{\theta}_i$ is J_L^T times the angular momentum $I_i \boldsymbol{\omega}_i$ — the momentum-side counterpart of $\boldsymbol{\omega} = J_L \dot{\boldsymbol{\theta}}$. For the infinitesimal rotation,

$$\begin{aligned} \mathbf{P}_i \cdot \delta \mathbf{X}_i &= m_i \dot{\mathbf{x}}_i \cdot (\boldsymbol{\epsilon} \times \mathbf{x}_i) + (J_L(\boldsymbol{\theta}_i)^T I_i \boldsymbol{\omega}_i) \cdot (J_L(\boldsymbol{\theta}_i)^{-1} \boldsymbol{\epsilon}), \\ &= \boldsymbol{\epsilon} \cdot (\mathbf{x}_i \times m_i \dot{\mathbf{x}}_i) + (I_i \boldsymbol{\omega}_i) \cdot (J_L(\boldsymbol{\theta}_i) J_L(\boldsymbol{\theta}_i)^{-1} \boldsymbol{\epsilon}), \\ &= [\mathbf{x}_i \times m_i \dot{\mathbf{x}}_i + I_i \boldsymbol{\omega}_i] \cdot \boldsymbol{\epsilon}. \end{aligned} \quad (7.21)$$

The two factors of J_L cancel, as they must (as $\mathbf{P}_i \cdot \delta \mathbf{X}_i$ is a scalar). Therefore

$$\frac{d}{dt} \sum_i [\mathbf{x}_i \times m_i \dot{\mathbf{x}}_i + I_i \boldsymbol{\omega}_i] \cdot \boldsymbol{\epsilon} = 0 \quad (7.22)$$

for every $\boldsymbol{\epsilon}$, i.e. the total angular momentum $\sum_i (\mathbf{x}_i \times m_i \dot{\mathbf{x}}_i + I_i \boldsymbol{\omega}_i)$ is conserved whenever the action is symmetric under an infinitesimal rotation. Since a finite rotation is a product of infinitesimal ones, angular momentum is conserved if the system is invariant under arbitrary rotations.

The question is now, how do these arguments translate to the discrete update. A full treatment of the discrete Noether theorem is given in [27]. We will look at two examples.

Firstly, consider again the uniform translation of the system, $\mathbf{x}_i \rightarrow \mathbf{x}_i + \epsilon \mathbf{a}$ for every body i . If the constraint is invariant under this transformation of the system then

$$c(\mathbf{x}_1 + \epsilon \mathbf{a}, \dots, \mathbf{x}_N + \epsilon \mathbf{a}) = c(\mathbf{x}_1, \dots, \mathbf{x}_N). \quad (7.23)$$

Differentiating with respect to ϵ at $\epsilon = 0$ gives

$$\sum_i \left. \frac{\partial c}{\partial \mathbf{x}_i} \right|_{\mathbf{X}} \mathbf{a} = 0. \quad (7.24)$$

The constraint force on body i is $\mathbf{f}_i = \left(\left. \frac{\partial c}{\partial \mathbf{x}_i} \right|_{\mathbf{X}} \right)^T \lambda$, so using this gives us

$$\mathbf{a} \cdot \sum_i \mathbf{f}_i = 0, \quad (7.25)$$

and since this holds for an arbitrary $\mathbf{a}$, the sum of all the constraint forces is zero, and the linear momentum is conserved.

This holds whichever configuration $\mathbf{X}$ the constraint derivative is evaluated at, because the translation is the same for every configuration. Position based dynamics therefore conserves linear momentum exactly, at every iteration.

Now consider the effect of a small rotation of the whole system about the origin, $\mathbf{x}_i \rightarrow \mathbf{x}_i + \epsilon \mathbf{a} \times \mathbf{x}_i$ and $R_i \rightarrow e^{\epsilon [\mathbf{a}] \times} R_i$. By (2.42) the change in orientation is a world-space angular displacement $\delta \phi_i = \epsilon \mathbf{a}$, or $\delta \theta_i = \epsilon J_L^{-1}(\theta_i) \mathbf{a}$ in axis-angle coordinates. The constraint invariance gives

$$c(\mathbf{x}_i + \epsilon \mathbf{a} \times \mathbf{x}_i, e^{\epsilon [\mathbf{a}] \times} R_i) = c(\mathbf{x}_i, R_i) + \mathcal{O}(\epsilon^2). \quad (7.26)$$

Differentiating with respect to ϵ at $\epsilon = 0$ gives

$$\sum_i (\mathcal{D}c)_i|_{\mathbf{X}} \begin{pmatrix} \mathbf{a} \times \mathbf{x}_i \\ \mathbf{a} \end{pmatrix} = 0, \quad (7.27)$$

where

$$(\mathcal{D}c)_i|_{\mathbf{X}} = \left(\left. \frac{\partial c}{\partial \mathbf{x}_i} \right|_{\mathbf{X}} \quad \left. \frac{\partial c}{\partial \theta_i} J_L^{-1}(\theta_i) \right|_{\mathbf{X}} \right) \quad (7.28)$$

is the block of the left-trivialized derivative for body i . The J_L^{-1} in the angular block is what converts the world-space rotation $\mathbf{a}$ into the change of θ_i .

The corresponding constraint force and torque on body i are

$$\begin{pmatrix} \mathbf{f}_i \\ \boldsymbol{\tau}_i \end{pmatrix} = ((\mathcal{D}c)_i|_{\mathbf{X}})^T \lambda \quad (7.29)$$

and therefore

$$\begin{aligned} \sum_i \begin{pmatrix} \mathbf{a} \times \mathbf{x}_i \\ \mathbf{a} \end{pmatrix}^T ((\mathcal{D}c)_i|_{\mathbf{X}})^T \lambda &= \sum_i [(\mathbf{a} \times \mathbf{x}_i) \cdot \mathbf{f}_i + \mathbf{a} \cdot \boldsymbol{\tau}_i], \\ &= \mathbf{a} \cdot \sum_i (\mathbf{x}_i \times \mathbf{f}_i + \boldsymbol{\tau}_i) = 0. \end{aligned} \quad (7.30)$$

Since $\mathbf{a}$ is arbitrary,

$$\sum_i (\mathbf{x}_i \times \mathbf{f}_i + \boldsymbol{\tau}_i) = 0, \quad (7.31)$$

so the constraint exerts no net torque about the origin. The moment arms, however, are the positions $\mathbf{x}_i$ of the configuration $\mathbf{X}$ at which $\mathcal{D}c$ was evaluated.

This is where the rotation differs from the translation. The generator $\mathbf{a} \times \mathbf{x}_i$ depends on the configuration, so (7.31) only guarantees zero net torque about that one configuration. To conserve angular momentum, it has to be the configuration whose moment arms the discrete angular momentum actually uses.

To find that configuration, use the corrected update for $\mathbf{x}^{t+1} = \mathbf{x}^t + \dot{\mathbf{x}}^{t+1} \Delta t$, $R^{t+1} = e^{[\boldsymbol{\omega}^{t+1} \Delta t] \times} R^t$.

$$\begin{aligned} m_i \Delta \mathbf{x}_i &= \mathbf{f}_i \Delta t^2, & I_i \Delta \phi_i &= \boldsymbol{\tau}_i \Delta t^2, \\ \dot{\mathbf{x}}'_i &= \dot{\mathbf{x}}_i^{t+1} + \Delta \mathbf{x}_i / \Delta t, & \boldsymbol{\omega}'_i &= \boldsymbol{\omega}_i^{t+1} + \Delta \phi_i / \Delta t, \\ \mathbf{x}'_i &= \mathbf{x}_i^{t+1} + \Delta \mathbf{x}_i, & R'_i &= e^{[\Delta \phi_i] \times} R^{t+1} \end{aligned} \quad (7.32)$$

Both the predicted and the corrected states therefore satisfy $\mathbf{x}_i = \mathbf{x}_i^t + \Delta t \dot{\mathbf{x}}_i$, and since $\dot{\mathbf{x}}_i \times \dot{\mathbf{x}}_i = 0$,

$$\mathbf{x}'_i \times m_i \dot{\mathbf{x}}'_i = \mathbf{x}_i^t \times m_i \dot{\mathbf{x}}'_i, \quad \mathbf{x}_i^{t+1} \times m_i \dot{\mathbf{x}}_i^{t+1} = \mathbf{x}_i^t \times m_i \dot{\mathbf{x}}_i^{t+1}. \quad (7.33)$$

The change in angular momentum caused by the constraint is then

$$\delta \mathbf{L} = \sum_i [\mathbf{x}_i^t \times m_i (\dot{\mathbf{x}}'_i - \dot{\mathbf{x}}_i^{t+1}) + I_i (\boldsymbol{\omega}'_i - \boldsymbol{\omega}_i^{t+1})] = \Delta t \sum_i (\mathbf{x}_i^t \times \mathbf{f}_i + \boldsymbol{\tau}_i). \quad (7.34)$$

The moment arms are the start-of-step positions $\mathbf{x}_i^t$. Whether $\delta \mathbf{L}$ vanishes depends on the forces. For rigid bodies the spin term assumes that the inertia does not rotate during the correction, so it is exact for spherical inertia and holds to first order in $\|\boldsymbol{\omega} \Delta t\|$ otherwise, as for the modified inertia (4.17). Comparing (7.34) with (7.31), the angular momentum is conserved exactly when the constraint derivative is evaluated at the start of the step, $\mathbf{X} = \mathbf{X}^t$. This is the variational update of §5.4. If instead $\mathcal{D}c$ is evaluated at the predicted configuration $\mathbf{X}^{t+1}$, as in position based dynamics, then (7.31) holds with arms $\mathbf{x}_i^{t+1}$, and

$$\delta \mathbf{L} = \Delta t \sum_i (\mathbf{x}_i^t - \mathbf{x}_i^{t+1}) \times \mathbf{f}_i = -\Delta t^2 \sum_i \dot{\mathbf{x}}_i^{t+1} \times \mathbf{f}_i. \quad (7.35)$$

This only vanishes when the bodies have no relative velocity across the constraint forces. Otherwise it is non-zero.

For the point constraint of the previous section §7.1

$$c(\mathbf{X}) = \|\delta \mathbf{p}\|, \quad \delta \mathbf{p} = \mathbf{p}_i - \mathbf{p}_j, \quad \mathbf{p} = \mathbf{x} + \mathbf{r}(\boldsymbol{\theta}), \quad (7.36)$$

both linear and angular momentum are conserved, since the constraint is invariant under translations and rotations, providing the Jacobian is evaluated at the start of the step (to first order in $\|\boldsymbol{\omega} \Delta t\|$ for non-spherical bodies, as above). If we had instead chosen a constraint

$$c(\mathbf{X}) = \delta \mathbf{p}, \quad (7.37)$$

i.e. a vector constraint, then that is also fine, but with some caveats. By itself it is not rotationally invariant, but now the Lagrange multiplier is a vector and also transforms under a rotation, and the combination $\boldsymbol{\lambda} \cdot \mathbf{c}$ in the Lagrangian is a scalar and invariant. Before convergence, however, each projection applies a net angular impulse $\mathbf{c} \times \Delta \boldsymbol{\lambda} \Delta t$, since the derivative of $\mathbf{c}$ along the rotation generator is $\mathbf{a} \times \mathbf{c}$ rather than zero, so this only holds once the constraint is satisfied.

If, however, we had chosen a constraint

$$\mathbf{c}(\mathbf{X}) = \delta \mathbf{p} - \mathbf{L}, \quad (7.38)$$

i.e. a vector constraint trying to maintain an offset $\mathbf{L}$, then this would NOT be invariant under rotations, and constraint forces would not conserve angular momentum.

Conserved quantities come from symmetries of the Lagrangian. The symmetries must be respected by the constraint term $\boldsymbol{\lambda} \cdot \mathbf{c}(\mathbf{X})$

- Conservation of linear momentum comes from translational symmetry $\mathbf{x} \rightarrow \mathbf{x} + \mathbf{a}$.
- Conservation of angular momentum comes from rotational symmetry $\mathbf{x}_i \rightarrow e^{[\mathbf{a}] \times} \mathbf{x}_i$, $R_i \rightarrow e^{[\mathbf{a}] \times} R_i$.

In order to conserve angular momentum, the constraint derivative term $\mathcal{D}\mathbf{c}$ must be evaluated in the starting configuration.

The vector constraint $\mathbf{c}(\mathbf{X}) = \delta \mathbf{p}$ only conserves angular momentum at convergence.

7.3 Energy Conservation

In the continuous system the constraint force does no work: its power is $\boldsymbol{\lambda} \cdot (\mathcal{D}\mathbf{c} \mathcal{V}) = \boldsymbol{\lambda} \cdot \dot{\mathbf{c}}$, and $\dot{\mathbf{c}} = 0$ on the constraint manifold. With conservative forces the constrained system therefore conserves energy. The discrete correction does not inherit this, it acts against the drift accumulated over the step, so does work.

Let $\hat{\mathbf{n}} = \hat{\delta \mathbf{p}}$ and write the constraint impulse in signed scalar form as

$$\mathbf{j} = j \hat{\mathbf{n}}, \quad j = -\xi \mu \frac{c(\mathbf{X}^{t+1} + \Delta \mathbf{X})}{\Delta t}. \quad (7.39)$$

The sign is carried by j ; $\|\mathbf{j}\|$ is, of course, non-negative. The impulse $\mathbf{j}$ is applied to body i and $-\mathbf{j}$ to body j . Holding the inertia tensors at their initial values, the kinetic energy of the two bodies changes by

$$\Delta T = \mathbf{j} \cdot \delta \dot{\mathbf{p}} + \frac{1}{2} \mathbf{j} \cdot \mathcal{M}^{-1} \mathbf{j}, \quad (7.40)$$

with $\delta \dot{\mathbf{p}}$ the relative velocity of the attachment points from (7.5) and $\mathcal{M}^{-1}$ the inverse effective mass matrix (7.11). Since $\dot{\mathbf{c}} = \delta \dot{\mathbf{p}} \cdot \hat{\mathbf{n}}$ and $\mu^{-1} = \hat{\mathbf{n}} \cdot \mathcal{M}^{-1} \hat{\mathbf{n}}$,

$$\Delta T = j \dot{\mathbf{c}} + \frac{1}{2} \mu^{-1} j^2. \quad (7.41)$$

Using $j = -\xi\mu c(\mathbf{X}^{t+1})/\Delta t$ for the first correction, we can write the error at the end of the unconstrained step as the error present at its start plus the drift

$$\|\delta\mathbf{p}(\mathbf{X}^{t+1})\| = c_0 + \dot{c}\Delta t + \mathcal{O}(\Delta t^2), \quad (7.42)$$

with $c_0 = c(\mathbf{X}^t)$. If the constraint was satisfied at the start of the step then $c_0 = 0$ and

$$\Delta T = -\xi \left(1 - \frac{\xi}{2}\right) \mu \dot{c}^2, \quad (7.43)$$

which is never positive for $0 < \xi < 2$. For $\xi = 1$ and general c_0 ,

$$\Delta T = -\frac{1}{2} \mu \dot{c}^2 + \frac{1}{2} \mu \left(\frac{c_0}{\Delta t}\right)^2. \quad (7.44)$$

For $c_0 = 0$, the update is equivalent to a perfectly inelastic impact along the joint: the update removes all of the kinetic energy in the relative motion of the attachment points along $\hat{\delta}\mathbf{p}$, measured with the effective mass. Subsequent iterations are not necessarily purely dissipative, however, because they generally operate with non-zero position error and therefore contain the positive c_0^2 term in (7.44).

The second term is position error already present at the start of the step converted into velocity, and it is never negative. It is the energy injection of Baumgarte stabilization in section §6.4, and the two terms of (7.44) are the two brackets of the split impulse decomposition (6.17): λ_{vel} produces the dissipative term and λ_{pos} the injection. Withholding λ_{pos} from the velocity update, as in (6.17), removes the second term and makes the correction purely dissipative. This is the energy argument for the split.

Equation (7.44) is one step. For a joint in motion the attachment points typically separate by $c(\mathbf{X}^{t+1}) \sim a_c \Delta t^2$, where a_c is the relative acceleration along $\hat{\delta}\mathbf{p}$ that the joint force has to supply (e.g. for a link swinging about a pin, the centripetal term $\omega^2 L$ plus the component of gravity along the link), and the joint force that supplies it is $\lambda = \mu a_c$. The update removes this position error by introducing a velocity correction of $c/\Delta t$, so per step

$$\Delta T \sim -\frac{1}{2} \mu \left(\frac{c}{\Delta t}\right)^2 \sim -\frac{\lambda^2}{2\mu} \Delta t^2, \quad \dot{E} \sim -\frac{\lambda^2}{2\mu} \Delta t, \quad (7.45)$$

There are two things to note:

- The loss associated with correcting the velocity drift is first order in Δt , so it is reduced by sub-stepping but not by additional constraint iterations at fixed Δt , another benefit of sub-stepping to add to the list in §6.3,
- The loss scales with the square of the joint force, so, all else being equal, constraints carrying the largest loads damp energy the most.

The reduction of damping when using sub-stepping to take smaller steps was noted in [26].

7.4 Inequality Constraints

A standard extension to Gauss–Seidel is Projected Gauss–Seidel where, as well as solving equality constraints $\mathbf{c}(\mathbf{X}) = 0$, we can extend the system to also solve inequality constraints $\mathbf{c}(\mathbf{X}) \geq 0$. This extension comes from treating the system as a Linear Complementarity Problem (LCP). Starting from the linearized constraint

$$\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}) \approx \mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c}\Delta\mathbf{X}, \quad (7.46)$$

the inequality $\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}) \geq 0$ leads to the convex quadratic program

$$\min_{\Delta\mathbf{X}} \frac{1}{2} \Delta\mathbf{X} \cdot M \Delta\mathbf{X} \quad \text{s.t.} \quad \mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c}\Delta\mathbf{X} \geq 0. \quad (7.47)$$

Eliminating $\Delta\mathbf{X}$ gives the LCP:

$$0 \leq \boldsymbol{\lambda} \perp \mathbf{c}(\mathbf{X}^{t+1}) + \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right) \boldsymbol{\lambda} \geq 0, \quad (7.48)$$

which is solved using a Projected Gauss–Seidel update of

$$\begin{aligned} \lambda_i^* &= \lambda_i^k - \left(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} \mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}^k)_i, \\ \lambda_i^{k+1} &= \begin{cases} \lambda_i^* & \text{if } \lambda_i^* \geq 0 \\ 0 & \text{otherwise} \end{cases} \end{aligned} \quad (7.49)$$

See, for example, [35] where they derive the projected Gauss–Seidel update as a solution to the LCP. Alternatively, [10] gives an impulse centric approach, where it is obvious that clamping the impulse to be one way in the constraint direction gives the correct solution.

7.5 Contact Constraint

A contact constraint is an example of an inequality constraint. The constraint is that for two points $\mathbf{p}_i$ and $\mathbf{p}_j$ on bodies i and j the distance between the points in the direction of a contact normal direction $\hat{\mathbf{n}}$ is positive.

$$c(\mathbf{X}) = (\mathbf{p}(\mathbf{X}_i) - \mathbf{p}(\mathbf{X}_j)) \cdot \hat{\mathbf{n}} \geq 0. \quad (7.50)$$

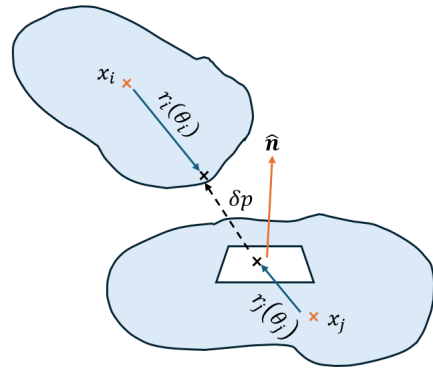
where, as in 7.1

$$\begin{aligned} \mathbf{p}(\mathbf{X}) &= \mathbf{x} + R(\boldsymbol{\theta})\mathbf{r}_0, \\ &= \mathbf{x} + \mathbf{r}(\boldsymbol{\theta}) \end{aligned} \quad (7.51)$$

for an offset $\mathbf{r}$ from a centre of mass $\mathbf{x}$.

The constraint derivative is

$$\mathcal{D}\mathbf{c} = \begin{pmatrix} \hat{\mathbf{n}}^T & (\mathbf{r}_i \times \hat{\mathbf{n}})^T & -\hat{\mathbf{n}}^T & -(\mathbf{r}_j \times \hat{\mathbf{n}})^T \end{pmatrix} \quad (7.52)$$



leading to an inverse effective mass of

$$\begin{aligned}\mu^{-1} &= \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T, \\ &= \hat{\mathbf{n}} \cdot \mathcal{M}^{-1} \hat{\mathbf{n}}\end{aligned}\tag{7.53}$$

where $\mathcal{M}^{-1}$ is the inverse of the effective mass matrix (7.11). I.e., we are projecting the inverse of the effective mass matrix in the direction $\hat{\mathbf{n}}$ to measure the resistance to movement in that direction.

With this, the update becomes

Contact normal constraint update

Lagrange multiplier update with clamping:

$$\begin{aligned}\lambda_i^* &= \lambda_i^k - \xi \mu \delta \mathbf{p}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k) \cdot \hat{\mathbf{n}}, \\ \lambda_i^{k+1} &= \begin{cases} \lambda_i^* & \text{if } \lambda_i^* \geq 0 \\ 0 & \text{otherwise} \end{cases}\end{aligned}\tag{7.54}$$

Delta:

$$\Delta = (\lambda_i^{k+1} - \lambda_i^k) \hat{\mathbf{n}}.\tag{7.55}$$

Configuration update:

$$\begin{aligned}\Delta \mathbf{x}'_i &= \Delta \mathbf{x}_i + \frac{1}{m_i} \Delta, \\ \Delta \phi'_i &= \Delta \phi_i + I_i^{-1} \mathbf{r}_i \times \Delta, \\ \Delta \mathbf{x}'_j &= \Delta \mathbf{x}_j - \frac{1}{m_j} \Delta, \\ \Delta \phi'_j &= \Delta \phi_j - I_j^{-1} \mathbf{r}_j \times \Delta.\end{aligned}\tag{7.56}$$

7.6 Contact Velocity Constraint

For collisions with restitution e the final relative velocity along the contact normal should be equal to $-e$ times the incoming relative velocity.

$$\dot{\delta \mathbf{p}}(\dot{\mathbf{X}}^{t+1}) \cdot \hat{\mathbf{n}} = -e \dot{\delta \mathbf{p}}(\dot{\mathbf{X}}^t) \cdot \hat{\mathbf{n}}\tag{7.57}$$

with

$$\dot{\mathbf{p}} = \dot{\mathbf{x}} + \boldsymbol{\omega} \times \mathbf{r}.\tag{7.58}$$

We can make this into a velocity constraint

$$c(\dot{\mathbf{X}}) = \dot{\delta \mathbf{p}}(\mathbf{X}^{t+1}) \cdot \hat{\mathbf{n}} + e \dot{\delta \mathbf{p}}(\mathbf{X}^t) \cdot \hat{\mathbf{n}}.\tag{7.59}$$

This is a velocity-level impact law rather than an ideal non-holonomic constraint in the geometric-mechanics sense. However, it uses exactly the same velocity-space Jacobian and effective-mass

machinery developed in section §3.3. To see that this constraint is the correct one, consider linear velocities along $\mathbf{n}$. The update would be

$$\begin{aligned}
\dot{\mathbf{x}}_i &\rightarrow \dot{\mathbf{x}}_i - \frac{1}{m_i} \mu (1+e) (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j), \\
\dot{\mathbf{x}}_j &\rightarrow \dot{\mathbf{x}}_j + \frac{1}{m_j} \mu (1+e) (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j), \\
\Rightarrow (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j) &\rightarrow (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j) - \mu^{-1} \mu (1+e) (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j), \\
&= -e (\dot{\mathbf{x}}_i - \dot{\mathbf{x}}_j).
\end{aligned} \tag{7.60}$$

The constraint derivative is

$$\mathcal{D}_{vc} = (\hat{\mathbf{n}}^T \quad (\mathbf{r}_i \times \hat{\mathbf{n}})^T \quad -\hat{\mathbf{n}}^T \quad -(\mathbf{r}_j \times \hat{\mathbf{n}})^T), \tag{7.61}$$

which is identical to $\mathcal{D}\mathbf{c}$ we derived from the contact normal constraint. So, the effective mass is also the same

$$\mu^{-1} = \hat{\mathbf{n}} \cdot \mathcal{M}^{-1} \hat{\mathbf{n}}. \tag{7.62}$$

With this, the update is

Contact normal velocity constraint with restitution
Accumulated normal impulse update with clamping:

$$\begin{aligned}
p_{n,i}^* &= p_{n,i}^k - \xi \mu \left(\delta \dot{\mathbf{p}}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k / \Delta t) \cdot \hat{\mathbf{n}} + e \delta \dot{\mathbf{p}}(\mathbf{X}^t) \cdot \hat{\mathbf{n}} \right), \\
p_{n,i}^{k+1} &= \begin{cases} p_{n,i}^* & \text{if } p_{n,i}^* \geq 0 \\ 0 & \text{otherwise.} \end{cases}
\end{aligned} \tag{7.63}$$

The impulse change produces a displacement over the timestep

$$\Delta = (p_{n,i}^{k+1} - p_{n,i}^k) \Delta t \hat{\mathbf{n}}. \tag{7.64}$$

Configuration update:

$$\begin{aligned}
\Delta \mathbf{x}'_i &= \Delta \mathbf{x}_i + \frac{1}{m_i} \Delta, \\
\Delta \phi'_i &= \Delta \phi_i + I_i^{-1} \mathbf{r}_i \times \Delta, \\
\Delta \mathbf{x}'_j &= \Delta \mathbf{x}_j - \frac{1}{m_j} \Delta, \\
\Delta \phi'_j &= \Delta \phi_j - I_j^{-1} \mathbf{r}_j \times \Delta.
\end{aligned} \tag{7.65}$$

Here p_n is an accumulated impulse, with units of momentum. Multiplying the impulse increment by Δt converts the resulting velocity correction into the displacement variable used by the unified position/velocity solver.

This is almost the same as (7.56). The difference is the displacement term

$$\begin{aligned}
\text{Position :} \quad \Delta &= -\xi \mu \delta \dot{\mathbf{p}}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k) \cdot \hat{\mathbf{n}} \hat{\mathbf{n}}, \\
\text{Velocity :} \quad \Delta &= -\xi \mu (1+e) \delta \dot{\mathbf{p}}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k / \Delta t) \cdot \hat{\mathbf{n}} \hat{\mathbf{n}} \Delta t
\end{aligned} \tag{7.66}$$

and

$$\delta \mathbf{p}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k) = \delta \mathbf{p}(\mathbf{X}^t) + \delta \dot{\mathbf{p}}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k / \Delta t) \Delta t. \quad (7.67)$$

So, to first order, the velocity constraint can be formulated as a position constraint with just a different target relative position.

7.7 Contact Friction Constraint

Friction acts to oppose relative tangential velocity at the contact point. The dry friction model divides the force into two modes: static friction for when two surfaces are not moving relative to each other, and dynamic friction when they are in relative motion in the contact plane. The model is that if the magnitude of the force is less than the static friction limit $\nu_s \|\mathbf{N}\|$ for some contact normal force $\mathbf{N}$ and static friction coefficient ν_s then the force prevents the points moving relative to each other in the contact plane. If, however, the magnitude of the force required to keep the points from moving relative to each other in the contact plane exceeds that limit, then the bodies will start moving, and a dynamic friction force of $-\nu_d \|\mathbf{N}\| \hat{\mathbf{s}}$ will be applied, where $\hat{\mathbf{s}}$ is the direction of the relative surface velocity and ν_d is a dynamic friction coefficient. The model is valid for $\nu_d \leq \nu_s$.

In [37] this scheme is derived as resulting from a principle of maximal dissipation.

Let the relative velocity at the contact points be

$$\dot{\delta \mathbf{p}} = \dot{\mathbf{p}}_i - \dot{\mathbf{p}}_j = (\dot{\mathbf{x}}_i + \boldsymbol{\omega}_i \times \mathbf{r}_i) - (\dot{\mathbf{x}}_j + \boldsymbol{\omega}_j \times \mathbf{r}_j). \quad (7.68)$$

We decompose this into normal and tangential components:

$$\dot{\delta \mathbf{p}}_{\perp} = \dot{\delta \mathbf{p}} - (\dot{\delta \mathbf{p}} \cdot \hat{\mathbf{n}}) \hat{\mathbf{n}}. \quad (7.69)$$

Where

$$\hat{\mathbf{t}} = \frac{\dot{\delta \mathbf{p}}_{\perp}}{\|\dot{\delta \mathbf{p}}_{\perp}\|}. \quad (7.70)$$

We impose the constraint

$$c(\dot{\mathbf{X}}) = \dot{\delta \mathbf{p}} \cdot \hat{\mathbf{t}}. \quad (7.71)$$

The derivative with respect to velocities is

$$\mathcal{D}_{\mathcal{V}} c = (\hat{\mathbf{t}}^T \quad (\mathbf{r}_i \times \hat{\mathbf{t}})^T \quad -\hat{\mathbf{t}}^T \quad -(\mathbf{r}_j \times \hat{\mathbf{t}})^T). \quad (7.72)$$

Giving an effective mass of

$$\mu_t = (\hat{\mathbf{t}} \cdot \mathcal{M}^{-1} \hat{\mathbf{t}})^{-1}, \quad (7.73)$$

where $\mathcal{M}^{-1}$ is the inverse effective mass matrix used in the normal contact constraint.

The constraint update is

Contact friction constraint update

Let $\mathbf{p}_t$ be the accumulated tangential impulse and p_n the accumulated normal impulse at the contact. Then

$$\begin{aligned} \mathbf{p}_t^* &= \mathbf{p}_t^k - \xi \mu_t \delta \dot{\mathbf{p}}(X^{t+1} + \Delta X^k / \Delta t) \cdot \hat{\mathbf{t}} \hat{\mathbf{t}}, \\ \mathbf{p}_t^{k+1} &= \begin{cases} \mathbf{p}_t^* & \text{if } \|\mathbf{p}_t^*\| \leq \nu_s p_n \\ \nu_d p_n \hat{\mathbf{p}}_t^* & \text{otherwise.} \end{cases} \end{aligned} \quad (7.74)$$

The corresponding position-level increment used by the unified solver is

$$\Delta_{\text{fric}} = (\mathbf{p}_t^{k+1} - \mathbf{p}_t^k) \Delta t. \quad (7.75)$$

The incremental position and orientation updates are the same as in the contact normal constraint (7.56), with Δ_{fric} in place of Δ . If the normal reaction is stored as a force $\mathbf{N}$ rather than an impulse, then $p_n = \|\mathbf{N}\| \Delta t$.

Note, this is a slight simplification. If there is relative tangential velocity it is good to construct the friction axis in that direction, but there can be other displacements from other constraints that would move the contact points in a direction off of this axis, but also perpendicular to the contact normal. There are a couple of different approaches to take:

1. **Apply the friction constraint along two orthogonal friction axes.**

So construct the orthonormal frame $\{\mathbf{n}, \hat{\mathbf{t}}_1, \hat{\mathbf{t}}_2 = \hat{\mathbf{t}}_1 \times \mathbf{n}\}$. Taking this approach requires a slight modification to the algorithm. We compute $\mathbf{p}_{t,1}^*$ and $\mathbf{p}_{t,2}^*$ for each of the friction axes, then compare the magnitude of the sum to the static friction limit $\|\mathbf{p}_{t,1}^* + \mathbf{p}_{t,2}^*\| \leq \nu_s p_n$. The reason for doing this is so that the magnitude of the friction is clamped to a circular limit in the friction plane, rather than a box if we were to clamp the individual directions.

2. **Recompute the friction axis during the solve**

This means storing the inverse effective mass matrix $\mathcal{M}^{-1}$ and re-projecting it onto a new $\hat{\mathbf{t}}$ each iteration.

Both of these get around the thing we brushed over that if the relative tangential velocity is zero when formulating the constraint we cannot construct the friction direction $\hat{\mathbf{t}}$ or compute the effective mass. In the case of option 1, if there is no initial tangential velocity we just use two arbitrary axes perpendicular to the normal direction.

One reason to choose option 1 would be if you were warm starting the friction solve. In this case, it is good to have consistent axes from frame to frame. Or you could just re-project the previous result onto the new axes.

Note that this friction model can also be included in a primal formulation as a Rayleigh dissipation function of the tangential relative velocity $\delta \dot{\mathbf{p}}_{\perp}$,

$$\mathcal{F} = \begin{cases} \frac{1}{2} k_t |\delta \dot{\mathbf{p}}_{\perp}|^2, & k_t |\delta \dot{\mathbf{p}}_{\perp}| < \nu \|\mathbf{N}\|, \\ \nu \|\mathbf{N}\| |\delta \dot{\mathbf{p}}_{\perp}| - \gamma, & \text{otherwise,} \end{cases} \quad \gamma = \frac{\nu^2 \|\mathbf{N}\|^2}{2k_t}, \quad (7.76)$$

where k_t is the stiffness of the sticky-regime and γ makes $\mathcal{F}_f$ continuous in the stick/slip transition. The normal force $\mathbf{N}$ is held fixed during the friction update. The friction force is

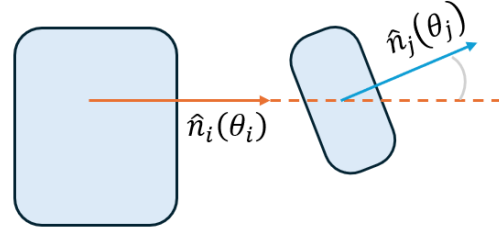
$$\mathbf{f} = -\frac{\partial \mathcal{F}}{\partial \delta \dot{\mathbf{p}}_{\perp}} = -\min\left(k_t, \frac{\nu \|\mathbf{N}\|}{|\delta \dot{\mathbf{p}}_{\perp}|}\right) \delta \dot{\mathbf{p}}_{\perp}, \quad (7.77)$$

which is the equivalent of (7.74) in primal form. See [24].

Primal / force-based approaches are developed in §8.1.

7.8 Angular Axis Constraint

For this constraint example we will look at a purely angular constraint. The goal is to limit the rotation of one body to be around an axis defined in the space of another body. Imagine a wheel constrained to rotate around an axis in the space of a car body, for instance.



There are a few different ways to define this constraint, but we choose

$$c(\boldsymbol{\theta}) = \sin^{-1} \|\hat{\mathbf{n}}_i \times \hat{\mathbf{n}}_j\|, \quad (7.78)$$

where $\hat{\mathbf{n}}$ are the axes on the respective bodies that we want to keep aligned.

The reason for choosing the sine of the angle being zero as the constraint, rather than looking at the cosine with $c(\boldsymbol{\theta}) = \hat{\mathbf{n}}_i \cdot \hat{\mathbf{n}}_j - 1$ is that around the point where the angle between the two is small sine is changing rapidly, whereas cosine is fairly flat, so the derivative around this point is more reliable with the sine formulation.

Note that the constraint is zero both when the axes are parallel as well as when they are anti-parallel. For small enough steps the error stays below 90° , so this is rarely a problem in practice, but a large violation, from an impact or the initial setup, can reach it. Using the full angle,

$$c(\boldsymbol{\theta}) = \text{atan2}(\|\hat{\mathbf{n}}_i \times \hat{\mathbf{n}}_j\|, \hat{\mathbf{n}}_i \cdot \hat{\mathbf{n}}_j), \quad (7.79)$$

avoids this. It agrees with the arcsine for small angles and has the same derivative $\mathcal{D}c$. At exactly $\theta = \pi$ the direction $\hat{\mathbf{w}}$ is undefined, and any axis perpendicular to $\hat{\mathbf{n}}_i$ can be used.

Taking the derivative with respect to $\boldsymbol{\theta}_i$ and $\boldsymbol{\theta}_j$ gives

$$\mathcal{D}c = (-\hat{\mathbf{w}}^T \quad \hat{\mathbf{w}}^T) \quad (7.80)$$

where

$$\hat{\mathbf{w}} = \frac{\hat{\mathbf{n}}_i \times \hat{\mathbf{n}}_j}{\|\hat{\mathbf{n}}_i \times \hat{\mathbf{n}}_j\|} \quad (7.81)$$

is the unit rotation axis required to align the axes.

As with previous constraints, if the initial error is zero then it is not possible to form $\hat{\mathbf{w}}$, so the projected error can be used instead. If both are zero, then an arbitrary axis can be used. In this case, $\hat{\mathbf{w}}$ must be computed per iteration using the projected error.

Using this, the effective inertia is

$$\mu = \left[\hat{\mathbf{w}} \cdot (I_i^{-1} + I_j^{-1}) \hat{\mathbf{w}} \right]^{-1}, \quad (7.82)$$

i.e. the projection of the effective inertia in the direction $\hat{\mathbf{w}}$.

With this, the update is

Axis alignment constraint

Lagrange multiplier update:

$$\lambda^{k+1} = \lambda_i^k - \mu \left(\sin^{-1} \|\hat{\mathbf{n}}_i(\boldsymbol{\theta}_i^{t+1} + \Delta \boldsymbol{\theta}_i) \times \hat{\mathbf{n}}_j(\boldsymbol{\theta}_j^{t+1} + \Delta \boldsymbol{\theta}_j)\| \right) \quad (7.83)$$

Delta:

$$\Delta = -(\lambda^{k+1} - \lambda^k) \hat{\mathbf{w}}. \quad (7.84)$$

Configuration update:

$$\begin{aligned} \Delta \phi'_i &= \Delta \phi_i + I_i^{-1} \Delta, \\ \Delta \phi'_j &= \Delta \phi_j - I_j^{-1} \Delta. \end{aligned} \quad (7.85)$$

8 Compliant Constraints

8.1 Generalized Implicit Non-Linear Forces

So far, we have assumed that we can, by integration scheme, find the solutions to the equations of motion when there are no constraints. But, if we have non-linear forces and torques and we want to solve for them implicitly (i.e. taking the force term at $F(\mathbf{X}^{t+1})$ when solving for $\mathbf{X}^{t+1}$) it is not necessarily easy. But in general updating with implicit forces and torques is a good thing for stability, so we want to find a solution. We proceed using a similar first order expansion that we made when solving the constrained equation, except that this time we assume that we have a solution $\mathbf{X}^{t+1}, \dot{\mathbf{X}}^{t+1}$ that satisfies the equations of motion in the absence of forces and torques and then add a small displacement $\Delta\mathbf{X}$ to the solution and solve the full equations of motion up to first order in $\Delta\mathbf{X}$.

The equations of motion are

$$\frac{d}{dt} (M(\mathbf{X})\mathcal{V}) = \mathbf{F}(\mathbf{X}), \quad (8.1)$$

Taking $\mathbf{X}^{t+1} \rightarrow \mathbf{X}^{t+1} + \Delta\mathbf{X}$ we get

$$M(\mathbf{X}^{t+1} + \Delta\mathbf{X})(\dot{\mathbf{X}}^{t+1}\Delta t + \Delta\mathbf{X}) - M(\mathbf{X}^{t+1})\dot{\mathbf{X}}^{t+1}\Delta t = \mathbf{F}(\mathbf{X}^{t+1})\Delta t^2 + (\mathcal{D}\mathbf{F})^T \Delta\mathbf{X}\Delta t^2, \quad (8.2)$$

which gives a linear equation for $\Delta\mathbf{X}$

Linearized system for an increment $\Delta\mathbf{X}$ required to add to the solution $\mathbf{X}^{t+1}$ of the free equations of motion that solves to first order the equations of motion with generalized forces:

$$(M + K)\Delta\mathbf{X} = \mathbf{F}(\mathbf{X}^{t+1})\Delta t^2 \quad (8.3)$$

with

$$K = -(\mathcal{D}\mathbf{F})^T \Delta t^2 \quad (8.4)$$

Since K is not necessarily block diagonal, we need to use an iterative approach to solve this system for $\Delta\mathbf{X}$. We again use the Gauss-Seidel algorithm. Following a similar method to section §4.3 results in an iterative update

Gauss-Seidel iteration for solving the force equation (8.3)

$$\Delta\mathbf{X}_i^{k+1} = \Delta\mathbf{X}_i^k + \xi [(M + K)_{ii}]^{-1} \left[\mathbf{F}_i(\mathbf{X}^{t+1} + \Delta\mathbf{X}^k)\Delta t^2 - M_i\Delta\mathbf{X}_i^k \right] \quad (8.5)$$

This is of the same form as the iterative update for the constraint forces, and so can be added as an additional term in the constraint solve update in the same iteration loop. The result is that, as we update the displacement $\Delta\mathbf{X}$ each iteration, we correct $\mathbf{X}$ with a change that corresponds to the force to be applied.

8.2 Connection to Vertex Block Descent

Notice in this approach that, instead of solving iteratively per constraint, we are solving per body. $\mathbf{F}_i$ is the sum over all of the forces on body i . This is the same methodology as applied in Vertex Block Descent (VBD) methods [13].

In the vertex block descent paper the starting point is the variational implicit-Euler objective. For a single mass, writing $\Delta\mathbf{x} = \mathbf{x} - \mathbf{y}$ relative to the inertial prediction $\mathbf{y}$, this is

$$G(\mathbf{X}) = \frac{m}{2\Delta t^2} \|\Delta\mathbf{x}\|^2 + E(\mathbf{X}). \quad (8.6)$$

Multiplying the whole objective by Δt^2 , which does not change its minimizer, gives the form that matches our equations directly,

$$\tilde{G}(\mathbf{X}) = \frac{m}{2} \|\Delta\mathbf{x}\|^2 + \Delta t^2 E(\mathbf{X}). \quad (8.7)$$

A local Newton step therefore solves

$$H\Delta\mathbf{x} = \mathbf{f}, \quad (8.8)$$

with

$$\begin{aligned} \mathbf{f} &= -m\Delta\mathbf{x} - \Delta t^2 \frac{\partial E}{\partial \mathbf{x}}, \\ H &= m\mathbf{1} + \Delta t^2 \frac{\partial^2 E}{\partial \mathbf{x}^2}. \end{aligned} \quad (8.9)$$

For conservative forces $\mathbf{F} = -\partial E/\partial \mathbf{X}$, this is the same tangent system as (8.3). The important algorithmic point is that, instead of solving one constraint multiplier at a time, VBD performs block coordinate descent on the global variational objective, taking a local Newton step for one vertex/body block while the remaining blocks are fixed. Our body-block force solve has this same primal Gauss–Seidel/Newton structure; when the forces come from the implicit-Euler variational objective it is the rigid-body analogue of VBD.

The generalized implicit-force update has the same primal body-block structure as Vertex Block Descent. For conservative forces derived from the implicit-Euler variational objective, it is the corresponding rigid-body VBD update.

8.3 Connection to the Lagrange Multiplier Method and PBD

The body-block update (8.5) solves for the displacements directly, which Macklin et al. [24] call the primal form. PBD instead solves for one multiplier per constraint, the dual form. When the forces act through constraint functions, the two are rearrangements of the same linear system.

Take the constraint potential used by XPBD, with damping added in §8.4,

$$V(\mathbf{X}) = \frac{1}{2} \mathbf{c}(\mathbf{X}) \cdot A \mathbf{c}(\mathbf{X}), \quad \mathbf{F}(\mathbf{X}) = -(\mathcal{D}\mathbf{c})^T A \mathbf{c}(\mathbf{X}), \quad (8.10)$$

where A is a diagonal matrix of stiffnesses. Linearizing about $\mathbf{X}^{t+1}$ and dropping the constraint Hessian (the Gauss–Newton approximation), the force equation (8.3) becomes the primal system

$$(M + (\mathcal{D}\mathbf{c})^T A \mathcal{D}\mathbf{c} \Delta t^2) \Delta\mathbf{X} = -(\mathcal{D}\mathbf{c})^T A \mathbf{c} \Delta t^2. \quad (8.11)$$

Now treat the linearized constraint force $\boldsymbol{\lambda} = -A(\mathbf{c} + \mathcal{D}\mathbf{c} \Delta\mathbf{X})$ as a separate unknown, and write $\tilde{\alpha} = (A \Delta t^2)^{-1}$ for the compliance. The primal system is then equivalent to the pair

$$\begin{aligned} M \Delta\mathbf{X} &= (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2, \\ \mathbf{c} + \mathcal{D}\mathbf{c} \Delta\mathbf{X} + \tilde{\alpha} \boldsymbol{\lambda} \Delta t^2 &= 0. \end{aligned} \quad (8.12)$$

The first equation is the same as in the hard-constraint solve of §4.3, and the second is the linearized constraint, softened by the compliance. Eliminating $\boldsymbol{\lambda}$ gives back the primal system. Eliminating $\Delta\mathbf{X}$ instead gives the dual system

$$(\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T + \tilde{\alpha}) \boldsymbol{\lambda} \Delta t^2 = -\mathbf{c}. \quad (8.13)$$

Both systems have the same solution. Gauss–Seidel over bodies on the primal system is the update (8.5), the rigid-body analogue of VBD. Gauss–Seidel over constraints on the dual system, with the constraint re-evaluated at the latest displacement, is the XPBD update derived with damping in §8.4. In the hard-constraint limit $\tilde{\alpha} \rightarrow 0$ it reduces to the PBD update (6.1).

The two iterations do behave differently. The primal matrix loses diagonal dominance as the stiffness $A \Delta t^2$ grows relative to the masses, and has no hard-constraint limit. The dual system stays well defined as $\tilde{\alpha} \rightarrow 0$, with $\tilde{\alpha}$ acting as the regularization of §4.8, but is instead sensitive to large mass ratios, as analysed for a chain in §11. The dual form also keeps each constraint force as an explicit variable, which makes clamping and warm starting straightforward, while the primal form handles any force for which a local Hessian can be formed.

For forces derived from a constraint potential $V = \frac{1}{2} \mathbf{c} \cdot A \mathbf{c}$, the linearized update can be solved in two equivalent forms, with $\tilde{\alpha} = (A \Delta t^2)^{-1}$:

$$\begin{aligned} \text{primal (per body, VBD):} \quad & (M + (\mathcal{D}\mathbf{c})^T A \mathcal{D}\mathbf{c} \Delta t^2) \Delta\mathbf{X} = -(\mathcal{D}\mathbf{c})^T A \mathbf{c} \Delta t^2, \\ \text{dual (per constraint, XPBD):} \quad & (\mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T + \tilde{\alpha}) \boldsymbol{\lambda} \Delta t^2 = -\mathbf{c}, \quad \Delta\mathbf{X} = M^{-1} (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2. \end{aligned}$$

The dual form with $\tilde{\alpha} = 0$ is PBD. Primal iteration degrades with stiffness, dual iteration with mass ratios.

8.4 Compliant Constraint Forces and XPBD

The method of Lagrange multipliers is good for modelling stiff constraints where we want the constraint equation $\mathbf{c}(\mathbf{X}) = 0$ to be satisfied exactly. Often though we want to model systems with some compliance in the constraints, as this can help with stability and also be used to model things with spring-like behaviour. For these types of systems, we replace the constraint with a potential term, the constraint potential (8.10) already used in §8.3,

$$V = \frac{1}{2} \mathbf{c}(\mathbf{X}) \cdot A \mathbf{c}(\mathbf{X}), \quad (8.14)$$

where $\mathbf{c}(\mathbf{X})$ are the constraint functions and A is a diagonal matrix of stiffness parameters. In addition we want to model damping. This is not naturally handled by the standard Lagrangian method, as that assumes conservative forces. The usual modification is to add a Rayleigh dissipation function [19]

$$\mathcal{F} = \frac{1}{2} \dot{\mathbf{X}} \cdot D \dot{\mathbf{X}} \quad (8.15)$$

and then the equations of motion are modified to

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\mathbf{X}}} \right) - \frac{\partial L}{\partial \mathbf{X}} + \frac{\partial \mathcal{F}}{\partial \dot{\mathbf{X}}} = 0. \quad (8.16)$$

In our case we want to apply damping to the constraint function, so we want a dissipation term

$$\frac{1}{2} \dot{\mathbf{c}}(\mathbf{X}) \cdot B \dot{\mathbf{c}}(\mathbf{X}) \quad (8.17)$$

for some diagonal matrix of damping constants B . Using $\dot{\mathbf{c}}(\mathbf{X}) = \mathcal{D}\mathbf{c}\dot{\mathbf{X}}$ we find

$$D = (\mathcal{D}\mathbf{c})^T B \mathcal{D}\mathbf{c}. \quad (8.18)$$

With these additional terms the equations of motion become

$$\frac{d}{dt} (M(\mathbf{X})\mathcal{V}) = -(\mathcal{D}\mathbf{c})^T A\mathbf{c}(\mathbf{X}) - (\mathcal{D}\mathbf{c})^T B \mathcal{D}\mathbf{c}\dot{\mathbf{X}}. \quad (8.19)$$

so the force is

$$\mathbf{F} = -(\mathcal{D}\mathbf{c})^T \left(A\mathbf{c}(\mathbf{X}) + B \mathcal{D}\mathbf{c}\dot{\mathbf{X}} \right). \quad (8.20)$$

To treat this force implicitly we linearize it about the predicted state, as in §8.3. Writing $\dot{\mathbf{X}}$ for the predicted velocity $\dot{\mathbf{X}}^{t+1}$,

$$\begin{aligned} \mathbf{F}(\mathbf{X}^{t+1} + \Delta\mathbf{X}) &= -(\mathcal{D}\mathbf{c})^T \Big|_{\mathbf{X}^{t+1} + \Delta\mathbf{X}} \left(A\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}) + B \mathcal{D}\mathbf{c}|_{\mathbf{X}^{t+1} + \Delta\mathbf{X}} (\dot{\mathbf{X}} + \Delta\mathbf{X}/\Delta t) \right), \\ &= \mathbf{F}(\mathbf{X}^{t+1}) - (\mathcal{D}\mathbf{c})^T (A + B/\Delta t) \mathcal{D}\mathbf{c} \Delta\mathbf{X} \\ &\quad - (\boldsymbol{\varphi} \cdot \mathcal{D}^2\mathbf{c}) \Delta\mathbf{X} - (\mathcal{D}\mathbf{c})^T B \left(\dot{\mathbf{X}} \cdot \mathcal{D}^2\mathbf{c} \right) \Delta\mathbf{X} + \mathcal{O}(\|\Delta\mathbf{X}\|^2), \end{aligned} \quad (8.21)$$

where $\mathcal{D}^2\mathbf{c}$ is the constraint Hessian using the left-trivialized derivative, derived in section §9.1. Here $\boldsymbol{\varphi} = A\mathbf{c}(\mathbf{X}^{t+1}) + B \mathcal{D}\mathbf{c}\dot{\mathbf{X}}$ is the spring-damper force per constraint at the predicted state, $\boldsymbol{\varphi} \cdot \mathcal{D}^2\mathbf{c} = \sum_j \varphi_j \mathcal{D}^2 c_j$, and $\dot{\mathbf{X}} \cdot \mathcal{D}^2\mathbf{c}$ denotes the matrix whose j -th row is $\dot{\mathbf{X}}^T \mathcal{D}^2 c_j$. Therefore, comparing to (8.3),

$$K = (\mathcal{D}\mathbf{c})^T (A\Delta t^2 + B\Delta t) \mathcal{D}\mathbf{c} + \Delta t^2 (\boldsymbol{\varphi} \cdot \mathcal{D}^2\mathbf{c}) + \Delta t^2 (\mathcal{D}\mathbf{c})^T B \left(\dot{\mathbf{X}} \cdot \mathcal{D}^2\mathbf{c} \right). \quad (8.22)$$

The first term is the material stiffness of the spring and damper. The other two involve the constraint Hessian. The second is the geometric stiffness generated by the current spring-damper force, and the third comes from the change in the damping direction with configuration and is not symmetric in general. Their origin and geometric interpretation are discussed in section §9. XPBD neglects both, which is equivalent to a Gauss–Newton approximation, so we will also ignore them for the purposes of comparison to XPBD. This leaves

$$K = (\mathcal{D}\mathbf{c})^T (A\Delta t^2 + B\Delta t) \mathcal{D}\mathbf{c}. \quad (8.23)$$

Primal form: Using this K , the per-body system is

$$\left(M + (\mathcal{D}\mathbf{c})^T (A\Delta t^2 + B\Delta t) \mathcal{D}\mathbf{c} \right) \Delta\mathbf{X} = -(\mathcal{D}\mathbf{c})^T \left(A\mathbf{c}(\mathbf{X}) + B \mathcal{D}\mathbf{c}\dot{\mathbf{X}} \right) \Delta t^2, \quad (8.24)$$

which can be solved with the body-block update (8.5).

Dual form: For the per-constraint solve we proceed as in §8.3. Define the linearized constraint force at the new configuration, now including the damping,

$$\boldsymbol{\lambda} = -A(\mathbf{c} + \mathcal{D}\mathbf{c}\Delta\mathbf{X}) - B\mathcal{D}\mathbf{c}\left(\dot{\mathbf{X}} + \Delta\mathbf{X}/\Delta t\right), \quad (8.25)$$

so that the primal system (8.24) is again $M\Delta\mathbf{X} = (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}\Delta t^2$, with $\boldsymbol{\lambda}$ having the units of a force as in section §4.3. Substituting $\Delta\mathbf{X} = M^{-1}(\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}\Delta t^2$ into the definition of $\boldsymbol{\lambda}$ and multiplying through by Δt^2 gives the dual system

$$\left(1 + (A\Delta t^2 + B\Delta t)\mathcal{D}\mathbf{c}M^{-1}(\mathcal{D}\mathbf{c})^T\right) \boldsymbol{\lambda}\Delta t^2 = -A\Delta t^2 \mathbf{c}(\mathbf{X}) - B\Delta t \mathcal{D}\mathbf{c}(\dot{\mathbf{X}}\Delta t). \quad (8.26)$$

Deriving it this way, rather than by substituting $\Delta\mathbf{X}$ into the primal system and cancelling a factor of $(\mathcal{D}\mathbf{c})^T$, means it holds even when the constraints are redundant.

The Gauss–Seidel iterative update for $\boldsymbol{\lambda}$ is then

$$\begin{aligned} \boldsymbol{\lambda}_i^{k+1}\Delta t^2 = \boldsymbol{\lambda}_i^k\Delta t^2 - \xi \left(1 + (A\Delta t^2 + B\Delta t)\mathcal{D}\mathbf{c}M^{-1}(\mathcal{D}\mathbf{c})^T\right)_{ii}^{-1} \left(A\Delta t^2 \mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}^k) \right. \\ \left. + B\Delta t \mathcal{D}\mathbf{c}(\dot{\mathbf{X}}^{t+1}\Delta t + \Delta\mathbf{X}^k) + \boldsymbol{\lambda}_i^k\Delta t^2\right)_i. \end{aligned} \quad (8.27)$$

To make the connection with XPBD we take

$$A_{ii}\Delta t^2 = \frac{1}{\tilde{\alpha}_i}, \quad (8.28)$$

$$B_{ii}\Delta t^2 = \tilde{\beta}_i, \quad (8.29)$$

$$\gamma_i = \frac{\tilde{\alpha}_i\tilde{\beta}_i}{\Delta t}. \quad (8.30)$$

Using these, the update becomes

$$\begin{aligned} \boldsymbol{\lambda}_i^{k+1}\Delta t^2 = \boldsymbol{\lambda}_i^k\Delta t^2 - \xi \left(\tilde{\alpha}_i + (1 + \gamma_i)\mathcal{D}\mathbf{c}M^{-1}(\mathcal{D}\mathbf{c})^T\right)_{ii}^{-1} \left(\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}^k) \right. \\ \left. + \gamma_i\mathcal{D}\mathbf{c}(\dot{\mathbf{X}}^{t+1}\Delta t + \Delta\mathbf{X}^k) + \tilde{\alpha}_i\boldsymbol{\lambda}_i^k\Delta t^2\right)_i \end{aligned} \quad (8.31)$$

which is exactly the update presented in the XPBD paper [25], once we identify the XPBD multiplier with $\boldsymbol{\lambda}\Delta t^2$ (for a distance constraint XPBD’s λ has the units of mass×length, whereas our $\boldsymbol{\lambda}$ is a force). With $B = 0$ this is Gauss–Seidel on the dual system (8.13) of §8.3.

Summary:

In XPBD, constraints are made compliant by replacing the hard constraint with a constraint potential and a Rayleigh dissipation function,

$$V = \frac{1}{2}\mathbf{c}(\mathbf{X}) \cdot A\mathbf{c}(\mathbf{X}), \quad \mathcal{F} = \frac{1}{2}\dot{\mathbf{c}}(\mathbf{X}) \cdot B\dot{\mathbf{c}}(\mathbf{X}), \quad (8.32)$$

which add a spring and damper to each constraint function, allowing for soft constraint resolution. Previously, in PBD, softness was added to a Lagrange multiplier system using solver

relaxation, which is iteration dependent. This was the big benefit of XPBD, and allowed for more consistent and stable systems.

The resulting forces can be solved per body, using the primal system (8.24) and the update (8.5), or per constraint, using the dual system (8.26). XPBD works with the dual form of §8.3: one multiplier per constraint, solved by Gauss–Seidel over constraints, which is what connects it directly to PBD.

8.4.1 XPBD as a Regularization on PBD

Note that the presence of the additional $\tilde{\alpha}_i$ term in the denominator acts similarly to the regularization parameter we explored in section §4.8. The same is visible directly in the dual matrix (8.13), where $\tilde{\alpha}$ is added to the diagonal of $\mathcal{D}\mathbf{c}M^{-1}(\mathcal{D}\mathbf{c})^T$. For large stiffness values $\tilde{\alpha} \rightarrow 0$ and we get back to something like the Lagrange multiplier result. For simplicity, let the damping be zero and just focus on the spring like term. Let $A_{ii}\Delta t^2 \rightarrow \infty$ so that $\tilde{\alpha}_i \rightarrow \epsilon$ for $\epsilon \rightarrow 0$. Then the XPBD update becomes

$$\boldsymbol{\lambda}_i^{k+1}\Delta t^2 = \boldsymbol{\lambda}_i^k\Delta t^2 - \xi \left(\epsilon + \mathcal{D}\mathbf{c}M^{-1}(\mathcal{D}\mathbf{c})^T \right)_{ii}^{-1} \left(\mathbf{c}(\mathbf{X}^{t+1} + \Delta\mathbf{X}^k) + \epsilon\boldsymbol{\lambda}_i^k\Delta t^2 \right)_i. \quad (8.33)$$

So for infinitely stiff springs, $A_{ii} \rightarrow \infty$, the update is exactly the same as the PBD update, but for non-infinite springs we get a relaxation term in the denominator, like we did when explicitly adding regularization in section §4.8, which helps with convergence of ill-posed problems (e.g. over-constrained systems).

For large but not infinite stiffness, compliance in XPBD regularizes the system in the same sense as the Tikhonov regularization. Physically, however, this term represents a finite constraint stiffness rather than a purely numerical regularization, but the equivalence shows why XPBD is stable.

This is explored more in [24].

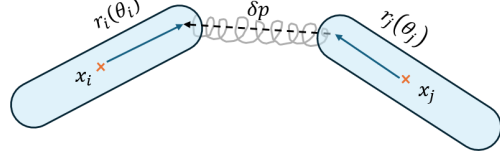
8.5 Compliant Constraint Examples

8.5.1 Spring Damper

Consider a system of two rigid bodies connected by a spring at an offset $\mathbf{r}_i$ and $\mathbf{r}_j$ from the respective centres of mass of the two bodies. The linear force is given by

$$\mathbf{f} = -k\delta\mathbf{p}(\mathbf{X}) - \nu\delta\dot{\mathbf{p}}(\mathbf{X}), \quad (8.34)$$

where k is the spring coefficient, ν is the damping coefficient, and $\delta\mathbf{p}(\mathbf{X})$ is the vector from the connection point on body i to body j .



$$\mathbf{p} = \mathbf{x} + \mathbf{r}(\boldsymbol{\theta}). \quad (8.35)$$

The torque is $\boldsymbol{\tau}_w = \mathbf{r} \times \mathbf{f}$, so the overall generalized force is

$$\mathbf{F}(\mathbf{X}) = \begin{pmatrix} \mathbf{f} \\ \mathbf{r}_i \times \mathbf{f} \\ -\mathbf{f} \\ -\mathbf{r}_j \times \mathbf{f} \end{pmatrix} = \begin{pmatrix} \mathbb{1} \\ [\mathbf{r}_i]_{\times} \\ -\mathbb{1} \\ -[\mathbf{r}_j]_{\times} \end{pmatrix} \mathbf{f}. \quad (8.36)$$

This force can be derived from a constraint

$$\mathbf{c}(\mathbf{X}) = \delta\mathbf{p}(\mathbf{X}) \quad (8.37)$$

and using the XPBD method of adding constraints as a spring-like potential and dissipation function

$$V = \frac{1}{2}k\|\delta\mathbf{p}\|^2, \quad \mathcal{F} = \frac{1}{2}\nu\|\delta\dot{\mathbf{p}}\|^2 \quad (8.38)$$

with

$$\mathcal{D}\mathbf{c} = \begin{pmatrix} \mathbb{1} & -[\mathbf{r}_i]_{\times} & -\mathbb{1} & [\mathbf{r}_j]_{\times} \end{pmatrix} \quad (8.39)$$

and

$$\begin{aligned} \mathbf{F}(\mathbf{X}) &= -(\mathcal{D}\mathbf{c})^T (k\mathbf{c}(\mathbf{X}) + \nu\dot{\mathbf{c}}(\mathbf{X})), \\ &= \begin{pmatrix} \mathbb{1} \\ [\mathbf{r}_i]_{\times} \\ -\mathbb{1} \\ -[\mathbf{r}_j]_{\times} \end{pmatrix} (-k\delta\mathbf{p} - \nu\delta\dot{\mathbf{p}}). \end{aligned} \quad (8.40)$$

We use the constraint derivative to compute the inverse effective mass

$$\begin{aligned} \mathcal{M}^{-1} &= \begin{pmatrix} \mathbb{1} & -[\mathbf{r}_i]_{\times} & -\mathbb{1} & [\mathbf{r}_j]_{\times} \end{pmatrix} \begin{pmatrix} \frac{1}{m_i}\mathbb{1} & & & \\ & I_i & & \\ & & \frac{1}{m_j}\mathbb{1} & \\ & & & I_j \end{pmatrix}^{-1} \begin{pmatrix} \mathbb{1} \\ [\mathbf{r}_i]_{\times} \\ -\mathbb{1} \\ -[\mathbf{r}_j]_{\times} \end{pmatrix}, \\ &= \left(\frac{1}{m_i} + \frac{1}{m_j} \right) \mathbb{1} - [\mathbf{r}_i]_{\times} I_i^{-1} [\mathbf{r}_i]_{\times} - [\mathbf{r}_j]_{\times} I_j^{-1} [\mathbf{r}_j]_{\times}. \end{aligned} \quad (8.41)$$

Comparing to (8.27) we find the per iteration update:

Spring damper constraint update.

Lagrange multiplier update:

$$\begin{aligned} \boldsymbol{\lambda}^{k+1} \Delta t^2 &= \boldsymbol{\lambda}^k \Delta t^2 - \xi (\mathbb{1} + \alpha \mathcal{M}^{-1})^{-1} \\ &\quad \left(k \Delta t^2 \delta \mathbf{p}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) + \nu \Delta t \delta \dot{\mathbf{p}}(\mathbf{X}^{t+1} + \Delta \mathbf{X}) \Delta t + \boldsymbol{\lambda}^k \Delta t^2 \right), \end{aligned} \quad (8.42)$$

where $\alpha = k \Delta t^2 + \nu \Delta t$

Delta:

$$\Delta = (\boldsymbol{\lambda}^{k+1} - \boldsymbol{\lambda}^k) \Delta t^2 \quad (8.43)$$

Configuration update:

$$\begin{aligned} \Delta \mathbf{x}'_i &= \Delta \mathbf{x}_i + \frac{1}{m_i} \Delta, \\ \Delta \boldsymbol{\phi}'_i &= \Delta \boldsymbol{\phi}_i + I_i^{-1} \mathbf{r}_i \times \Delta, \\ \Delta \mathbf{x}'_j &= \Delta \mathbf{x}_j - \frac{1}{m_j} \Delta, \\ \Delta \boldsymbol{\phi}'_j &= \Delta \boldsymbol{\phi}_j - I_j^{-1} \mathbf{r}_j \times \Delta, \end{aligned} \quad (8.44)$$

8.5.2 Volumetric Constraint

Up until this point we have been developing constraints between two rigid bodies, but it's worth showing how the method can be used for constraints between multiple bodies. As an example, consider a constraint between four particles making up the nodes of a tetrahedron. We introduce a constraint on the volume V of the tetrahedron

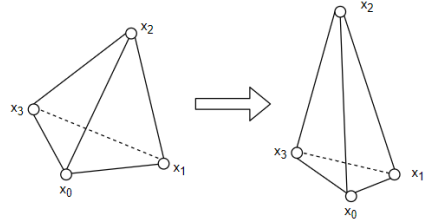
$$c(\mathbf{X}) = 6(V(\mathbf{X}) - V_0). \quad (8.45)$$

If $\mathbf{x}_a$, $a = 0, \dots, 3$, are the positions of the particles then the volume of the tetrahedron is

$$V(\mathbf{X}) = \frac{1}{6} \mathbf{d}_1 \cdot (\mathbf{d}_2 \times \mathbf{d}_3), \quad \mathbf{d}_a = \mathbf{x}_a - \mathbf{x}_0. \quad (8.46)$$

As we are dealing with point particles, there are no angular degrees of freedom and the constraint derivative is

$$\begin{aligned} \mathcal{D}c &= \left(\frac{\partial c}{\partial \mathbf{x}_0} \quad \frac{\partial c}{\partial \mathbf{x}_1} \quad \frac{\partial c}{\partial \mathbf{x}_2} \quad \frac{\partial c}{\partial \mathbf{x}_3} \right), \\ &= -[(\mathbf{d}_2 \times \mathbf{d}_3) + (\mathbf{d}_3 \times \mathbf{d}_1) + (\mathbf{d}_1 \times \mathbf{d}_2)]^T \quad (\mathbf{d}_2 \times \mathbf{d}_3)^T \quad (\mathbf{d}_3 \times \mathbf{d}_1)^T \quad (\mathbf{d}_1 \times \mathbf{d}_2)^T, \\ &= 2 \begin{pmatrix} \mathbf{A}_0^T & \mathbf{A}_1^T & \mathbf{A}_2^T & \mathbf{A}_3^T \end{pmatrix}, \end{aligned} \quad (8.47)$$



where $\mathbf{A}_a$ is the directed area (area times normal) of the triangle opposite the point a , e.g. $\mathbf{A}_1 = \frac{1}{2}\mathbf{d}_2 \times \mathbf{d}_3$, and $\mathbf{A}_0 = -(\mathbf{A}_1 + \mathbf{A}_2 + \mathbf{A}_3)$.
Using this, the effective mass is

$$\mu = \frac{1}{4} \left(\sum_{a=0}^3 \frac{1}{m_a} \|\mathbf{A}_a\|^2 \right)^{-1}. \quad (8.48)$$

Using this in an XPBD update we get

$$\lambda^{k+1} \Delta t^2 = \lambda^k \Delta t^2 - \xi \left(\tilde{\alpha} + (1 + \gamma) \mu^{-1} \right)^{-1} \left(6(V(\mathbf{x}^{t+1} + \Delta \mathbf{x}^k) - V_0) + 2\gamma \sum_{a=0}^3 \mathbf{A}_a \cdot (\dot{\mathbf{x}}_a^{t+1} \Delta t + \Delta \mathbf{x}_a^k) + \tilde{\alpha} \lambda^k \Delta t^2 \right) \quad (8.49)$$

with

$$\Delta \mathbf{x}'_a = \Delta \mathbf{x}_a + \frac{1}{m_a} 2\mathbf{A}_a (\lambda^{k+1} - \lambda^k) \Delta t^2. \quad (8.50)$$

So, each body is pushed in the direction of the normal of the triangle opposite to it.

9 Geometric Stiffness

The position-based methods developed so far use only the first derivative of the constraint functions. At each iteration the constraint is linearised about the current configuration and the mass matrix defines the metric in which the correction is made. This is the usual first-order PBD projection.

We can go one step further and retain the variation of the constraint gradient itself. This introduces the second derivative of the constraint, or *constraint Hessian*. As we will see, when the constrained equations are linearised this second derivative appears as a constraint-dependent contribution to the mass matrix, usually referred to as *geometric stiffness*.

9.1 The Second Left-Trivialized Derivative

For the rotational degrees of freedom we previously defined the left-trivialized derivative by

$$\mathcal{D}c = \frac{\partial c}{\partial \boldsymbol{\theta}} J_L^{-1}, \quad (9.1)$$

so that it differentiates with respect to a spatial angular displacement rather than directly with respect to the exponential coordinates $\boldsymbol{\theta}$.

Applying the operator a second time gives

$$\mathcal{D}(\mathcal{D}c) = \frac{\partial}{\partial \boldsymbol{\theta}} \left(\frac{\partial c}{\partial \boldsymbol{\theta}} J_L^{-1} \right) J_L^{-1}. \quad (9.2)$$

More explicitly, along angular directions $\mathbf{u}$ and $\mathbf{v}$,

$$\mathcal{D}(\mathcal{D}c)(\mathbf{u}, \mathbf{v}) = \mathbf{u}^T J_L^{-T} \frac{\partial^2 c}{\partial \boldsymbol{\theta}^2} J_L^{-1} \mathbf{v} + \frac{\partial c}{\partial \boldsymbol{\theta}} \left(\sum_k [J_L^{-1} \mathbf{v}]_k \frac{\partial J_L^{-1}}{\partial \theta_k} \right) \mathbf{u}. \quad (9.3)$$

The first term is the ordinary coordinate Hessian transformed into the angular tangent space. The second appears because J_L^{-1} itself changes with orientation. This second term is not, in general, symmetric in $\mathbf{u}$ and $\mathbf{v}$ due to the non-commutativity of rotations.

For the second-order model of a scalar constraint, it is only the symmetric part that is relevant. Writing $\mathbf{X} \oplus \Delta \mathbf{X}$ for the rigid-body configuration obtained by translating by $\Delta \mathbf{x}$ and applying the spatial rotational increment $e^{[\Delta \phi] \times}$, consider the expansion

$$c(\mathbf{X} \oplus \Delta \mathbf{X}) = c(\mathbf{X}) + (\mathcal{D}c)\Delta \mathbf{X} + \frac{1}{2} \Delta \mathbf{X} \cdot (\mathcal{D}^2 c) \Delta \mathbf{X} + \mathcal{O}(\|\Delta \mathbf{X}\|^3). \quad (9.4)$$

The quantity we want is the symmetric part of the second derivative, which we define as $\mathcal{D}^2 c$, a symmetric bilinear form. The anti-symmetric part does not contribute because $\Delta \mathbf{X}^T A \Delta \mathbf{X} = 0$ for any anti-symmetric matrix A .

Fortunately, constraint functions do not normally need to be differentiated through J_L^{-1} explicitly. The constraints considered previously can be written in terms of centres of mass and vectors fixed to the bodies and rotated into world space,

$$\mathbf{r}(\boldsymbol{\theta}) = R(\boldsymbol{\theta}) \mathbf{r}_0, \quad (9.5)$$

so that, for a two-body constraint with one such vector on each body,

$$c(\mathbf{X}) = f(\mathbf{x}_1, \mathbf{r}_1(\boldsymbol{\theta}_1), \mathbf{x}_2, \mathbf{r}_2(\boldsymbol{\theta}_2)). \quad (9.6)$$

Here $i, j \in \{1, 2\}$ below, with $\mathbf{r}_i$ denoting the world-space vector associated with body i . For an angular update $\Delta\boldsymbol{\phi}$,

$$\mathbf{r} \rightarrow e^{[\Delta\boldsymbol{\phi}]_\times} \mathbf{r} = \mathbf{r} + \Delta\boldsymbol{\phi} \times \mathbf{r} + \frac{1}{2} \Delta\boldsymbol{\phi} \times (\Delta\boldsymbol{\phi} \times \mathbf{r}) + \mathcal{O}(\Delta\boldsymbol{\phi}^3). \quad (9.7)$$

The first-order term gives the rotational block of the constraint Jacobian,

$$\mathcal{D}^{\phi_i} c = -\frac{\partial f}{\partial \mathbf{r}_i} [\mathbf{r}_i]_\times. \quad (9.8)$$

The second-order term contributes to the constraint Hessian.

We can collect the second-order terms as a 12×12 symmetric matrix for the two-body case,

$$\mathcal{D}^2 c = \begin{pmatrix} (\mathcal{D}^2 c)_{11} & (\mathcal{D}^2 c)_{12} \\ (\mathcal{D}^2 c)_{21} & (\mathcal{D}^2 c)_{22} \end{pmatrix}, \quad (9.9)$$

with one 6×6 block for each pair of bodies,

$$(\mathcal{D}^2 c)_{ij} = \begin{pmatrix} \mathcal{D}^{x_i x_j} c & \mathcal{D}^{x_i \phi_j} c \\ \mathcal{D}^{\phi_i x_j} c & \mathcal{D}^{\phi_i \phi_j} c \end{pmatrix}. \quad (9.10)$$

Finally, the individual 3×3 blocks are

$$\begin{aligned} \mathcal{D}^{x_i x_j} c &= \frac{\partial^2 f}{\partial \mathbf{x}_i \partial \mathbf{x}_j}, \\ \mathcal{D}^{x_i \phi_j} c &= -\frac{\partial^2 f}{\partial \mathbf{x}_i \partial \mathbf{r}_j} [\mathbf{r}_j]_\times, \\ \mathcal{D}^{\phi_i x_j} c &= [\mathbf{r}_i]_\times \frac{\partial^2 f}{\partial \mathbf{r}_i \partial \mathbf{x}_j}, \\ \mathcal{D}^{\phi_i \phi_j} c &= [\mathbf{r}_i]_\times^T \frac{\partial^2 f}{\partial \mathbf{r}_i \partial \mathbf{r}_j} [\mathbf{r}_j]_\times + \delta_{ij} \left[\frac{1}{2} \left(\mathbf{r}_i \frac{\partial f}{\partial \mathbf{r}_i} + \left(\frac{\partial f}{\partial \mathbf{r}_i} \right)^T \mathbf{r}_i^T \right) - \left(\frac{\partial f}{\partial \mathbf{r}_i} \mathbf{r}_i \right) \mathbb{1} \right]. \end{aligned} \quad (9.11)$$

There are two distinct contributions here. The terms containing the ordinary Hessian of f are the curvature of the constraint function itself, pulled back through the first-order rigid body kinematics. The final term in (9.11) is present even when f is linear in the rotated vectors. It arises from the second-order terms in (9.7). This last curvature term only appears in the diagonal angular blocks $\mathcal{D}^{\phi_i \phi_i} c$.

9.2 Geometric Stiffness

We can now use this to see the effect of retaining the second-order variation of the constraint gradient. For a single scalar constraint we have shown that the constraint force equation is

$$M \Delta \mathbf{X} = \mathbf{F} \Delta t^2 = (\mathcal{D} c)^T \lambda \Delta t^2. \quad (9.12)$$

At the displaced configuration, the constraint gradient is

$$(\mathcal{D}c)^T \rightarrow (\mathcal{D}c)^T + (\mathcal{D}^2c)\Delta\mathbf{X}, \quad (9.13)$$

so retaining this first variation of the Jacobian gives

$$M\Delta\mathbf{X} = [(\mathcal{D}c)^T + (\mathcal{D}^2c)\Delta\mathbf{X}] \lambda \Delta t^2. \quad (9.14)$$

This can be written as

$$(M - \lambda(\mathcal{D}^2c)\Delta t^2) \Delta\mathbf{X} = (\mathcal{D}c)^T \lambda \Delta t^2, \quad (9.15)$$

so the effect of adding this second-order term is a modification of the mass matrix

$$M \rightarrow M' = M - \lambda(\mathcal{D}^2c)\Delta t^2. \quad (9.16)$$

For several scalar constraints c_α , the corresponding result is

$$M' = M - \Delta t^2 \sum_{\alpha} \lambda_{\alpha} \mathcal{D}^2 c_{\alpha}. \quad (9.17)$$

This additional term is called the *geometric stiffness*. It is the geometric-stiffness term described by Tournier et al. [41]; see also [2].

Writing the constraint Jacobian as $J = \mathcal{D}c$, Tournier et al. decompose the configuration derivative of a constraint force into a term due to the change in force magnitude and a term due to the change in the constraint direction,

$$J^T \frac{\partial \lambda}{\partial c} J + \lambda \mathcal{D}^2 c. \quad (9.18)$$

The first is commonly called the *material stiffness*, while the second is the geometric stiffness. For the hard constraints considered here, λ is an independent Lagrange multiplier rather than a prescribed function of c , so $\partial \lambda / \partial c$ does not enter the configuration derivative. Its variation is solved separately through the Lagrange-multiplier equation. For a penalty law such as $\lambda = -\rho c$, the force derivative contains $-\rho(\mathcal{D}c)^T(\mathcal{D}c)$; with the sign convention used in the effective matrix this produces the familiar positive contribution $\rho(\mathcal{D}c)^T(\mathcal{D}c)\Delta t^2$. This is the same material-stiffness structure encountered for compliant constraints in section §8.4.

For rigid-body constraints the ordered left-trivialized second derivative need not itself be symmetric. Tournier et al. discuss using the symmetric part of the rigid-body geometric stiffness, which we define as $\mathcal{D}^2c$ in (9.4).

Notice that the Lagrange multiplier appears in the geometric stiffness. This is expected. The Lagrange multiplier gives a measure of the constraint force, and geometric stiffness is produced by an existing force acting through a geometry that is changing. If the constraint force is zero, changing the direction in which that force acts has no effect on the equations of motion.

The dependence of M' on λ makes the system nonlinear. The multiplier appearing in the stiffness matrix is the value about which the equations are currently being linearized. There are several possible choices for how often M' is updated. In [41] they fix the geometric stiffness term once at the start of each simulation frame. This is possible if the system is using warm starting §6.2. In a direct Newton interpretation it is rebuilt each time the system is relinearized, so a convenient implementation is to recompute M' once per iteration. Alternatively, M' can be updated after every

individual constraint projection using the latest available Gauss–Seidel values of the multipliers. This more closely follows a nonlinear Gauss–Seidel method §4.6, but is more expensive.

One important difference from the standard PBD solve is that the mass matrix M is block diagonal, whereas M' need not be, and applying $(M')^{-1}$ exactly may couple several bodies. A practical approximation is to retain only local or block-diagonal contributions. We will see a related approximation in the AVBD discussion in section §10.1.

9.3 Point to Point Constraint

We can see the structure explicitly for the point to point constraint. For the first-order PBD projection we used the scalar radial constraint $c = \|\delta\mathbf{p}\|$, since this directly gives a correction along the point separation. For the second-order treatment, however, it is more natural to return to the vector constraint

$$\mathbf{c}(\mathbf{X}) = \mathbf{p}_i - \mathbf{p}_j = \delta\mathbf{p}, \quad \mathbf{p} = \mathbf{x} + \mathbf{r}(\boldsymbol{\theta}), \quad (9.19)$$

as, unlike with the scalar constraint, the derivatives of this constraint remain well defined as $\delta\mathbf{p} \rightarrow 0$.

The first derivative is

$$\mathcal{D}\mathbf{c} = \begin{pmatrix} \mathbb{1} & -[\mathbf{r}_i]_{\times} & -\mathbb{1} & [\mathbf{r}_j]_{\times} \end{pmatrix}. \quad (9.20)$$

For a vector constraint the Lagrange multiplier is itself a vector, $\boldsymbol{\lambda} \in \mathbb{R}^3$, and the second-order contribution is obtained by contracting the constraint Hessian with the multiplier,

$$-\boldsymbol{\lambda} \cdot \mathcal{D}^2\mathbf{c}. \quad (9.21)$$

Since $\mathbf{c}$ is linear in the centre-of-mass positions, and the orientations of the two bodies enter independently through $\mathbf{r}_i$ and $\mathbf{r}_j$, this term is block diagonal:

$$-\boldsymbol{\lambda} \cdot \mathcal{D}^2\mathbf{c} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & G_i(\boldsymbol{\lambda}) & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & G_j(\boldsymbol{\lambda}) \end{pmatrix}, \quad (9.22)$$

where

$$G_i(\boldsymbol{\lambda}) = -\frac{1}{2} (\boldsymbol{\lambda} \mathbf{r}_i^T + \mathbf{r}_i \boldsymbol{\lambda}^T) + (\boldsymbol{\lambda} \cdot \mathbf{r}_i) \mathbb{1}, \quad (9.23)$$

$$G_j(\boldsymbol{\lambda}) = \frac{1}{2} (\boldsymbol{\lambda} \mathbf{r}_j^T + \mathbf{r}_j \boldsymbol{\lambda}^T) - (\boldsymbol{\lambda} \cdot \mathbf{r}_j) \mathbb{1}. \quad (9.24)$$

These terms come entirely from the curvature of the rotational map

$$\mathbf{r} \rightarrow e^{[\Delta\boldsymbol{\phi}]_{\times}} \mathbf{r} = \mathbf{r} + \Delta\boldsymbol{\phi} \times \mathbf{r} + \frac{1}{2} \Delta\boldsymbol{\phi} \times (\Delta\boldsymbol{\phi} \times \mathbf{r}) + \dots. \quad (9.25)$$

There is no corresponding translational curvature, since the vector constraint is linear in the centre-of-mass positions.

The second-order effective mass matrix is therefore

$$M' = M - (\boldsymbol{\lambda} \cdot \mathcal{D}^2\mathbf{c}) \Delta t^2. \quad (9.26)$$

Equivalently, the two inertia blocks become

$$I'_i = I_i - \Delta t^2 \left[\frac{1}{2} (\boldsymbol{\lambda} \mathbf{r}_i^T + \mathbf{r}_i \boldsymbol{\lambda}^T) - (\boldsymbol{\lambda} \cdot \mathbf{r}_i) \mathbb{1} \right], \quad (9.27)$$

$$I'_j = I_j + \Delta t^2 \left[\frac{1}{2} (\boldsymbol{\lambda} \mathbf{r}_j^T + \mathbf{r}_j \boldsymbol{\lambda}^T) - (\boldsymbol{\lambda} \cdot \mathbf{r}_j) \mathbb{1} \right]. \quad (9.28)$$

We will reuse this result directly in the Augmented Lagrangian and AVBD formulation in section §10.1. There the same contracted constraint Hessian appears, with the hard-constraint multiplier $\boldsymbol{\lambda}$ replaced by the current augmented constraint-force coefficient $\boldsymbol{\lambda} - \rho \mathbf{c}$.

10 Augmented Lagrangian Method

The augmented Lagrangian method [21, 31] combines the constraint penalty method with the method of Lagrange multipliers.

We saw in (5.5) that solving the constrained equations to first order can be interpreted as minimizing

$$\frac{1}{2} \Delta \mathbf{X} \cdot M \Delta \mathbf{X} \quad (10.1)$$

subject to the linearized constraint

$$\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c}\Delta\mathbf{X} = 0. \quad (10.2)$$

Using the same sign convention for the Lagrange multiplier as in the equations of motion, where the generalized constraint force is

$$\mathbf{F}_c = (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda}, \quad (10.3)$$

we can write the corresponding Lagrangian as

$$\frac{1}{2} \Delta \mathbf{X} \cdot M \Delta \mathbf{X} - \boldsymbol{\lambda} \cdot (\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c}\Delta\mathbf{X}) \Delta t^2. \quad (10.4)$$

Stationarity with respect to $\Delta\mathbf{X}$ gives

$$M\Delta\mathbf{X} = (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2, \quad (10.5)$$

consistent with the constrained force equation derived previously.

The augmented Lagrangian method adds the penalty term $\frac{1}{2} \mathbf{c}(\mathbf{X}) \cdot \rho \mathbf{c}(\mathbf{X})$, for a diagonal matrix of penalty coefficients ρ . Retaining the constraint expansion to second order, using the Hessian derived in section §9.1, gives

$$\begin{aligned} \min_{\Delta\mathbf{X}} & \frac{1}{2} \Delta \mathbf{X} \cdot M \Delta \mathbf{X} \\ & - \boldsymbol{\lambda} \cdot \left(\mathbf{c} + \mathcal{D}\mathbf{c}\Delta\mathbf{X} + \frac{1}{2} \Delta \mathbf{X} \cdot \mathcal{D}^2 \mathbf{c} \Delta \mathbf{X} \right) \Delta t^2 \\ & + \frac{1}{2} \left(\mathbf{c} + \mathcal{D}\mathbf{c}\Delta\mathbf{X} + \frac{1}{2} \Delta \mathbf{X} \cdot \mathcal{D}^2 \mathbf{c} \Delta \mathbf{X} \right) \cdot \rho \left(\mathbf{c} + \mathcal{D}\mathbf{c}\Delta\mathbf{X} + \frac{1}{2} \Delta \mathbf{X} \cdot \mathcal{D}^2 \mathbf{c} \Delta \mathbf{X} \right) \Delta t^2, \end{aligned} \quad (10.6)$$

where the constraint quantities on the right hand side are evaluated at $\mathbf{X}^{t+1}$.

Keeping terms through second order in the objective gives a linear equation for the displacement.

The second-order Augmented Lagrangian system is

$$H\Delta\mathbf{X} = (\mathcal{D}\mathbf{c})^T (\boldsymbol{\lambda} - \rho \mathbf{c}(\mathbf{X}^{t+1})) \Delta t^2, \quad (10.7)$$

where

$$H = M + (\mathcal{D}\mathbf{c})^T \rho \Delta t^2 \mathcal{D}\mathbf{c} - (\boldsymbol{\lambda} - \rho \mathbf{c}(\mathbf{X}^{t+1})) \cdot \mathcal{D}^2 \mathbf{c} \Delta t^2. \quad (10.8)$$

The two stiffness contributions have the same interpretation as in section §9.2. The term

$$(\mathcal{D}\mathbf{c})^T \rho \mathcal{D}\mathbf{c} \quad (10.9)$$

is the material-stiffness term arising from the change in force magnitude, while

$$-(\boldsymbol{\lambda} - \rho \mathbf{c}) \cdot \mathcal{D}^2 \mathbf{c} \quad (10.10)$$

is the geometric-stiffness term generated by the current augmented constraint force. Notice that, for $\rho = 0$, this reduces directly to the hard-constraint result from section §9.2,

$$M' = M - \boldsymbol{\lambda} \cdot \mathcal{D}^2 \mathbf{c} \Delta t^2. \quad (10.11)$$

The augmented Lagrangian method solves this iteratively by first fixing $\boldsymbol{\lambda}^k$ and solving for $\Delta \mathbf{X}$, and then updating the Lagrange multiplier.

$$\boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}^k - \rho \mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X}). \quad (10.12)$$

This is then repeated for $k = 1$ to N iterations.

Notice that if the solution converges then $\boldsymbol{\lambda}^{k+1} \sim \boldsymbol{\lambda}^k$, and so $\mathbf{c}(\mathbf{X}^\infty) \sim 0$ and the constraint equations are satisfied.

Using Gauss–Seidel to solve for the displacements, the linearized update is

$$\Delta \mathbf{X}_i^{k+1} = \Delta \mathbf{X}_i^k - \xi H_{ii}^{-1} \left(M \Delta \mathbf{X}^k - (\mathcal{D}\mathbf{c})^T \left(\boldsymbol{\lambda}^k - \rho \mathbf{c}(\mathbf{X}^{t+1}) \right) \Delta t^2 \right)_i. \quad (10.13)$$

This can be rewritten in the same way as section §4.6 as a non-linear Gauss–Seidel update.

Non-linear iteration update for the augmented Lagrangian method:

$$\Delta \mathbf{X}_i^{k+1} = \Delta \mathbf{X}_i^k - \xi H_{ii}^{-1} \left(M \Delta \mathbf{X}^k - (\mathcal{D}\mathbf{c})^T \left(\boldsymbol{\lambda}^k - \rho \mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k) \right) \Delta t^2 \right)_i. \quad (10.14)$$

I.e. we compute the constraint and its derivatives at the current state $\mathbf{X}^{t+1} + \Delta \mathbf{X}^k$. Comparing this to (8.5), this is the force update computed earlier with

$$\mathbf{F}(\mathbf{X}) = (\mathcal{D}\mathbf{c})^T \left(\boldsymbol{\lambda}^k - \rho \mathbf{c}(\mathbf{X}) \right). \quad (10.15)$$

After calculating the displacement update at fixed $\boldsymbol{\lambda}^k$, we update the Lagrange multipliers according to

$$\boldsymbol{\lambda}_j^{k+1} = \boldsymbol{\lambda}_j^k - (\rho \mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X}))_j. \quad (10.16)$$

10.1 Augmented Vertex Block Descent

The method presented above is closely related to the Augmented Vertex Block Descent (AVBD) method [18], but is written using the same sign convention and exponential-map representation of rotation used throughout these notes.

There is one sign-convention difference worth pointing out though. AVBD writes the augmented constraint energy for a scalar constraint as

$$E_{\text{AVBD}} = \frac{1}{2}k c^2 + \lambda_{\text{AVBD}}c, \quad (10.17)$$

with a corresponding constraint force of

$$\mathbf{F}_c = -(\mathcal{D}c)^T (\lambda_{\text{AVBD}} + kc). \quad (10.18)$$

In these notes, λ was defined earlier as the physical Lagrange multiplier appearing directly in the equations of motion,

$$\mathbf{F}_c = (\mathcal{D}c)^T \lambda. \quad (10.19)$$

The two conventions are therefore related by

$$\boxed{\lambda_{\text{AVBD}} = -\lambda} \quad (10.20)$$

with $k = \rho$, so that

$$-(\lambda_{\text{AVBD}} + \rho c) = \lambda - \rho c. \quad (10.21)$$

The multiplier update consequently also has the opposite sign. AVBD writes

$$\lambda_{\text{AVBD}}^{k+1} = \lambda_{\text{AVBD}}^k + \rho c, \quad (10.22)$$

whereas with the convention used here

$$\lambda^{k+1} = \lambda^k - \rho c. \quad (10.23)$$

There is no physical difference between these formulations - the dual variable has just been defined with the opposite sign.

AVBD also allows its penalty coefficient k to change between iterations. In the derivations here ρ is treated as fixed while constructing each local linear system, so at a given AVBD iteration we can identify ρ with the current value of k .

One important feature of the AVBD algorithm is how it handles the geometric-stiffness term. In the notation of AVBD, the local Hessian contribution for a constraint j acting on a body i contains

$$G_{ij} = (\lambda_{\text{AVBD},j} + k_j c_j) \frac{\partial^2 c_j}{\partial \mathbf{X}_i^2}. \quad (10.24)$$

Using (10.21), this is exactly the term

$$-(\lambda_j - \rho_j c_j) \mathcal{D}^2 c_j \quad (10.25)$$

appearing in (10.8).

For each local geometric-stiffness block G_{ij} , AVBD constructs a non-negative diagonal approximation $\widehat{G}_{ij} = \text{diag}(\mathbf{g}_{ij})$, with each diagonal entry given by the norm of the corresponding column,

$$g_{ij,c} = \|G_{ij,c}\|. \quad (10.26)$$

This approximation keeps the local Hessian symmetric positive definite and therefore avoids failures caused by an indefinite geometric-stiffness term.

Another relevant paper with a similar idea is [14]. In this they propose a modification to the PBD Gauss–Seidel update that instead solves per body rather than per constraint, allowing multiple constraints acting on a single body to be resolved together.

10.1.1 AVBD, PBD and Geometric Stiffness

We can now make the connection to PBD. Write

$$M' = M - (\boldsymbol{\lambda} - \rho \mathbf{c}(\mathbf{X}^{t+1})) \cdot \mathcal{D}^2 \mathbf{c} \Delta t^2. \quad (10.27)$$

Then (10.7) can be written

$$(M' + (\mathcal{D}\mathbf{c})^T \rho \Delta t^2 \mathcal{D}\mathbf{c}) \Delta \mathbf{X} = (\mathcal{D}\mathbf{c})^T (\boldsymbol{\lambda} - \rho \mathbf{c}(\mathbf{X}^{t+1})) \Delta t^2. \quad (10.28)$$

Rearranging gives

$$M' \Delta \mathbf{X} = (\mathcal{D}\mathbf{c})^T [\boldsymbol{\lambda} - \rho (\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta \mathbf{X})] \Delta t^2. \quad (10.29)$$

At convergence the linearized constraint is satisfied,

$$\mathbf{c}(\mathbf{X}^{t+1}) + \mathcal{D}\mathbf{c} \Delta \mathbf{X} = 0. \quad (10.30)$$

Equation (10.29) therefore reduces to

$$M' \Delta \mathbf{X} = (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2. \quad (10.31)$$

Multiplying by $\mathcal{D}\mathbf{c} M'^{-1}$ and using $\mathcal{D}\mathbf{c} \Delta \mathbf{X} = -\mathbf{c}(\mathbf{X}^{t+1})$ gives

$$\boxed{\mathcal{D}\mathbf{c} M'^{-1} (\mathcal{D}\mathbf{c})^T \boldsymbol{\lambda} \Delta t^2 = -\mathbf{c}(\mathbf{X}^{t+1})}. \quad (10.32)$$

So the converged linearized equation for the Lagrange multipliers has exactly the same form as the PBD equation, but with

$$M \rightarrow M' = M - (\boldsymbol{\lambda} - \rho \mathbf{c}) \cdot \mathcal{D}^2 \mathbf{c} \Delta t^2. \quad (10.33)$$

The penalty coefficient changes the conditioning and therefore the route taken to the solution, but at convergence the material-stiffness term drops out of the hard-constraint multiplier equation. The remaining difference from ordinary first-order PBD is the geometric-stiffness correction to the metric.

AVBD does not in general use this exact geometric-stiffness term. Instead, the local geometric-stiffness blocks are replaced by the positive diagonal approximation described above. Since this correction is local and block diagonal together with the body mass matrix, it is simple to invert and keeps the local system symmetric positive definite.

For the comparison with PBD there are two important changes in the AVBD formulation:

1. The solve is performed in body space, with a Gauss–Seidel update per body rather than per constraint.
2. The exact geometric-stiffness contribution is replaced by a positive diagonal approximation, keeping the local Hessian symmetric positive definite.

The material-stiffness term

$$(\mathcal{D}\mathbf{c})^T \rho \mathcal{D}\mathbf{c} \quad (10.34)$$

changes the conditioning of the body-space problem, while the exact geometric-stiffness contribution would modify the mass matrix according to

$$M \rightarrow M - (\boldsymbol{\lambda} - \rho\mathbf{c}) \cdot \mathcal{D}^2 \mathbf{c} \Delta t^2. \quad (10.35)$$

Care is needed because this term is not in general positive definite. AVBD replaces it locally by a positive diagonal approximation.

10.2 Convergence Comparison vs. PBD For Large Mass Ratios

It is interesting to compare the matrix being inverted by the augmented Lagrangian method and the force based method of section §8.1 with the constraint-space matrix inverted by PBD.

$$\begin{aligned} \text{PBD:} \quad & A = \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T, \\ \text{AVBD:} \quad & A = M + (\mathcal{D}\mathbf{c})^T \rho \Delta t^2 \mathcal{D}\mathbf{c} - (\boldsymbol{\lambda} - \rho\mathbf{c}(\mathbf{X}^{t+1})) \cdot \mathcal{D}^2 \mathbf{c} \Delta t^2, \\ \text{Force:} \quad & A = M - (\mathcal{D}\mathbf{F})^T \Delta t^2. \end{aligned} \quad (10.36)$$

In PBD the inverse masses are sandwiched between constraint Jacobians, whereas in the body-space solves the masses appear directly on the block diagonal. The two therefore respond to mass ratios and stiffness in opposite ways. PBD struggles when heavy bodies are joined through light ones, because a uniform tension through the light bodies barely changes the constraint error. The body-space solve struggles when light bodies are coupled by stiff penalties. Section §11 makes this precise for a simple chain of jointed rigid bodies.

The benefit of the augmented Lagrangian method is that it combines the Lagrange multiplier method of PBD with a finite constraint penalty. It can therefore represent soft constraints as well as hard constraints. In a pure penalty formulation, a hard constraint requires the penalty stiffness to approach infinity. With the augmented Lagrangian method the multiplier can instead build up the force required to satisfy the constraint while the penalty remains finite.

10.3 Point to Point Constraint

We can now apply the Augmented Lagrangian formulation to the point to point stiff constraint. In section §9.3 we used the vector constraint

$$\mathbf{c}(\mathbf{X}) = \mathbf{p}_i - \mathbf{p}_j = \delta \mathbf{p}, \quad \mathbf{p} = \mathbf{x} + \mathbf{r}(\boldsymbol{\theta}), \quad (10.37)$$

and derived its first and second left-trivialized derivatives. We will reuse those results here rather than deriving the constraint Hessian again.

The first derivative is

$$Q = \mathcal{D}\mathbf{c} = \begin{pmatrix} \mathbb{1} & -[\mathbf{r}_i]_{\times} & -\mathbb{1} & [\mathbf{r}_j]_{\times} \end{pmatrix}. \quad (10.38)$$

For simplicity, take the penalty coefficient to be isotropic, $\rho\mathbb{1}$, and define the current augmented constraint force

$$\mathbf{f}_c = \boldsymbol{\lambda} - \rho\delta\mathbf{p}. \quad (10.39)$$

The generalized force acting on the two bodies is therefore

$$Q^T \mathbf{f}_c. \quad (10.40)$$

The complete second-order Augmented Lagrangian Hessian is

$$H = M + \rho\Delta t^2 Q^T Q - \mathbf{f}_c \cdot \mathcal{D}^2 \mathbf{c} \Delta t^2. \quad (10.41)$$

The first additional term is the material stiffness. The second is exactly the geometric-stiffness term derived in section §9.3, with the replacement

$$\boldsymbol{\lambda} \rightarrow \mathbf{f}_c = \boldsymbol{\lambda} - \rho\delta\mathbf{p}. \quad (10.42)$$

Defining

$$M' = M - \mathbf{f}_c \cdot \mathcal{D}^2 \mathbf{c} \Delta t^2, \quad (10.43)$$

the Hessian can be written

$$H = M' + \rho\Delta t^2 Q^T Q. \quad (10.44)$$

For the vector point-to-point constraint the geometric-stiffness term is block diagonal. Using the result from section §9.3, the corresponding modified inertia tensors are

$$\begin{aligned} I'_i &= I_i - \Delta t^2 \left[\frac{1}{2} (\mathbf{f}_c \mathbf{r}_i^T + \mathbf{r}_i \mathbf{f}_c^T) - (\mathbf{f}_c \cdot \mathbf{r}_i) \mathbb{1} \right], \\ I'_j &= I_j + \Delta t^2 \left[\frac{1}{2} (\mathbf{f}_c \mathbf{r}_j^T + \mathbf{r}_j \mathbf{f}_c^T) - (\mathbf{f}_c \cdot \mathbf{r}_j) \mathbb{1} \right]. \end{aligned} \quad (10.45)$$

The linear blocks of M' are unchanged.

As discussed in section §10.1, the exact geometric-stiffness contribution is not guaranteed to be positive definite. The exact system below can be used provided the resulting M' remains invertible. AVBD instead replaces the local geometric-stiffness blocks by the positive diagonal approximation described above.

10.3.1 Exact Solution of the Local System

For fixed $\boldsymbol{\lambda}$, $\mathbf{c}$, $\mathcal{D}\mathbf{c}$ and $\mathcal{D}^2\mathbf{c}$, the second-order system is

$$H\Delta\mathbf{X} = Q^T \mathbf{f}_c \Delta t^2. \quad (10.46)$$

Although H is a 12×12 matrix, its structure means that the inverse can be reduced to a 3×3 constraint-space solve.

Using the Woodbury matrix identity

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}, \quad (10.47)$$

we have

$$H^{-1} = M'^{-1} - M'^{-1}Q^T \left(\frac{1}{\rho\Delta t^2} \mathbb{1} + QM'^{-1}Q^T \right)^{-1} QM'^{-1}. \quad (10.48)$$

Define the inverse effective mass matrix

$$\mu^{-1} = QM'^{-1}Q^T. \quad (10.49)$$

Since the linear mass blocks are unchanged and the angular blocks are the modified inertias above,

$$\mu^{-1} = \left(\frac{1}{m_i} + \frac{1}{m_j} \right) \mathbb{1} - [\mathbf{r}_i]_{\times} I_i'^{-1} [\mathbf{r}_i]_{\times} - [\mathbf{r}_j]_{\times} I_j'^{-1} [\mathbf{r}_j]_{\times}. \quad (10.50)$$

Using

$$\left(\frac{1}{\rho\Delta t^2} \mathbb{1} + \mu^{-1} \right)^{-1} = \rho\Delta t^2 (\mathbb{1} + \rho\Delta t^2 \mu^{-1})^{-1}, \quad (10.51)$$

the displacement can be written as

$$M'\Delta\mathbf{X} = Q^T (\mathbb{1} + \rho\Delta t^2 \mu^{-1})^{-1} \mathbf{f}_c \Delta t^2. \quad (10.52)$$

Defining

$$\mathbf{f} = (\mathbb{1} + \rho\Delta t^2 \mu^{-1})^{-1} \mathbf{f}_c \Delta t^2, \quad (10.53)$$

we obtain

$$\begin{aligned} \Delta\mathbf{x}_i &= \frac{1}{m_i} \mathbf{f}, \\ \Delta\phi_i &= I_i'^{-1} (\mathbf{r}_i \times \mathbf{f}), \\ \Delta\mathbf{x}_j &= -\frac{1}{m_j} \mathbf{f}, \\ \Delta\phi_j &= -I_j'^{-1} (\mathbf{r}_j \times \mathbf{f}). \end{aligned} \quad (10.54)$$

This has the same overall structure as the first-order point-to-point projection from section §7.1, but there are now two important differences. First, the resistance is the full 3×3 effective mass matrix rather than its scalar projection along $\delta\hat{\mathbf{p}}$. Second, the body inertias are modified by the geometric stiffness generated by the current augmented constraint force.

10.3.2 Body-Block Iterative Solution

AVBD does not invert the complete 12×12 system for each constraint. Instead, it performs a body-block Gauss–Seidel iteration, solving for the six degrees of freedom of one body at a time.

For body i , define

$$Q_i = (\mathcal{D}\mathbf{c})_i = (\mathbb{1} \quad -[\mathbf{r}_i]_{\times}). \quad (10.55)$$

Let M'_i denote the corresponding diagonal body block of (10.43). For the exact geometric-stiffness system its angular block is I'_i from (10.45); for the actual AVBD update this geometric-stiffness block is replaced by its positive diagonal approximation.

The local Hessian block is

$$H_i = M'_i + \rho \Delta t^2 Q_i^T Q_i. \quad (10.56)$$

Again, its inverse can be evaluated using the Woodbury identity:

$$H_i^{-1} = M_i'^{-1} - M_i'^{-1} Q_i^T \left(\frac{1}{\rho \Delta t^2} \mathbb{1} + Q_i M_i'^{-1} Q_i^T \right)^{-1} Q_i M_i'^{-1}. \quad (10.57)$$

Defining the single-body inverse effective mass

$$\mu_i^{-1} = Q_i M_i'^{-1} Q_i^T, \quad (10.58)$$

we have

$$\mu_i^{-1} = \frac{1}{m_i} \mathbb{1} - [\mathbf{r}_i]_{\times} I_i'^{-1} [\mathbf{r}_i]_{\times}. \quad (10.59)$$

Thus each body update requires only the inversion of a 3×3 matrix in addition to the local body metric.

At iteration k , let

$$\mathbf{f}_c^k = \boldsymbol{\lambda}^k - \rho \mathbf{c}(\mathbf{X}^{t+1} + \Delta \mathbf{X}^k). \quad (10.60)$$

The body configuration update is then

$$\Delta \mathbf{X}_i^{k+1} = \Delta \mathbf{X}_i^k - \xi H_i^{-1} \left(M_i \Delta \mathbf{X}_i^k - Q_i^T \mathbf{f}_c^k \Delta t^2 \right). \quad (10.61)$$

After updating body i , the constraint is re-evaluated before solving for body j . The second body therefore sees the newest configuration of the first, which is the usual Gauss–Seidel ordering. The force applied to the two bodies is not necessarily equal and opposite within a single body update, so momentum is not in general conserved per iteration. At convergence, however, the body blocks satisfy the same coupled stationarity equations as the complete augmented system.

For the vector point-to-point constraint the second-order Augmented Lagrangian Hessian has the form

$$H = M' + \rho \Delta t^2 Q^T Q, \quad M' = M - (\boldsymbol{\lambda} - \rho \delta \mathbf{p}) \cdot \mathcal{D}^2 \mathbf{c} \Delta t^2.$$

The material stiffness couples the bodies through the constraint Jacobian, while the geometric stiffness modifies the local body metric. AVBD preserves this structure but, in its local body solve, replaces the potentially indefinite geometric-stiffness blocks by positive diagonal approximations.

Note, the convention in AVBD is $\lambda_{\text{AVBD}} + \rho \delta \mathbf{p}$.

11 Convergence of Constraint Chains

In this section, we can bring together the results from the previous sections and analyze the convergence of a simple chain of constrained rigid bodies using the different methodologies. We will look at how convergence is affected by the number of bodies in the chain and by their mass ratios. For more detail on the matrix iteration results used in this section see, for example, [43].

11.1 Setup

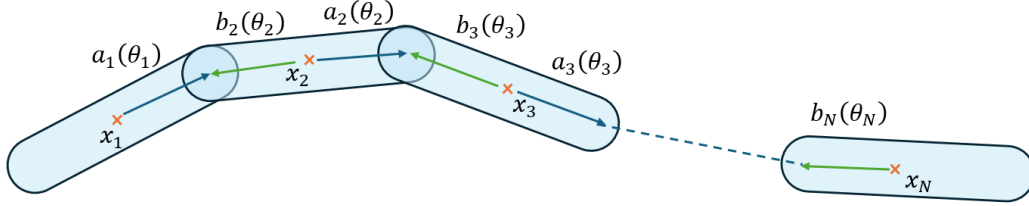


Figure 1: Rigid chain connected by point to point constraints.

The setup is a set of bodies $i = 1, \dots, N$ with point to point constraints between adjacent bodies. For the general rigid body system, the matrix

$$A = \mathcal{D}\mathbf{c} M^{-1} (\mathcal{D}\mathbf{c})^T \quad (11.1)$$

is tridiagonal, since each constraint only couples two neighbouring bodies and adjacent constraints share at most one body.

We will consider the simple case shown in figure 1, where the bodies and constraint directions are all aligned. In this configuration the angular terms vanish and the convergence problem reduces to the corresponding one-dimensional chain of masses. This is the case where corrections to one constraint couple most directly into the neighbouring constraints, and so is a useful simple model for studying convergence.

Let B be the incidence matrix of the chain,

$$B = \begin{pmatrix} 1 & -1 & & & \\ & 1 & -1 & & \\ & & \ddots & \ddots & \\ & & & 1 & -1 \end{pmatrix}, \quad (11.2)$$

so that the linearized constraint error is $B\mathbf{x}$. If

$$M_x = \text{diag}(m_1, \dots, m_N) \quad (11.3)$$

is the translational mass matrix, then the PBD constraint matrix is simply

$$A = B M_x^{-1} B^T. \quad (11.4)$$

Explicitly,

$$A = \begin{pmatrix} \left(\frac{1}{m_1} + \frac{1}{m_2}\right) & -\frac{1}{m_2} & & & \\ -\frac{1}{m_2} & \left(\frac{1}{m_2} + \frac{1}{m_3}\right) & -\frac{1}{m_3} & & \\ & -\frac{1}{m_3} & \ddots & \ddots & \\ & & \ddots & \left(\frac{1}{m_{N-1}} + \frac{1}{m_N}\right) \end{pmatrix}. \quad (11.5)$$

Notice that only the first and last rows are strictly diagonally dominant. For every interior row,

$$\frac{\sum_{j \neq i} |A_{ij}|}{|A_{ii}|} = 1. \quad (11.6)$$

The matrix is nevertheless symmetric positive definite, so Gauss–Seidel converges, but there is no strong diagonal dominance to make that convergence fast.

11.2 Effect of Iteration Count

First assume that all masses are equal. The overall scale of A does not affect the iteration matrix, so we can set $m_i = 1$. Then

$$A = \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & \ddots & \ddots \\ & & \ddots & 2 \end{pmatrix}. \quad (11.7)$$

The Gauss–Seidel iteration matrix is

$$G = -(D + L)^{-1} L^T, \quad (11.8)$$

and for this tridiagonal system its spectral radius is

$$\rho(G) = \cos^2 \left(\frac{\pi}{N} \right). \quad (11.9)$$

For large N ,

$$\rho(G) \sim 1 - \frac{\pi^2}{N^2}. \quad (11.10)$$

So, as the number of bodies in the chain increases, the spectral radius approaches one and convergence becomes progressively slower.

For the dominant error mode,

$$\begin{aligned} |\epsilon^k| &\sim \rho(G)^k |\epsilon^0|, \\ &\sim \cos^{2k} \left(\frac{\pi}{N} \right) |\epsilon^0|, \\ &\sim \exp \left(-\frac{k\pi^2}{N^2} \right) |\epsilon^0| \quad \text{for large } N. \end{aligned} \quad (11.11)$$

So, to reduce the error to $1 - \alpha$ times its original value requires approximately

$$k \sim \frac{N^2}{\pi^2} \ln \left(\frac{1}{1 - \alpha} \right). \quad (11.12)$$

The important result is that the number of local Gauss–Seidel iterations scales as N^2 . This is the familiar slow propagation of long-wavelength errors through a chain.

See figure 2 for an example showing the number of iterations required to remove different fractions of the original error for a chain of length 10.

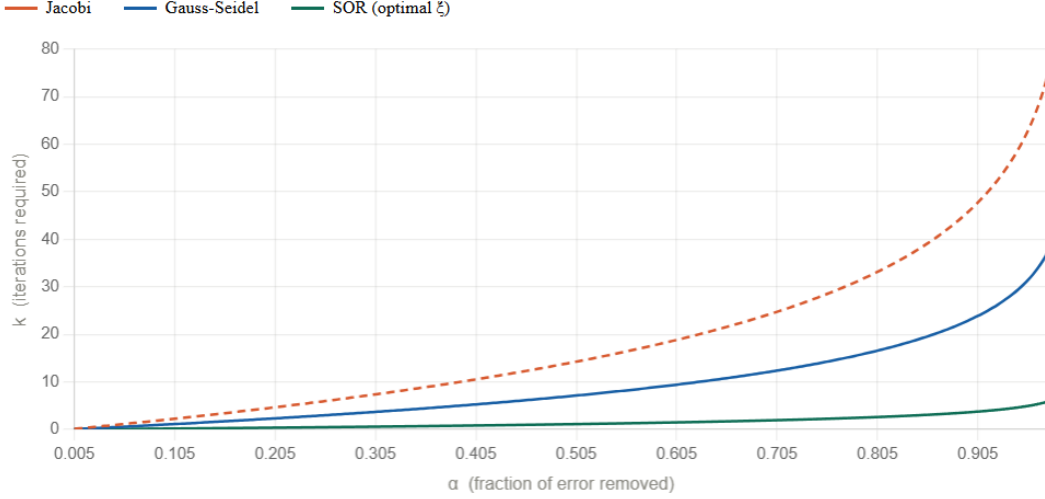


Figure 2: Comparing Gauss–Seidel, Jacobi and Gauss–Seidel with optimal SOR on the number of iterations required to reduce a fraction of the error in a chain of 10 bodies. The optimal relaxation for this case is $\xi^* \sim 1.5$.

11.2.1 Relaxation

The addition of a relaxation factor ξ modifies the iteration matrix to

$$G_\xi = (D + \xi L)^{-1} ((1 - \xi)D - \xi L^T). \quad (11.13)$$

For the equal-mass chain the Jacobi iteration matrix has eigenvalues

$$\mu_k = \cos\left(\frac{k\pi}{N}\right), \quad k = 1, \dots, N-1. \quad (11.14)$$

Since the matrix is consistently ordered and tridiagonal, Young’s result [43] gives the optimal SOR parameter directly as

$$\xi^* = \frac{2}{1 + \sqrt{1 - \rho(G_J)^2}}. \quad (11.15)$$

Using

$$\rho(G_J) = \cos\left(\frac{\pi}{N}\right) \quad (11.16)$$

gives

$$\xi^* = \frac{2}{1 + \sin\left(\frac{\pi}{N}\right)}. \quad (11.17)$$

The corresponding spectral radius is

$$\rho(G_{\xi^*}) = \xi^* - 1 = \frac{1 - \sin\left(\frac{\pi}{N}\right)}{1 + \sin\left(\frac{\pi}{N}\right)}. \quad (11.18)$$

For large N ,

$$\xi^* \sim 2 \left(1 - \frac{\pi}{N}\right), \quad \rho(G_{\xi^*}) \sim 1 - \frac{2\pi}{N}. \quad (11.19)$$

The corresponding iteration count is

$$k \sim \frac{N}{2\pi} \ln \left(\frac{1}{1 - \alpha} \right). \quad (11.20)$$

So optimal SOR changes the asymptotic scaling from N^2 to N .

The practical difficulty with SOR in a general physics solver is that the optimal value depends on the constraint matrix, which changes as the constraint topology, masses and configurations change.

11.2.2 Jacobi Iterations

For comparison, the Jacobi iteration matrix is

$$G_J = -D^{-1}(L + L^T). \quad (11.21)$$

For the straight equal-mass chain,

$$\rho(G_J) = \cos \left(\frac{\pi}{N} \right). \quad (11.22)$$

Young's theorem gives

$$\rho(G) = \rho(G_J)^2, \quad (11.23)$$

so for large N the Jacobi iteration count is

$$k \sim \frac{2N^2}{\pi^2} \ln \left(\frac{1}{1 - \alpha} \right). \quad (11.24)$$

This is twice the asymptotic number of iterations required by Gauss–Seidel, which is one of the reasons Gauss–Seidel is generally preferred.

11.2.3 AVBD Iterations

For the same straight chain, the angular and geometric-stiffness terms vanish in the degrees of freedom being considered, and the body space Hessian reduces to

$$H = M_x + \rho \Delta t^2 B^T B. \quad (11.25)$$

It is useful here to define

$$s = \rho \Delta t^2 \quad (11.26)$$

so that we do not confuse the penalty coefficient with the spectral-radius notation $\rho(G)$.

For equal masses,

$$H = \begin{pmatrix} m+s & -s & & & \\ -s & m+2s & -s & & \\ & -s & m+2s & -s & \\ & & \ddots & \ddots & \ddots \\ & & & -s & m+s \end{pmatrix}. \quad (11.27)$$

The important difference from PBD is that the mass matrix now appears directly on the diagonal. For an interior row of the Jacobi iteration matrix the absolute row sum is

$$\beta = \frac{2s}{m + 2s}, \quad (11.28)$$

while for either end row it is

$$\frac{s}{m + s} < \beta. \quad (11.29)$$

So

$$\rho(G_J) \leq \beta, \quad \rho(G) \leq \beta^2. \quad (11.30)$$

For any fixed finite penalty, $\beta < 1$, so the convergence rate of the inner body-space sweep is bounded independently of the length of the chain. Using the spectral-radius bound gives the characteristic iteration count

$$k \sim \frac{\ln(1/(1 - \alpha))}{-2 \ln \beta}. \quad (11.31)$$

In particular,

$$k = O(1) \quad \text{for fixed } \beta < 1 \text{ as } N \rightarrow \infty. \quad (11.32)$$

This makes sense physically. If the penalties are soft compared with the body masses, moving one body has only a weak effect on its neighbours, and a local body update does not need to propagate a large correction along the whole chain.

There is an important qualification though. This is only the convergence of the *inner* body-space solve at fixed λ . The Augmented Lagrangian method also updates the multipliers,

$$\lambda^{k+1} = \lambda^k - \rho \mathbf{c}^{k+1}, \quad (11.33)$$

and this separate iteration has its own convergence rate.

For the linear problem, assuming the body-space system is solved exactly for each fixed multiplier, one multiplier update contracts the constraint error by

$$\mathbf{c}^{k+1} = (\mathbb{1} + sA)^{-1} \mathbf{c}^k, \quad A = BM_x^{-1} B^T. \quad (11.34)$$

For the equal-mass chain,

$$\lambda_{\min}(A) = \frac{2 - 2 \cos(\pi/N)}{m} \sim \frac{\pi^2}{mN^2}. \quad (11.35)$$

The slowest multiplier mode therefore contracts by

$$q_{\text{out}} = \frac{1}{1 + s\lambda_{\min}(A)} \sim 1 - \frac{\pi^2 s}{mN^2} \quad (11.36)$$

when $s\lambda_{\min}(A) \ll 1$. The corresponding number of outer updates is

$$k_{\text{out}} \sim \frac{mN^2}{\pi^2 s} \ln \left(\frac{1}{1 - \alpha} \right). \quad (11.37)$$

So reducing the penalty makes the inner body-space sweep better conditioned, but makes the multiplier update slower. Increasing the penalty has the opposite effect. The long wavelength mode has not disappeared, the Augmented Lagrangian formulation just changes where that slow mode appears.

In practice, AVBD interleaves the body updates and multiplier updates rather than converging one complete loop before starting the other, and can also change the penalty during the solve. So the separate rates above are useful for understanding the conditioning of the two parts of the algorithm, but they should not be interpreted as a complete convergence analysis of the interleaved AVBD iteration.

11.3 Effect of Mass Ratios

So far we have assumed that all masses are equal. Now let the interior masses be

$$m_i = m, \quad i = 2, \dots, N-1, \quad (11.38)$$

and make the two end masses

$$m_1 = m_N = M, \quad M > m. \quad (11.39)$$

Define

$$\gamma = \frac{m}{M}. \quad (11.40)$$

For PBD the first and last rows of the constraint-space matrix have diagonal-dominance ratio

$$r_{\text{ends}} = \frac{1/m}{1/m + 1/M} = \frac{1}{1 + \gamma}. \quad (11.41)$$

As M/m becomes large, $r_{\text{ends}} \rightarrow 1$ and all of the rows of the matrix become only weakly diagonally dominant.

Setting the common interior mass $m = 1$ for the eigenvalue analysis, the matrix is

$$A = \begin{pmatrix} 1 + \gamma & -1 & & \\ -1 & 2 & -1 & \\ & -1 & \ddots & \ddots \\ & & \ddots & 1 + \gamma \end{pmatrix}. \quad (11.42)$$

If μ is an eigenvalue of the Jacobi iteration matrix, then

$$A\mathbf{x} = (1 - \mu)D\mathbf{x}. \quad (11.43)$$

For $\gamma = 0$, the vector

$$\mathbf{v} = (1 \quad 1 \quad \dots \quad 1)^T \quad (11.44)$$

is an exact null vector. For small γ this becomes the slow mode. Using $\mathbf{v}$ in the generalized Rayleigh quotient gives, to first order in γ ,

$$\begin{aligned} 1 - \mu &\sim \frac{\mathbf{v}^T A \mathbf{v}}{\mathbf{v}^T D \mathbf{v}}, \\ &= \frac{2\gamma}{2(N-2 + \gamma)}, \\ &\sim \frac{\gamma}{N-2}. \end{aligned} \quad (11.45)$$

Therefore

$$\rho(G_J) \sim 1 - \frac{\gamma}{N-2}, \quad (11.46)$$

and

$$\rho(G) = \rho(G_J)^2 \sim 1 - \frac{2\gamma}{N-2}. \quad (11.47)$$

The number of Gauss–Seidel iterations required to reduce the error by a factor $1 - \alpha$ is therefore

$$k \sim \frac{N-2}{2\gamma} \ln \left(\frac{1}{1-\alpha} \right). \quad (11.48)$$

So the convergence is directly proportional to the mass ratio $M/m = 1/\gamma$. For example, with $N = 10$ and $M/m = 100$ the coefficient is approximately

$$\frac{N-2}{2\gamma} = 400, \quad (11.49)$$

compared with $N^2/\pi^2 \simeq 10$ for the equal-mass chain. So in this example the heavy end masses increase the required iteration count by roughly a factor of 40.

11.3.1 Relaxation

The same SOR result can be applied to this slow mode. Since

$$\rho(G) \sim 1 - \frac{2\gamma}{N-2}, \quad (11.50)$$

the optimal relaxation parameter is approximately

$$\xi^* = \frac{2}{1 + \sqrt{1 - \rho(G)}} \sim \frac{2}{1 + \sqrt{2\gamma/(N-2)}}. \quad (11.51)$$

The corresponding spectral radius is

$$\rho(G_{\xi^*}) \sim 1 - 2\sqrt{\frac{2\gamma}{N-2}}, \quad (11.52)$$

and hence

$$k \sim \frac{1}{2} \sqrt{\frac{N-2}{2\gamma}} \ln \left(\frac{1}{1-\alpha} \right). \quad (11.53)$$

So optimal SOR changes the mass-ratio scaling from

$$O\left(\frac{N}{\gamma}\right) \quad (11.54)$$

to

$$O\left(\sqrt{\frac{N}{\gamma}}\right). \quad (11.55)$$

For $N = 10$ and $M/m = 100$, the coefficient falls from approximately 400 to approximately 10.

11.3.2 AVBD Iterations

For the AVBD body-space solve, the same heavy end masses give

$$H = \begin{pmatrix} M+s & -s & & & \\ -s & m+2s & -s & & \\ & -s & m+2s & -s & \\ & & \ddots & \ddots & \ddots \\ & & & -s & M+s \end{pmatrix}. \quad (11.56)$$

The end-row Jacobi ratio is

$$r_{\text{ends}} = \frac{s}{M+s}, \quad (11.57)$$

whereas the interior ratio remains

$$\beta = \frac{2s}{m+2s}. \quad (11.58)$$

Increasing the end masses therefore improves the diagonal dominance of the body-space system, which is the opposite behaviour to the PBD constraint-space solve.

For $M > m/2$,

$$\frac{s}{M+s} < \beta, \quad (11.59)$$

so the Jacobi and Gauss–Seidel spectral radii obey

$$\rho(G_J) \leq \beta, \quad \rho(G) \leq \beta^2 \quad (11.60)$$

independently of both M and N .

We can also look at the limit $M \rightarrow \infty$. The Jacobi eigenproblem for an interior body is

$$x_{i-1} + x_{i+1} = \frac{2\mu}{\beta} x_i, \quad (11.61)$$

so the interior modes have the form

$$x_i = \sin(i\theta + \phi), \quad \mu = \beta \cos \theta. \quad (11.62)$$

The end equations are

$$x_1 = \frac{s}{\mu(M+s)} x_2, \quad x_N = \frac{s}{\mu(M+s)} x_{N-1}. \quad (11.63)$$

As $M \rightarrow \infty$,

$$x_1, x_N \rightarrow 0, \quad (11.64)$$

so the heavy end bodies behave as fixed anchors and the $N-2$ interior bodies form a chain with fixed ends. Therefore

$$\rho(G_J) \rightarrow \beta \cos \left(\frac{\pi}{N-1} \right), \quad \rho(G) \rightarrow \beta^2 \cos^2 \left(\frac{\pi}{N-1} \right). \quad (11.65)$$

This shows that heavy end masses that produce an increasingly slow mode in the PBD constraint-space iteration do not produce the same problem in the AVBD body-space sweep. In the limit they become fixed boundary conditions.

The situation reverses for a very light body. If

$$M \ll s, \quad (11.66)$$

then

$$\frac{s}{M+s} \rightarrow 1, \quad (11.67)$$

and the diagonal-dominance bound is lost. A light body coupled by a stiff penalty is therefore the difficult case for the body-space iteration, as anticipated in section §10.2.

There is still one important caveat. The outer multiplier update depends on

$$A = BM_x^{-1}B^T, \quad (11.68)$$

so it inherits the mass-ratio dependence of the constraint-space problem.

For very heavy end masses, $\gamma = m/M \ll 1$, the smallest eigenvalue of A is obtained to first order by using the nearly constant mode,

$$\lambda_{\min}(A) \sim \frac{2\gamma}{m(N-1)}. \quad (11.69)$$

The slowest outer multiplier mode therefore contracts by

$$q_{\text{out}} \sim \left(1 + \frac{2s\gamma}{m(N-1)}\right)^{-1}. \quad (11.70)$$

When the quantity in the denominator is small,

$$q_{\text{out}} \sim 1 - \frac{2s\gamma}{m(N-1)}, \quad (11.71)$$

and hence

$$k_{\text{out}} \sim \frac{m(N-1)}{2s\gamma} \ln \left(\frac{1}{1-\alpha} \right). \quad (11.72)$$

So the heavy end masses are largely invisible to the body-space sweep, but they are not invisible to the multiplier iteration.

This illustrates the difference between the primal and dual formulations - changing from a constraint-space to a body-space solve changes where the difficult modes appear, rather than necessarily removing them from the complete Augmented Lagrangian iteration.

11.4 Results

These results are for the simplified straight chain, where the angular terms vanish and the convergence problem reduces to the corresponding one-dimensional mass chain.

For an equal-mass hard constraint chain, the PBD Gauss–Seidel iteration requires $O(N^2)$ iterations to remove a fixed fraction of the error. Jacobi has the same scaling with approximately twice the iteration count, while optimal SOR improves the scaling to $O(N)$.

For the AVBD body-space system with a fixed finite penalty,

$$\beta = \frac{2\rho\Delta t^2}{m + 2\rho\Delta t^2} < 1,$$

the inner Gauss–Seidel sweep has

$$\rho(G) \leq \beta^2,$$

so its convergence is bounded independently of the chain length. The outer multiplier iteration still contains the constraint-space matrix

$$A = \mathcal{D}\mathbf{c}M^{-1}(\mathcal{D}\mathbf{c})^T,$$

and therefore retains the long-wavelength modes of the chain. Softening the penalty improves the conditioning of the inner body-space solve but slows the multiplier update.

Large masses at the ends of the chain make the PBD constraint-space solve much worse. For $\gamma = m/M \ll 1$,

$$k_{\text{PBD}} \sim \frac{N-2}{2\gamma} \ln \left(\frac{1}{1-\alpha} \right).$$

Optimal SOR improves this to a $\sqrt{N/\gamma}$ scaling.

The AVBD inner body-space sweep behaves in the opposite way: heavy end bodies improve its diagonal dominance and, as $M \rightarrow \infty$, simply act as fixed anchors. Light bodies coupled by stiff penalties are instead the difficult case for the inner body-space solve.

The outer multiplier iteration still depends on the constraint-space matrix, so sufficiently heavy end bodies also slow that part of the Augmented Lagrangian iteration. The body-space formulation therefore changes the conditioning of the local sweep, but does not remove all of the slow modes from the complete coupled algorithm.

The actual AVBD algorithm interleaves the body and multiplier updates, so these separate inner and outer analyses should be viewed as explaining where the difficult modes arise rather than as a complete convergence bound for the interleaved solver.

12 Closing Remarks and Further Reading

The aim of these notes has been to understand position-based rigid-body methods from the underlying mechanics, rather than to treat the algorithms as a collection of independent techniques, and to give a careful treatment for the angular degrees of freedom. Starting from constrained Lagrangian mechanics, we saw how a first-order approximation to the constrained equations of motion, solved iteratively using Gauss–Seidel, leads naturally to the PBD algorithm. We also saw how other approaches relate to this: Sequential Impulses use essentially the same iterative structure at the velocity level, while the PBD update can also be viewed as a mass-weighted projection. XPBD introduces constraint compliance, which can also be understood as a form of regularization, and supplies a material-stiffness term, and going to a second-order expansion introduces the geometric stiffness. These same terms appear again in VBD and AVBD, which, while solving similar constrained dynamics problems, use body-based rather than constraint-based updates, and consequently have different convergence characteristics in different regimes. We showed this explicitly for a simple chain example.

Good references for further information on PBD, XPBD and AVBD are [28], [25] and [18], respectively. These original papers are the best places to go for more detail on the algorithms themselves, their implementation, and their numerical results. For rigid-body PBD in particular, [29] gives a much more implementation-focused treatment than I have attempted here.

For a much more complete treatment of discrete variational mechanics, Marsden and West [27] is an excellent reference, including the extension of these ideas to Lie groups and the geometric structure preserved by variational integrators.

If, like me, you are interested in the mathematical physics behind these ideas, I would also recommend David Tong’s lecture notes on classical dynamics [39]. I will also point out a really interesting paper by Shapere and Wilczek [34], which connects the motion of deformable bodies to the mathematics of gauge theory. This is somewhat removed from the rigid-body simulation considered here, but is a nice example of how the geometric ideas behind rotations and configuration spaces extend into more general mechanical systems.

If I were to continue these notes, the next obvious direction would be to apply the same approach to reduced-coordinate systems for articulated rigid bodies. Featherstone [17] gives the standard treatment of this subject. It is an interesting extension to develop the position-based Lagrangian viewpoint used here directly in reduced coordinates, rather than representing joints as constraints between bodies in maximal coordinates. The focus of these notes, however, was deliberately kept to maximal-coordinate rigid-body methods, and I had to stop somewhere.

References

- [1] Hans C. Andersen. Rattle: A “velocity” version of the shake algorithm for molecular dynamics calculations. *Journal of Computational Physics*, 52(1):24–34, 1983.
- [2] Sheldon Andrews, Marek Teichmann, and Paul G. Kry. Geometric stiffness for real-time constrained multibody dynamics. *Computer Graphics Forum*, 36(2):235–246, 2017.
- [3] David Baraff. An introduction to physically based modeling: Rigid body simulation. *SIG-GRAPH Course Notes*, 1997.
- [4] Joao Barata and Mahir Hussein. The moore-penrose pseudoinverse. a tutorial review of the theory. *Brazilian Journal of Physics*, 42, 10 2011.
- [5] Timothy D. Barfoot. *State Estimation for Robotics*. Cambridge University Press, Cambridge, UK, 2017.
- [6] Joachim Baumgarte. Stabilization of constraints and integrals of motion in dynamical systems. *Computer Methods in Applied Mechanics and Engineering*, 1(1):1–16, 1972.
- [7] Jan Bender, Matthias Müller, and Miles Macklin. A survey on position based dynamics, 2017. In *Eurographics 2017 – Tutorials*. The Eurographics Association, 2017.
- [8] Dennis S. Bernstein, Ankit Goel, and Omran Kouba. Deriving euler’s equation for rigid-body rotation via lagrangian dynamics with generalized coordinates. *Mathematics*, 11(12):2727, June 2023.
- [9] Anthony M. Bloch. *Nonholonomic Mechanics and Control*, volume 24 of *Interdisciplinary Applied Mathematics*. Springer, New York, 2003.
- [10] Erin Catto. Iterative dynamics with temporal coherence. In *Game Developers Conference (GDC)*, 2005.
- [11] Erin Catto. Contact manifolds. In *Game Developers Conference*, 2007.
- [12] Erin Catto. Numerical methods. Game Developers Conference, 2015.
- [13] Anka He Chen, Ziheng Liu, Yin Yang, and Cem Yuksel. Vertex block descent. *ACM Transactions on Graphics*, 43(4):1–16, 2024.
- [14] Yizhou Chen, Yushan Han, Jingyu Chen, Zhan Zhang, Alex McAdams, and Joseph Teran. Position-based nonlinear gauss-seidel for quasistatic hyperelasticity. *ACM Transactions on Graphics*, 43(4):115:1–115:15, 2024.
- [15] Crispin Deul, Patrick Charrier, and Jan Bender. Position-based rigid-body dynamics. *Computer Animation and Virtual Worlds*, 27(2):103–112, 2016.
- [16] Chris Doran and Anthony Lasenby. *Geometric Algebra for Physicists*. Cambridge University Press, Cambridge, UK, 2003.
- [17] Roy Featherstone. *Rigid Body Dynamics Algorithms*. Springer, 2008.
- [18] Chris Giles, Elie Diaz, and Cem Yuksel. Augmented vertex block descent. *ACM Transactions on Graphics*, 44(4):1–12, July 2025.

- [19] Herbert Goldstein, Charles P. Poole, and John L. Safko. *Classical Mechanics*. Addison-Wesley, San Francisco, 3 edition, 2002.
- [20] David Hestenes. *New Foundations for Classical Mechanics*, volume 15 of *Fundamental Theories of Physics*. Reidel / Kluwer Academic Publishers, Dordrecht, Netherlands, 1986. Second revised edition published 1999, ISBN 978-0792355144.
- [21] Magnus R. Hestenes. Multiplier and gradient methods. *Journal of Optimization Theory and Applications*, 4(5):303–320, 1969.
- [22] Thomas Jakobsen. Advanced character physics. In *Game Developers Conference 2001 Proceedings*, pages 383–401, 2001.
- [23] Claude Lacoursière. Stabilizing gyroscopic forces in rigid multibody simulations. Technical Report UMINF 06.05, Department of Computing Science, Umeå University, Umeå, Sweden, 2006.
- [24] Miles Macklin, Kenny Erleben, Matthias Müller, Nuttapong Chentanez, Stefan Jeschke, and Tae-Yong Kim. Primal/dual descent methods for dynamics. *Computer Graphics Forum*, 39(8):89–100, 2020.
- [25] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. Xpbd: Position-based simulation of compliant constrained dynamics. *ACM SIGGRAPH Motion in Games*, 2016.
- [26] Miles Macklin, Kier Storey, Michelle Lu, Pierre Terdiman, Nuttapong Chentanez, Stefan Jeschke, and Matthias Müller. Small steps in physics simulation. In *Proceedings of the 18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation, SCA '19*, Los Angeles, CA, USA, 2019. ACM.
- [27] J. E. Marsden and M. West. Discrete mechanics and variational integrators. *Acta Numerica*, 10:357–514, 2001.
- [28] Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. Position based dynamics. In *Proceedings of the 3rd Workshop on Virtual Reality Interaction and Physical Simulation (VRIPHYS)*, pages 71–80. The Eurographics Association, 2006.
- [29] Matthias Müller, Miles Macklin, Nuttapong Chentanez, Stefan Jeschke, and Tae-Yong Kim. Detailed rigid body simulation with extended position based dynamics. *Computer Graphics Forum*, 39(8):101–112, 2020. Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA 2020).
- [30] Roger Penrose. A generalized inverse for matrices. *Proceedings of the Cambridge Philosophical Society*, 51:406–413, 01 1955.
- [31] M. J. D. Powell. A method for nonlinear constraints in minimization problems. *Optimization*, pages 283–298, 1969.
- [32] Xavier Provot. Deformation constraints in a mass-spring model to describe rigid cloth behavior. In *Proceedings of Graphics Interface 1995*, pages 147–154, 1995.
- [33] J.-P. Ryckaert, G. Ciccotti, and H. J. C. Berendsen. Numerical integration of the cartesian equations of motion of a system with constraints: molecular dynamics of n -alkanes. *J. Comput. Phys.*, 23:327–341, 1977.

- [34] Alfred Shapere and Frank Wilczek. Gauge kinematics of deformable bodies. *American Journal of Physics*, 57(6):514–518, 1989.
- [35] Morten Silcowitz, Sarah Niebe, and Kenny Erleben. Projected gauss–seidel subspace minimization method for interactive rigid body dynamics. In *Proceedings of the International Conference on Computer Graphics Theory and Applications (GRAPP)*. SciTePress, 2010.
- [36] Joan Solà, Jérémie Deray, and Dinesh Atchuthan. A micro Lie theory for state estimation in robotics, 2018.
- [37] David E. Stewart. Rigid-body dynamics with friction and impact. *SIAM Review*, 42(1):3–39, 2000.
- [38] A Tikhonov and V Arsenin. *Solution of Ill-Posed Problem*. 01 1977.
- [39] David Tong. Rigid body dynamics. <https://www.damtp.cam.ac.uk/user/tong/dynamics/three.pdf>, 2019. Lecture notes, Department of Applied Mathematics and Theoretical Physics, University of Cambridge.
- [40] Richard Tonge, Feodor Benevolenski, and Andrey Voroshilov. Mass splitting for jitter-free parallel rigid body simulation. *ACM Transactions on Graphics*, 31(4):105:1–105:8, 2012.
- [41] Maxime Tournier, Matthieu Nesme, Benjamin Gilles, and François Faure. Stable constrained dynamics. *ACM Transactions on Graphics*, 34(4):132:1–132:10, 2015. Proceedings of ACM SIGGRAPH 2015.
- [42] Lloyd N. Trefethen and David Bau III. *Numerical Linear Algebra*. SIAM, Philadelphia, PA, USA, 1997.
- [43] Richard S. Varga. *Matrix Iterative Analysis*. Springer, New York, 2 edition, 2000.
- [44] Anthony Zee. *Group Theory in a Nutshell for Physicists*. In a Nutshell. Princeton University Press, 2016.